



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 1

Introduction to C Programming

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3 - 4
	1.1 Overview in C	-	-	
	1.2 History and evolution of the C programming	-	-	
	1.3 Objectives	-	-	
2	Features of C and its Basic Structure	1	1, 2	5 - 10
	2.1 Programming Style:	-	-	
	2.2 Simple C Programs	-	-	
3	Summary	-	-	11
4	Terminal Questions	-	-	12
5	Answers to Self Assessment Questions	-	-	12
6	Answers to Terminal Questions	-	-	12
7	Exercises	-	-	12

1. INTRODUCTION

1.1. Overview in C

C is a general-purpose, structured programming language. Its instructions consist of terms that resemble algebraic expressions, augmented by certain English keywords such as if, else, for, do and while.

1.2. History and evolution of the C programming

C was the offspring of the 'Basic Combined Programming Language' (BPCL) called B, developed in the 1960's at Cambridge University. B language was modified by Dennis Ritchie and was implemented at Bell laboratories in 1972. The new language was named C. Since it was developed along with the UNIX operating system, it is strongly associated with UNIX. This operating system, which was also developed at Bell laboratories, was coded almost entirely in C. In C, the type of a variable determines what kinds of values it may take on. The type of an object determines the set of values it can have and what operations can be performed on it. This is a fairly formal, mathematical definition of what a type is, but it is traditional (and meaningful). There are several implications to remember:

1. The "set of values" is finite. C's int type cannot represent all of the integers; its float type cannot represent all floating-point numbers.
2. When you're using an object (that is, a variable) of some type, you may have to remember what values it can take on and what operations you can perform on it. For example, there are several operators which play with the binary (bit-level) representation of integers, but these operators are not meaningful for and may not be applied to floating-point operands.
3. When declaring a new variable and picking a type for it, you have to keep in mind the values and operations you'll be needing.

This unit will introduce you to C programming with its various features, structure and how the C programs are constructed with simple examples.

1.3. Objectives:

At studying this unit, you should be able to:

- ❖ *Discuss the features of C programming language*
- ❖ *Explain the basic structure of a C program*



2. FEATURES OF C AND ITS BASIC STRUCTURE

C is characterized by the ability to write very concise source programs, due in part to the large number of operators included within the language.

It has a relatively small instruction set, though actual implementations include extensive library functions which enhance the basic instructions.

The language encourages users to write additional library functions of their own. Thus, the features and capabilities of the language can easily be extended by the user.

C compilers are commonly available for computers of all sizes. The compilers are usually compact, and they generate object programs that are small and highly efficient when compared with programs compiled from other high-level languages.

Another important characteristic of C is that its programs are highly portable, even more so than with other high-level languages. The reason for this is that C relegates most computer dependent features to its library functions. Thus, every version of C is accompanied by its own set of library functions, which are written for the particular characteristics of the host computer.

A C program can be viewed as a group of building blocks called functions. A function is a subroutine that may include one or more statements designed to perform a specific task. To write a C program we first create functions and then put them together. A C program may contain one or more sections shown in Fig. 1.1.

The documentation section consists of a set of comment (remarks) lines giving the name of the program, the author and other details which the programmer would like to use later. Comments may appear anywhere within a program, as long as they are placed within the delimiters `/*` and `*/` (e.g., `/*this is a comment*/`). Such comments are helpful in identifying the program's principal features or in explaining the underlying logic of various program features.

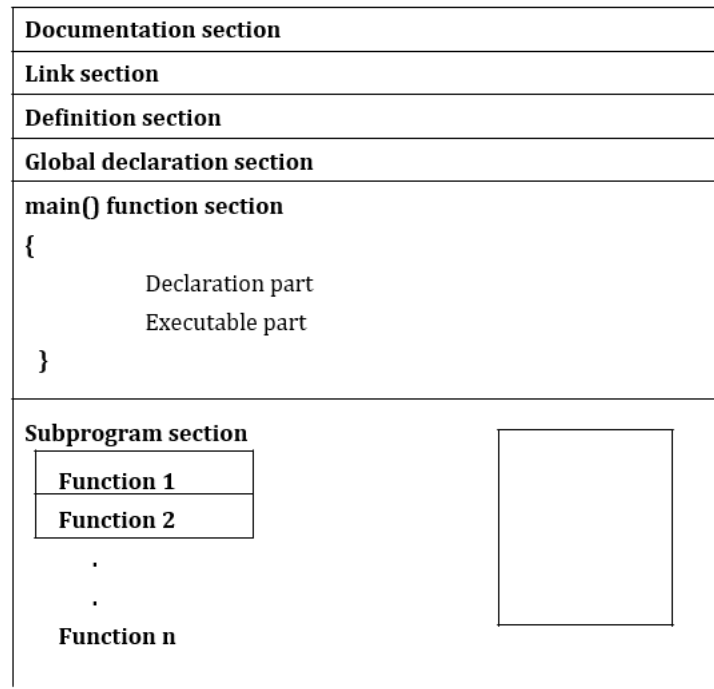


Figure 1.1: Sections in a C program

The link section provides instructions to the compiler to link functions from the system library. The definition section defines all symbolic constants.

There are some variables that are used in more than one function. Such variables are called *global* variables and are declared in the global declaration section that is outside of all the functions.

Every C program must have one **main** function section. This section contains two parts, declaration part and executable part. The declaration part declares all the variables used in the executable part. There is at least one statement in the executable part. These two parts must appear between opening and closing braces ({ and }). The program execution begins at the opening brace and ends at the closing brace. The closing brace of the main function section is the logical end of the program. All statements in the declaration and executable parts end with a semicolon(;).

The subprogram section contains all the user-defined functions that are called in the main function. User-defined functions are generally placed immediately after the **main** function, although they may appear in any order.

All sections, except the **main** function section may be absent when they are not required.

2.1. Programming Style:

Programming style in C involves writing code in a readable and consistent manner. This includes choosing meaningful variable and function names, using indentation for clarity, and following a consistent coding convention. Adopting a good programming style enhances code maintainability and makes it easier for others to understand and collaborate on the codebase.

SELF-ASSESSMENT QUESTIONS - 1

1. Using C language programmers can write their own library functions. (True/False)
2. C is a _____ level programming language.
3. The documentation section contains a set of _____ lines.
4. Every C program must have one **main()** function. (True/False)

2.2. Simple C Programs

Let us see a simple c program given below:

```
#include <stdio.h>
main()
{
printf("Hello, world!\n");
return 0;
}
```

2.3. Executing a C Program:

To execute a C program, it undergoes a compilation process that translates the human-readable source code into machine-readable object code. The resulting executable file can then be run on a computer. Various compilers, such as GCC (GNU Compiler Collection), are commonly used to compile C programs.

If you have a C compiler, the first thing to do is figure out how to type this program in and compile it and run it and see where its output went.

The first line is practically boilerplate; it will appear in almost all programs we write. It asks that some definitions having to do with the "Standard I/O Library" be included in our program; these definitions are needed if we are to call the library function **printf** correctly.

The second line says that we are defining a function named **main**. Most of the time, we can name our functions anything we want, but the function name **main** is special: it is the function that will be "called" first when our program starts running. The empty pair of parentheses indicates that our main function accepts no *arguments*, that is, there isn't any information which needs to be passed in when the function is called.

The braces { and } surround a list of statements in C. Here, they surround the list of statements making up the function **main**.

The line **printf("Hello, world!\n");** is the first statement in the program. It asks that the function **printf** be called; **printf** is a library function which prints formatted output. The parentheses surround **printf's** argument list: the information which is handed to it which it should act on. The semicolon at the end of the line terminates the statement.

printf's first (and, in this case, only) argument is the string which it should print. The string, enclosed in double quotes (""), consists of the words "Hello, world!" followed by a special sequence: `\n`. In strings, any two-character sequence beginning with the backslash `\` represents a single special character. The sequence `\n` represents the " 'new line' " character, which prints a carriage return or line feed or whatever it takes to end one line of output and

move down to the next. (This program only prints one line of output, but it's still important to terminate it.)

The second line in the **main** function is

return 0;

In general, a function may return a value to its caller, and **main** is no exception. When **main** returns (that is, reaches its end and stops functioning), the program is at its end, and the return value from **main** tells the operating system (or whatever invoked the program that **main** is the main function of) whether it succeeded or not. By convention, a return value of 0 indicates success.

Program: Area of a circle

Here is an elementary C program that reads in the radius of a circle, calculates the area and then writes the calculated result.

```
#include <stdio.h>                                /* Library file access */
/* program to calculate the area of a circle
*/                                                  /* Title (Comment) */
main()                                              /* Function heading */

float radius, area;                                /*Variable declarations */
printf("Radius=?");                                /* Output statement(prompt) */
scanf("%f", &radius);                              /* Input statement */
area=3.14159*radius*radius;                         /* Assignment statement */
printf("Area=%f",area);                             /* Output statement */
```

Program: Print a few numbers

Here is a program to illustrate a simple loop

```
#include <stdio.h>
/* print a few numbers, to illustrate a simple loop */
main()
{
    int i;
    for(i = 0; i < 10; i = i + 1) /* Looping statement */ printf("i is %d\n", i);

    return 0;
}
```

Program: Program to add two numbers

```
#include <stdio.h>
main()
{
    int i,j,k; // Defining variables
    i = 6;    // Assign values
    j = 8;
    k = i + j;
    printf("sum of two numbers is %d \n",k); // Printing results
}
```

SELF-ASSESSMENT QUESTIONS - 2

5. The information that needs to be passed in when a function is called is_____.
6. The main() function doesn't return any value. (True/False)

3. SUMMARY

C is a pivotal programming language with a rich history and evolution that traces back to its development in the early 1970s by Dennis Ritchie at Bell Labs. Serving as the successor to the B programming language, C became integral to the creation of the Unix operating system and influenced the development of numerous other programming languages. The standardization of C occurred with the establishment of the ANSI C standard in 1989 and the subsequent ISO C standard in 1999.

The basic structure of a C program is characterized by a preprocessor directive, function declarations, the main function, and additional user-defined functions. Programs are organized into modular functions, each tasked with specific responsibilities. Syntax rules dictate declarations and statements within functions.

Programming style in C emphasizes readability and consistency. Adopting meaningful variable and function names, using indentation, and adhering to coding conventions contribute to a more maintainable and understandable codebase. A well-defined programming style facilitates collaboration and code comprehension.

To execute a C program, a compilation process is employed. This process translates human-readable source code into machine-readable object code, resulting in an executable file. Popular compilers like GCC play a crucial role in compiling C programs. Overall, C programming offers a robust foundation for software development, and understanding its basic structure, adhering to programming style guidelines, and navigating the compilation and execution process are essential aspects of effective C programming.

4. TERMINAL QUESTIONS

1. Explain the history of C language.
2. What are the advantages of C language?
3. What are the major components of a C program?
4. What significance is attached to the function main?

5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. True
2. High
3. Comment
4. True
5. Arguments
6. False

6. ANSWERS TO TERMINAL QUESTIONS

1. C was developed by Dennis Ritchie and came after B in the year 1972. (Refer Section 1.1 for more details)
2. C is fast, efficient and structured. (Refer to section 1.2 for more details)
3. Documentation section, Link section, Definition section, Global declaration section, main() function section, Subprogram section
4. main() is the function that will be "called" first when our program starts running.

7. EXERCISES

1. Explain the basic structure of a C program with an example.
2. What are the different steps in executing a C program?
3. Write a C program to convert Celsius to Fahrenheit and vice versa.
4. Write a program in C to add, subtract, multiply any 3 numbers.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 2

Basics of C Programming

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Data types in C	1, 2, 3	1, 2, 3, 4	6 - 16
2.1	Floating-point Numbers	-	-	
2.2	Program to calculate the average of N numbers	-	-	
2.3	Program to compute the roots of a quadratic equation	-	-	
2.4	Converting Integers to Floating-point and vice-versa	-	-	
2.5	Mixed-mode Expressions	-	-	
2.6	The type cast Operator	-	-	
2.7	The type char	-	-	
2.8	Keywords	-	-	
3	Concept of an Integer and Variable	4,	5, 6	17 - 21
3.1	Rules for naming Variables and assigning values to variables	-	-	
4	Constants	5, 6	7	22 - 25
4.1	Integer constants	-	-	
4.2	Real constants	-	-	
4.3	Character constants	-	-	
4.4	String constants	-	-	
4.5	Backslash character constants	-	-	
5	Summary	-	-	26

6	Terminal Questions	-	-	27
7	Answers to Self Assessment Questions	-	-	27
8	Answers to Terminal Questions	-	-	28
9	Exercises	-	-	28



1. INTRODUCTION

Data Types play a fundamental role in C programming, providing a way to classify and work with different types of data. This knowledge is crucial for understanding the behaviour of variables and constants in C programs. The objectives of delving into data types are to ensure precise representation of data and facilitate efficient program execution.

Floating-point Numbers are a specific data type in C that allows representation of real numbers, including decimal fractions. Understanding their characteristics and usage is vital for dealing with mathematical computations and real-world applications where precision is essential.

The type cast Operator is a powerful tool for explicitly converting one data type into another. This capability is particularly useful when working with mixed data types in expressions, allowing programmers to control how data is interpreted and processed.

The type char represents character data in C. It is a fundamental data type for handling single characters, making it indispensable for tasks involving text processing and manipulation.

Keywords in C are reserved words that have predefined meanings. These words are crucial for constructing statements and expressions in a C program. Mastery of keywords ensures proper communication with the compiler, facilitating accurate program execution.

Variables, introduced in section 2.2, serve as containers for storing and manipulating data during program execution. Proper declaration and usage of variables are essential for writing effective and readable C programs.

Simple C programs involve applying the concepts discussed earlier, including variables and data types, to solve specific problems. These programs serve as practical examples to reinforce understanding and application of C programming principles.

Constants in C are values that remain unchanged during program execution. Integer Constants, Real Constants, Character Constants, String Constants, and Backslash Character Constants are various types that cater to different data representations and scenarios.

In conclusion, this comprehensive introduction to data types and related concepts in C programming lays the foundation for constructing robust and efficient programs. Mastering these topics is essential for programmers to harness the full potential of the C language in diverse applications.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Understand the role of data types in C programming.*
- ❖ *Demonstrate proficiency in declaring and using variables.*
- ❖ *Apply the type cast operator*
- ❖ *Differentiate between integer, real, character, string, and backslash character constants.*

2. DATA TYPES IN C

Data types in C programming are foundational elements that define the nature of variables and guide data manipulation. The primary objective is to understand their impact on data representation in computer memory, optimizing storage and execution efficiency. Core data types, including int, float, double, and char, require exploration to make informed choices in variable selection based on precision and range needs. Going beyond built-in types, learning to create user-defined types using structures and enumerations enhances code organization. Finally, delving into type modifiers (e.g., short, long) and qualifiers (e.g., const, volatile) refines data type characteristics, enabling programmers to tailor variables for specific tasks and ensuring the creation of efficient, readable, and resilient C programs.

2.1. Floating-point Numbers

Floating point (or real) numbers are stored in 32 bit (on all 16 bit and 32 bit machines), with 6 digits of precision. Floating point numbers are defined in C by the keyword float. When the accuracy provided by a float number is not sufficient, the type double can be used to define the number. A double data type number uses 64 bits giving a precision of 14 digits. These are known as double precision numbers. To extend the precision further, we may use long double which uses 80 bits. The table 3.1 shows all the allowed combinations of floating point numbers and qualifiers and their size and range on a 16-bit machine.

Table 2.1.1: Floating point numbers and qualifiers and their size and range on a 16-bit machine.

Type	Size (bits)	Range
Float	32	3.4E-38 to 3.4E+38
Double	64	1.7E-308 to 1.7E+308
long double	80	3.4E-4932 to 1.1E+4932

Program 2.1.1: The following program illustrates typical declarations, assignments and values stored in various types of variables.

```
main()
{
/* .....DECLARATIONS..... */
    float x, p;
    double y, q;
    unsigned k;
/* .....DECLARATIONS AND ASSIGNMENTS..... */
    int m=54321;
    long int n=1234567890;
/* .....ASSIGNMENTS..... */
    x = 1.234567890000;
    y = 9.87654321;
    k = 54321;
    p=q=1.0;
/* .....PRINTING..... */
    printf("m=%d\n",m);
    printf("n=%ld\n",n);
    printf("x=%.12lf\n",x);
    printf("x=%f\n",x);
    printf("y=%.12lf\n",y);
    printf("y=%lf\n",y);
    printf("k=%u    p= %f  q=%.12lf\n",k,p,q);
}
```

Output

m= -11215

n= 1234567890

x = 1.234567880630

x = 1.234568

y = 9.876543210000

y = 9.876543

k = 54321 p = 1.000000 q= 1.000000000000

2.2. Program to calculate the average of N numbers

Program 2.1.2: Program to calculate the average of N numbers

```
#define N 10 /* SYMBOLIC CONSTANT */
main()
{
    Int count;                /* DECLARATION OF
    float    sum, average, number;    VARIABLES */
    sum = 0;                  /* * INITIALIZATION OF
    count = 0;                VARIABLES*/
    while (count<N)
    {
        scanf("%f", &number);
        sum = sum + number;
        count = count + 1;
    }
    average = sum / N;
    printf("N = %d      Sum = %f", N, sum);
    printf("Average = %f", average);
```

Output

1
2.3
4.67
1.42
7
3.67
4.08
2.2
4.25
8.21
N= 10 Sum= 38.799999 Average= 3.880000

2.3. Program to compute the roots of a quadratic equation

Program 2.1.3: Program to compute the roots of a quadratic equation

```
#include <math.h>
main()
{
    float a,b,c,discriminant, root1, root2;
    printf("input the values of a,b and c\n");
    scanf ("%f %f %f", &a, &b, &c);
    discriminant = b * b - 4 * a * c;
    if (discriminant<0)
        printf("roots are imaginary\n");
    else
    {
        root1 = (-b + sqrt(discriminant)) / (2 * a);
        root2 = (-b - sqrt(discriminant)) / (2 * a);
        printf ("Root1 = %5.2f \n Root2 = %5.2f \n", root1, root2); }
}
```

Output

input the values of a,b and c

24 -16 Root1 = 2.00 Root2 = -4.00

input the values of a,b and c

12 3

roots are imaginary

2.4. Converting Integers to Floating-point and vice-versa

C permits mixing of constants and variables of different types in an expression, but during evaluation it adheres to very strict rules of type conversion. We know that the computer considers one operator at a time, involving two operands.

If the operands are of different types, the 'lower' type is automatically converted to the 'higher' type before the operation proceeds. The result is of higher type.

Given below is the sequence of rules that are applied while evaluating expressions.

All **short** type are automatically converted to **int** ; then

- If one of the operands is **long double**, the other will be converted to **long double** and the result will be **long double**;
- else, if one of the operands is **double**, the other will be converted to **double** and the result will be **double**;
- else, if one of the operands is **float**, the other will be converted to **float** and the result will be **float**;
- else, if one of the operands is **unsigned long int**, the other will be converted to **unsigned long int** and the result will be **unsigned long int**;
- else if one of the operands is **long int** and the other is **unsigned int**, then:
 - if **unsigned int** can be converted to **long int**, the unsigned int operand will be converted as such and the result will be **long int**;
 - else, both operands will be converted to **unsigned long int** and the result will be **unsigned long int**;
- else, if one of the operands is **long int** , the other will be converted to **long int** and the result will be **long int**;
- else, if one of the operands is **unsigned int** , the other will be converted to **unsigned int** and the result will be **unsigned int**;

The final result of an expression is converted to type of the variable on the left of the assignment sign before assigning the value to it. However, the following changes are introduced during the final assignment:

- float to int causes truncation of the fractional part.
- double to float causes rounding of digits.
- long int to int causes dropping of the excess higher order bits

2.5. Mixed-mode Expressions

When one of the operands is real and the other is integer, the expression is called a mixed-mode arithmetic expression. If either operand is of the real type, then only the real operation is performed and the result is always real number. Thus

$$25 / 10.0 = 2.5$$

Whereas $25 / 10 = 2$

SELF-ASSESSMENT QUESTIONS - 1

1. When the accuracy provided by a **float** number is not sufficient, the type **long float** can be used to define the number. (True/False)
2. A **double** data type uses _____ bits.
3. If the operands are of different data types, the 'lower' type is automatically converted to the 'higher' type before the operation proceeds. (True/False)
4. During the final assignment _____ to int causes dropping of the excess higher order bits.
5. The value of the expression $22.0/10$ is _____.

2.6. The type cast Operator

C performs type conversions automatically. However, there are instances when we want to force a type conversion in a way that is different from the automatic conversion. Consider, for example, the calculation of ratio of doctors to engineers in a town.

$$\text{Ratio} = \text{doctor_number} / \text{engineer_number}$$

Since `doctor_number` and `engineer_number` are declared as integers in the program, the decimal part of the result of the division would be lost and `Ratio` would represent a wrong figure. This problem can be solved by converting locally one of the variables to the floating point as shown below:

$$\text{Ratio} = (\text{float}) \text{doctor_number} / \text{engineer_number}$$

The operator **(float)** converts the `doctor_number` to floating point for the purpose of evaluation of the expression. Then using the rule of automatic conversion, the division is performed in floating point mode, thus retaining the fractional part of the result. Note that in no way does the operator **(float)** affect the value of the variable `doctor_number`. And also, the type of `doctor_number` remains as **int** in the other parts of the program.

The process of such local conversion is known as casting a value. The general form of cast is:

(type-name) expression

where `type-name` is one of the standard C data types. The expression may be a constant, variable or an expression. The Table 3.2 shows some examples of casts and their actions:

Table 2.1.2: Use of Casts

Example	Action
X=(int) 8.5	8.5 is converted to integer by truncation.
A=(int) 21.3 / (int) 4.5	Evaluated as 21/4 and the result would be 5.
B=(double) sum/n	Division is done in floating point mode.
Y= (int) (a+b)	The result of a+b is converted to integer.
Z= (int) a+b	a is converted to integer and then added to b.
P=cos((double)x)	Converts x to double before using it.

Program: The following program shows the use of casts

```
main()
{
    /* Program to find average of two integers */
    float avg;
    int n=2,n1,n2;
    printf("enter any 2 numbers\n");
    scanf("%d %d",&n1,&n2);
    avg=(n1+n2)/(float)n;
    printf(" their average is\n",avg);
}
```

Casting can be used to round-off a given value. Consider the following statement:

X= (**int**) (y+0.5);

If y is 37.7, y+0.5 is 38.2 and on casting, the result becomes 38, the value that is assigned to X. Of course, the expression, being cast is not changed. When combining two different types

of variables in an expression, never assume the rules of automatic conversion. It is always a good practice to explicitly force the conversion. It is more safer and more portable. For example, when **y** and **p** are double and **m** is **int**, the following two statements are equivalent.

```
y = p + m;
```

```
y = p + (double)m;
```

However, the second statement is preferable. It will work the same way on all machines and is more readable.

SELF-ASSESSMENT QUESTIONS - 2

6. Casting can be used to round-off a given value. (True/False)
7. The value of A in the expression `A=(int)11.35/(int)14.5` is _____.
8. If the value of X is 35.2, then the value of A in the expression: `A = (int)(X+0.5);` is _____.

2.7. The type char

A single character can be defined as a character(**char**) type data. Characters are usually stored in 8 bits (one byte) of internal storage. The qualifier **signed** or **unsigned** may be explicitly applied to **char**. While **unsigned chars** have values between 0 and 255, signed chars have values from -128 to 127.

A character constant is formed by enclosing the character within a pair of single quote marks. So 'b', '.' and '5' are all valid examples of character constants. Note that a character constant, which is a single character enclosed in single quotes is different from a character string, which is any number of characters enclosed in double quotes.

The format characters `%c` can be used in a `printf` statement to display the value of a char variable at the terminal.

Program: The following program illustrates how to use char data type.

```
#include<stdio.h>
main()
{
char c='A';
int a=65;
printf("%c\n", c);
printf("%d\n", c);
printf("%c\n",a);
}
```

Output

A

65

A

Note that with the format characters %d, the ASCII number of the character is displayed. With the format character %c, the character corresponding to the given ASCII number is displayed.

SELF-ASSESSMENT QUESTIONS - 3

9. What is the format character to display the value of a char variable?
10. The output of the following C statement: `printf("%c", 70);` is _____.

2.8. Keywords

Keywords are the reserved words of a programming language. All the keywords have fixed meanings and these meanings cannot be changed.

Keywords serve as basic building blocks for program statements. The list of all keywords in ANSI C are listed in the Table 3.3.

Table 2.1.3: ANSI C Keywords

auto	Double	int	struct
break	Else	long	switch
case	Enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

All keywords must be written in lowercase. Some compilers may use additional keywords that must be identified from the C manual.

SELF-ASSESSMENT QUESTIONS - 4

11. All keywords must be written in _____.
12. **default** is not a valid keyword in C. (True/False)

3. CONCEPT OF AN INTEGER AND VARIABLE

Integers are whole numbers with a range of values supported by a particular machine. Generally, integers occupy one word of storage, and since the word sizes of machines vary (typically, 16 or 32 bits) the size of an integer that can be stored depends on the computer. If we use 16 bit word length, the size of the integer value is limited to the range -32768 to +32767 (that is, -2^{15} to $+2^{15} - 1$). A signed integer uses one bit for sign and 15 bits for the magnitude of the number. Similarly, a 32 bit word length can store an integer ranging from -2,147,483,648 to 2,147,483,647.

In order to provide some control over the range of numbers and storage space, C has three classes of integer storage, namely **short int**, **int**, and **long int**, in both signed and unsigned forms. For example, short int represents fairly small integer values and requires half the amount of storage as a regular **int number** uses. A signed integer uses one bit for sign and 15 bits for the magnitude of the number, therefore, for a 16 bit machine, the range of unsigned integer numbers will be from 0 to 65,535.

We declare **long** and **unsigned integers** to increase the range of values. The use of qualifier **signed** on integers is optional because the default declaration assumes a signed number. The Table 1.2 shows all the allowed combinations of basic types and qualifiers and their size and range on a 16-bit machine.

Table 2.2.1: Basic types, qualifiers and their size

Type	Size (bits)	Range
int or signed int	16	-32,768 to 32,767
unsigned int	16	0 to 65535
short int or signed short int	8	-128 to 127
unsigned short int	8	0 to 255
long int or signed long int	32	-2,147,483,648 to 2,147,483,647
unsigned long int	32	0 to 4,294,967,295

Informally, a *variable* (also called an *object*) is a place where you can store a value so that you can refer to it unambiguously. A variable needs a name. You can think of the variables in your program as a set of boxes, each with a label giving its name; you might imagine that storing a value “in” a variable consists of writing the value on a slip of paper and placing it in the box.

Now let us see how integers and variables are declared. A *declaration* tells the compiler the name and type of a variable you'll be using in your program.

In its simplest form, a declaration consists of the type, the name of the variable, and a terminating semicolon:

```
int i;
```

The above statement declares an integer variable *i*.

```
long int i1, i2;
```

We can also declare several variables of the same type in one declaration, separating them with commas as shown above.

The placement of declarations is significant. You can't place them just anywhere (i.e. they cannot be interspersed with the other statements in your program). They must either be placed at the beginning of a function, or at the beginning of a brace-enclosed block of statements, or outside of any function. Furthermore, the placement of a declaration, as well as its storage class, controls several things about its *visibility* and *lifetime*, as we'll see later.

You may wonder *why* variables must be declared before use. There are two reasons:

- It makes things somewhat easier on the compiler; it knows right away what kind of storage to allocate and what code to emit to store and manipulate each variable; it doesn't have to try to intuit the programmer's intentions.
- It forces a bit of useful discipline on the programmer: you cannot introduce variables wherever you wish; you must think about them enough to pick appropriate types for them. (The compiler's error messages to you, telling you that you apparently forgot to declare a variable, are as often helpful as they are a nuisance: they're helpful when they

tell you that you misspelled a variable, or forgot to think about exactly how you were going to use it.)

Most of the time, it is recommended to write one declaration per line. For the most part, the compiler doesn't care what order declarations are in. You can order the declarations alphabetically, or in the order that they're used, or to put related declarations next to each other. Collecting all variables of the same type together on one line essentially orders declarations by type, which isn't a very useful order (it's only slightly more useful than random order).

A declaration for a variable can also contain an initial value. This initializer consists of an equal sign and an expression, which is usually a single constant:

```
int i = 1;
```

```
int i1 = 10, i2 = 20;
```

SELF-ASSESSMENT QUESTIONS - 5

13. The size of the Integers in C language is same in all the machines. (True/False)
14. A _____ is a place where we can store values.
15. Size of int is _____ bits.
16. Which of the following tells the compiler the name and type of a variable you'll be using in your program? (declaration)
 - (a) declaration
 - (b) variables
 - (c) integers
 - (d) assignments
17. The _____ consists of an equal sign and an expression, which is usually a single constant. (initializer)
18. A single declaration statement can contain variables of different types. (True/False)

3.1. Rules for naming Variables and assigning values to variables

Within limits, you can give your variables and functions any names you want. These names (the formal term is “identifiers”) consist of letters, numbers, and underscores. For our purposes, names must begin with a letter. Theoretically, names can be as long as you want, but extremely long ones get tedious to type after a while, and the compiler is not required to keep track of extremely long ones perfectly. (What this means is that if you were to name a variable, say, *supercalafragalisticespialidocious*, the compiler might get lazy and pretend that you'd named it *super-calafragalisticespialidocio*, such that if you later misspelled it *supercalafragalisticespialidociouz*, the compiler wouldn't catch your mistake. Nor would the compiler necessarily be able to tell the difference if for some perverse reason you deliberately declared a second variable named *supercalafragalisticespialidociouz*.)

The capitalization of names in C is significant: the variable names *variable*, *Variable*, and *VARIABLE* (as well as silly combinations like *variAble*) are all distinct.

A final restriction on names is that you may not use keywords (the words such as **int** and **for** which are part of the syntax of the language) as the names of variables or functions (or as identifiers of any kind).

Now let us see how you can assign values to variables. The assignment operator = assigns a value to a variable. For example,

```
x = 1;
```

sets x to 1, and

```
a = b;
```

sets a to whatever b's value is. The expression

```
i = i + 1;
```

is, as we've mentioned elsewhere, the standard programming idiom for increasing a variable's value by 1: this expression takes i's old value, adds 1

to it, and stores it back into i. (C provides several “shortcut” operators for modifying variables in this and similar ways, which we’ll meet later. We’ve called the = sign the “assignment operator” and referred to “assignment expressions” because, in fact, = is an operator just like + or -. C does not have “assignment statements”; instead, an assignment like `a = b` is an expression and can be used wherever any expression can appear. Since it's an expression, the assignment `a = b` has a value, namely, the same value that's assigned to a. This value can then be used in a larger expression; for example, we might write

```
c = a = b;
```

Which is equivalent to?

```
c = (a = b);
```

and assigns b’s value to both a and c. (The assignment operator, therefore, groups from right to left.) Later we’ll see other circumstances in which it can be useful to use the value of an assignment expression. It's usually a matter of style whether you initialize a variable with an initializer in its declaration or with an assignment expression near where you first use it. That is, there's no particular difference between

```
int a = 10;
```

and

```
int a;
```

```
/* later... */
```

```
a = 10;
```

SELF-ASSESSMENT QUESTIONS - 6

19. In C, variable names are case sensitive. (True/False)

20. A variable name in C consists of letters, numbers and _____.

4. CONSTANTS

Constants in C refer to fixed values that do not change during the execution of a program. C supports several types of constants as illustrated in Fig. 1.1.

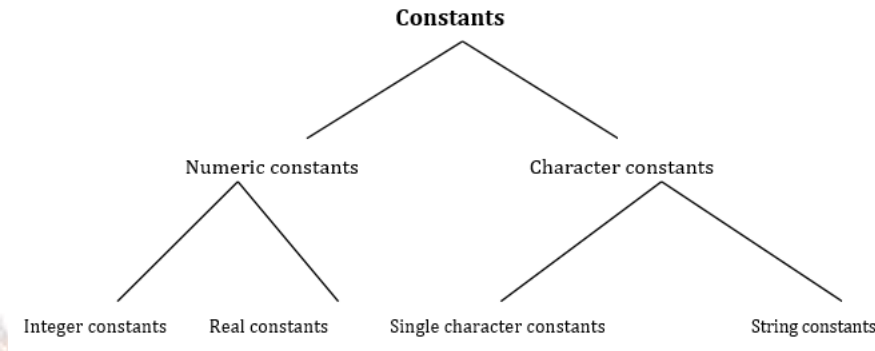


Fig. 2.3.1

4.1. Integer constants

An integer constant refers to a sequence of *digits*. There are three types of integers, namely *decimal*, *octal* and *hexadecimal*.

Decimal integers consist of a set of digits, 0 through 9, preceded by an optional – or +.

Examples: 12, -546, 0, 354647, +56

An *octal* integer constant consists of any combination of digits from the set 0 through 7, with a leading 0.

Examples: 045, 0, 0567

A sequence of digits preceded by 0x or 0X is considered as hexadecimal integer. They may also include alphabets A through F or a through f. The letters A through F represent numbers 10 through 15.

Examples: 0X6, 0x5B, 0Xbcd, 0X

The largest integer value that can be stored is machine-dependent. It is 32767 on 16-bit machines and 2,147,483,647 on 32-bit machines. It is also possible to store larger integer constants on these machines by appending *qualifiers* such as U, L and UL to the constants.

Examples: 54637U or 54637u (unsigned integer)

65757564345UL or 65757564345ul (unsigned long integer)

7685784L or 7685784l (long integer)

Program: Program to represent integer constants on a 16-bit computer

```
/* Integer numbers on 16-bit machine */
main()
{
    printf("Integer values\n\n");
    printf("%d %d %d\n", 32767, 32767+1, 32767+10); printf("\n");
    printf("Long integer values\n\n");
    printf("%ld %ld %ld\n", 32767L, 32767L+1L, 32767L+10L);
}
```

Type and execute the above program and observe the output

4.2. Real constants

The numbers containing fractional parts like 67.45 are called real (or floating point) constants.

Examples: 0.0045, -8.5, +345.678

A real number may also be expressed in exponential (scientific) notation.

The general form is:

mantissa e exponent

The mantissa is either a real number expressed in decimal notation or an integer. The exponent is an integer number with an optional plus or minus sign. The letter e separating the mantissa and the exponent can be written in either lowercase or uppercase.

Examples: 04e4, 12e-2, -1.3E-2

7500000000 may be written as 7.5E9 or 75E8.

Floating point constants are normally represented as double-precision quantities. However, the suffixes f or F may be used to force single precision and l or L to extend double-precision further.

4.3. Character constants

A single character constant (or simple character constant) contains a single character enclosed within a pair of single quote marks.

Examples: '6', 'X', ';'

Character constants have integer values known as ASCII values. For example, the statement

```
printf("%d", 'a');
```

would print the number 97, the ASCII value of the letter a. Similarly, the statement

```
printf("%c", 97);
```

would print the letter a.

4.4. String constants

A string constant is a sequence of characters enclosed within double quotes. The characters may be letters, numbers, special characters and blank space.

Examples: "Hello!", "1947", "5+3"

4.5. Backslash character constants

C supports some special backslash character constants that are used in output functions. A list of such backslash character constants is given in Table 1.1. Note that each one of them represents one character, although they consist of two characters. These character combinations are called *escape sequences*.

Table 2.3'1: Backslash character constants

Constant	Meaning
'\b'	back space
'\f'	form feed
'\n'	new line
'\r'	carriage return
'\t'	horizontal tab
'\v'	vertical tab
'\"'	single quote
'\"'	double quote
'\\'	backslash
'\0'	null

SELF-ASSESSMENT QUESTIONS - 7

21. An octal integer constant consists of any combination of digits from the set _____ through _____.
22. A sequence of digits preceded by 0x or 0X is considered as _____ integer.
23. A string constant is a sequence of _____ enclosed within double quotes.

5. SUMMARY

In the realm of C programming, this overview unfolds with an introduction, highlighting the significance of various components. Section 2.1 delves into data types, elucidating their role in shaping how variables store and process information. It covers fundamental elements like floating-point numbers, the type cast operator, 'char' type, and keywords, establishing the groundwork for precise programming.

The subsequent sections, 2.2 and 2.3, navigate through variables and simple C programs, respectively. Understanding variables is essential for effective data manipulation, while the creation of simple programs reinforces practical application.

The exploration extends to constants, encompassing integer, real, character, string, and backslash character constants. This comprehensive approach, emphasizing objectives and practicality, ensures a well-rounded comprehension of the foundational elements necessary for proficient C programming.

6. TERMINAL QUESTIONS

1. Distinguish between signed and unsigned integers.
2. What are the components of declaration statement?
3. State the rules for naming a variable in C.
4. Clarify the different constants present in the C programming language.?

7. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. False
2. 64
3. True
4. long in
5. 2.2
6. True
7. 0
8. 35
9. %c
10. False
11. Lower Case
12. False
13. False
14. Variable
15. 16
16. (a) declaration
17. Initializer
18. False
19. True
20. Underscore
21. (0,7)
22. Hexadecimal
23. characters

8. ANSWERS TO TERMINAL QUESTIONS

1. A signed integer uses one bit for sign and 15 bits for the magnitude of the number. (Refer Section 2.2 for more details)
2. A declaration tells the compiler the name and type of a variable you'll be using in your program. In its simplest form, a declaration consists of the type, the name of the variable, and a terminating semicolon:

```
int i;
```

The above statement declares an integer variable i.

```
long int i1, i2;
```

3. In the C programming language, a variable is a named storage location that holds a value. It is a fundamental concept used for storing and manipulating data within a program. (For variable refer section 2.2)
4. Constants in C are values that remain unchanged during program execution. (For variable refer section 2.3)

9. EXERCISE

1. Write a program that takes two integers as input and swaps their values using a temporary variable.
2. Write a program that calculates the area of a rectangle using constants for length and width.
3. Convert the temperature in Fahrenheit to Celsius by taking user input.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 3

I/O Operations in C

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Formatted input and output	1, 2	1	4 - 10
3	Interactive Programming	-	-	11 - 12
4	Summary	-	-	13
5	Terminal Questions	-	-	14
6	Answers to Self Assessment Questions	-	-	15
7	Answers to Terminal Questions	-	-	15
8	Exercises	-	-	15

1. INTRODUCTION

The printf and scanf functions are commonly used in the C programming language for input and output operations. printf formats and prints output in C, using format specifiers like %d and %s. scanf reads formatted input, requiring variable addresses as arguments, e.g., scanf("%d", &num);. Maintaining specifier alignment between printf and scanf is vital. Careful input handling is essential to thwart security risks, such as buffer overflows. These functions play a fundamental role in robust input/output management in C programming.

The functions scanf() and printf() permit the transfer of single character, numerical values and strings; the functions gets() and puts() facilitate the input and output of strings. These functions can be accessed within a program by including the header file stdio.h.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Understand and implement the printf function in C for formatted output*
- ❖ *Gain proficiency in using scanf for formatted input in C programs*

2. FORMATTED INPUT AND OUTPUT

Input data can be entered into the computer from a standard input device by means of the standard C library function **scanf()**. This function can be used to enter any combination of numerical values, single character and strings. The function returns the number of data items that have been entered successfully.

The syntax of scanf() function is as follows:

scanf(control string, arg1, arg2, ...argn)

where control string refers to a string containing certain required formatting information, and *arg1, arg2, ..., argn* are arguments that represent the individual input data items. The arguments represent pointers that indicate addresses of the data items within the computer's memory.

The control string consists of control characters, whitespace characters, and non-whitespace characters. The control characters are preceded by a % sign, and are listed in Table 3.1.1.

Table 3.1.1: Control characters and its explanations

Control Character	Explanation
%c	a single character
%d	a decimal integer
%i	an integer
%e, %f, %g	a floating-point number
%o	an octal number
%s	a string
%x	a hexadecimal number

%p	a pointer
%n	an integer equal to the number of characters read so far
%u	an unsigned integer
%[]	a set of characters
%%	a percent sign

scanf() reads the input, matching the characters from format. When a control character is read, it puts the value in the next variable. Whitespaces (tabs, spaces, etc) are skipped. Non-whitespace characters are matched to the input, then discarded. If a number comes between the % sign and the control character, then only that many characters will be entered into the variable. If **scanf()** encounters a set of characters, denoted by the %[] control character, then any characters found within the brackets are read into the variable. The return value of **scanf()** is the number of variables that were successfully assigned values, or EOF if there is an error.

Program : Program to use scanf() to read integers, floats, characters and strings from the user.

```
#include<stdio.h>
main()
{
int i;
float f;
char c;
char str[10];
scanf("%d %f %c %s", &i, &f, &c, str);
printf("%d %f %c %s", i, f, c, str);
}
```

Execute this program and observe the result.

Note that for a **scanf()** function, the addresses of the variable are used as the arguments for an **int**, **float** and a **char** type variable. But this is not true for a string variable because a string name itself refers to the address of a string variable.

A *control* character is used to enter strings to string variables. A string that includes whitespace characters can not be entered. There are ways to work with strings that include whitespace characters. One way is to use the **getchar()** function within a loop. Another way is to use **gets()** function which will be discussed later.

It is also possible to use the **scanf()** function to enter such strings. To do so, the s-control character must be replaced by a sequence of characters enclosed in square brackets, designated as [...]. Whitespace characters may be included within the brackets, thus accommodating strings that contain such characters.

Example:

```
#include<stdio.h>
main()
{
char str[80];
...
scanf("%[ ABCDEFGHIJKLMNOPQRST]", str);
...
}
```

This example illustrates the use of the **scanf()** function to enter a string consisting of uppercase letters and blank spaces. Please note that if you want to allow lowercase letters to be entered, all the lowercase letters (i.e. from a-z) must be included in the list of control string.

Formatted Output

Output data can be written from the computer onto a standard output device using the library function **printf()**. This function can be used to output any combination of numerical

values, single characters and strings. It is similar to the input function **scanf()**, except that its purpose is to display data rather than enter into the computer.

The syntax of the **printf** function can be written as follows:

printf(control string, arg1, arg2, ..., argn)

where control string refers to a string that contains formatting information, and arg1, arg2, ..., argn are arguments that represent the individual output data items. The arguments can be written as constants, single variable or array names, or more complex expressions.

Examples:

```
printf("Hello, world!\n");
```

```
printf("i is %d\n", i);
```

```
printf("%d", 10);
```

```
printf("%d", i+j);
```

The first statement simply displays the string given as argument to the printf() function. In the second statement, printf() function replaces the two characters %d with the value of the variable i. In the third statement the argument to be printed is a constant and in the fourth, the argument is an expression.

There are quite a number of format specifiers for printf(). Some of them are listed in Table 3.1.2.

Table 3.1.2: Format specifiers for printf().

%d	Print an int argument in decimal
%ld	print a long int argument in decimal
%c	print a character
%s	print a string
%f	print a float or double argument
%e	same as %f, but use exponential notation
%g	use %e or %f, whichever is better
%o	print an int argument in octal (base 8)
%x	print an int argument in hexadecimal (base 16)
%%	print a single %

It is also possible to specify the width and precision of numbers and strings as they are inserted ; For example, a notation like %3d means to print an int in a field at least 3 spaces wide; a notation like %5.2f means to print a float or **double** in a field at least 5 spaces wide, with two places to the right of the decimal.)

To illustrate with a few more examples: the call

```
printf("%c %d %f %e %s %d%%\n", '3', 4, 3.24, 66000000, "nine", 8);
```

 would print

```
3 4 3.240000 6.600000e+07 nine 8%
```

The call

```
printf("%d %o %x\n", 100, 100, 100);
```

would print

```
100 144 64
```

Successive calls to `printf()` just build up the output a piece at a time, so the calls

```
printf("Hello, ");
```

```
printf("world!\n");
```

would also print `Hello, world!` (on one line of output).

Earlier we learned that C represents characters internally as small integers corresponding to the characters' values in the machine's character set (typically ASCII). This means that there isn't really much difference between a character and an integer in C; most of the difference is in whether we choose to interpret an integer as an integer or a character. `printf` is one place where we get to make that choice: `%d` prints an integer value as a

string of digits representing its decimal value, while `%c` prints the character corresponding to a character set value. So the lines

```
char c = 'A';
```

```
int i = 97;
```

```
printf("c = %c, i = %d\n", c, i);
```

would print `c` as the character `A` and `i` as the number `97`. But if, on the other hand, we called

```
printf("c = %d, i = %c\n", c, i);
```

we'd see the decimal value (printed by `%d`) of the character `'A'`, followed by the character (whatever it is) which happens to have the decimal value `97`.

You have to be careful when calling **`printf()`**. It has no way of knowing how many arguments you've passed it or what their types are other than by looking for the format specifiers in the format string. If there are more format specifiers (that is, more `%` signs) than the arguments, or if the arguments have the wrong types for the format specifiers, **`printf()`** can misbehave

badly, often printing nonsense numbers or (even worse) numbers which mislead you into thinking that some other part of your program is broken.

Because of some automatic conversion rules which we haven't covered yet, you have a small amount of latitude in the types of the expressions you pass as arguments to **printf()**. The argument for **%c** may be of type **char** or **int**, and the argument for **%d** may be of type **char** or **int**. The string argument for **%s** may be a string constant, an array of characters, or a pointer to some characters. Finally, the arguments corresponding to **%e**, **%f**, and **%g** may be of types **float** or **double**. But other combinations do not work reliably: **%d** will not print a long int or a **float** or a **double**; **%ld** will not print an int; **%e**, **%f**, and **%g** will not print an **int**.

SELF-ASSESSMENT QUESTIONS - 1

1. The _____ string consists of control characters, whitespace characters, and non-whitespace characters.
2. The control string used to read a hexadecimal character is_____.
3. scanf() functions needs address of the data item to be read as the argument. (True/False).
4. The output of the following statement is _____. printf("%d %o %x\n", 64, 10, 75);
5. To print an int argument in octal, you can use _____ format string.
6. The output of the following program segment is _____.

```
int a=97;
```

```
printf("%c", a);
```

2. INTERACTIVE PROGRAMMING

Creating interactive dialog between the computer and the user is a modern style of programming. These dialogs usually involve some form of question-answer interaction, where the computer asks the questions and the user provides the answer, or vice versa. In C, such dialogs can be created by alternate use of the `scanf()` and `printf()` functions.

Program : Program to calculate the simple interest

```
#include<stdio.h>
main()
{
/* Sample interactive program*/
float principle, rate, time, interest;
printf(" Please enter the principle amount: ");
scanf("%f", &principle);
printf(" Please enter the rate of interest: ");
scanf("%f", &rate);
printf(" Please enter the period of deposit: ");
scanf("%f", &time);
interest=principle*rate*time/100.0;
printf("Principle=%7.2f\n", principle);
printf("Rate of interest=%5.2f\n",rate);
printf("Period of deposit=%5.2f\n", time);
printf("Interest=%7.2f\n", interest);
}
```

Execute the above program and observe the result.

Example Input:

Please enter the principle amount: 1000

Please enter the rate of interest: 5

Please enter the period of deposit: 2

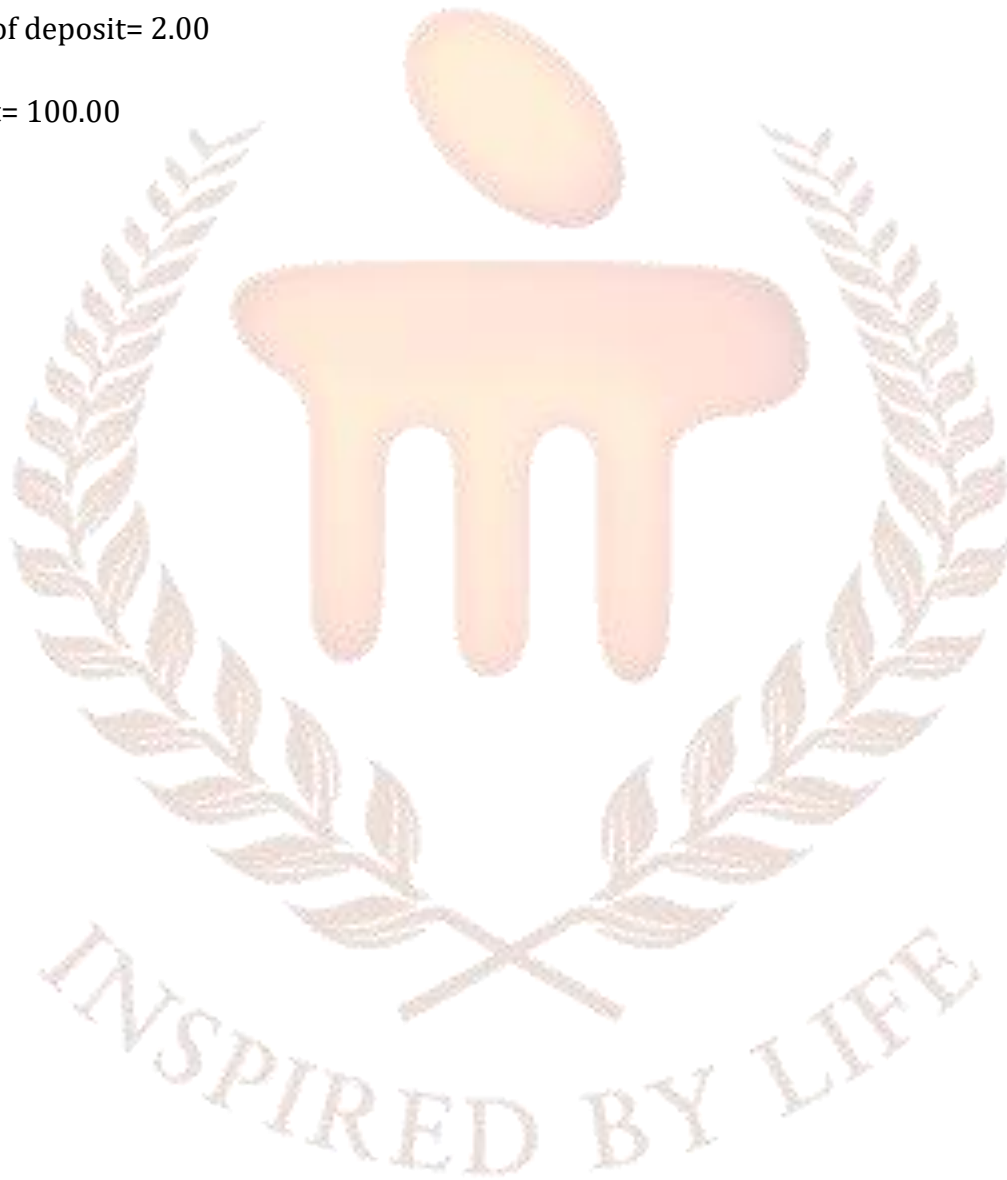
Corresponding Output:

Principle=1000.00

Rate of interest= 5.00

Period of deposit= 2.00

Interest= 100.00



3. SUMMARY

scanf() and **printf()** are the two formatted input/output functions. These functions can handle characters, numerical values and strings as well. In C, printf formats output using placeholders like %d or %s, while scanf handles formatted input with specifiers such as %d for variables, ensuring data type alignment. For example, scanf("%d", &num); reads an integer. Secure input practices, like validating and sanitizing user inputs, are crucial to prevent vulnerabilities like buffer overflows. Proper specifier matching between printf and scanf is essential for program reliability. Additional concepts include escape sequences for special characters and modifiers for field width and precision. Understanding these functions is fundamental for controlled, secure, and versatile input/output operations in C, contributing to the development of robust and effective programs.

4. TERMINAL QUESTIONS

1. Explain the role of format specifiers in the printf function in C. Provide examples of different format specifiers and their corresponding data types.
2. Describe how the scanf function is used for input in C. Include examples with different format specifiers and discuss the importance of using the correct specifiers for variable types.
3. Discuss the significance of the & (ampersand) operator in the context of the scanf function. Provide an example to illustrate its usage.



5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. Control
2. %x
3. True
4. 64, 12, 4B
5. %o
6. a

6. ANSWERS TO TERMINAL QUESTIONS

1. Refer section 2
2. Refer section 2
3. Refer section 2

7. EXERCISES

1. Write C that calculates the factorial of a given number.
2. Write a C program that takes two integers as input from the user, calculates their sum and product, and prints the results.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

CODE
C PROGRAMMING

Unit 4

Operators and Expressions

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Arithmetic Operators	-	1	4 - 5
3	Unary Operators	-	-	6
4	Relational and Logical Operators	-	2	7 - 11
5	The Conditional Operator	-	-	12
6	Library Functions	1	3	13 - 16
7	Bitwise Operators	-	4	17 - 18
8	The Increment and Decrement Operators	-	5	19 - 22
9	The Size of Operator	-	-	23 - 24
10	Precedence of operators	2	-	25 - 27
11	Summary	-	-	27
12	Terminal Questions	-	-	28
13	Answers to Self Assessment Questions	-	-	29
14	Answers to Terminal Questions	-	-	29
15	Exercises	-	-	30

1. INTRODUCTION

In the previous unit, you learned about the various features and structure of C programs. You also learned how the variables in C are declared. In this unit, you will learn about the operators that are available in C and how the expressions can be formed to get the solutions of any problems.

C supports a rich set of operators. An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations. Operators are used in programs to manipulate data and variables. They usually form a part of the mathematical or logical expressions.

C operator can be classified into a number of categories. They include:

1. Arithmetic operators
2. Unary operator
3. Relational operators
4. Logical operators
5. Conditional operator
6. Bitwise operators
7. Increment and Decrement operators

1.1. Objectives:

After studying this subject, you should be able to:

- ❖ *Explain different categories of operators*
- ❖ *Use operators on many operands*
- ❖ *Distinguish precedence and associativity of operators*
- ❖ *Explain library functions and their use*
- ❖ *Write small programs using different types of operators*

2. ARITHMETIC OPERATORS

The basic operators for performing arithmetic are the same in many computer languages:

- + addition
- subtraction
- * multiplication
- / division
- % modulus (remainder)

The `-` operator can be used in two ways: to subtract two numbers (as in `a - b`), or to negate one number (as in `-a + b` or `a + -b`).

When applied to integers, the division operator `/` discards any remainder, so `1 / 2` is 0 and `7 / 4` is 1. But when either operand is a floating-point quantity (a real number), the division operator yields a floating-point result, with a potentially nonzero fractional part. So `1 / 2.0` is 0.5, and `7.0 / 4.0` is 1.75.

The *modulus* operator `%` gives you the remainder when two integers are divided: `1 % 2` is 1; `7 % 4` is 3. (The modulus operator can only be applied to integers.)

An additional arithmetic operation you might be wondering about is exponentiation. Some languages have an exponentiation operator (typically `^` or `**`), but C doesn't. (To square or cube a number, just multiply it by itself.)

Multiplication, division, and modulus all have higher *precedence* than addition and subtraction. The term "precedence" refers to how "tightly" operators bind to their operands (that is, to the things they operate on). In mathematics, multiplication has higher precedence than addition, so `1 + 2 * 3` is 7, not 9. In other words, `1 + 2 * 3` is equivalent to `1 + (2 * 3)`. C is the same way.

All of these operators “group” from left to right, which means that when two or more of them have the same precedence and participate next to each other in an expression, the evaluation conceptually proceeds from left to right. For example, $1 - 2 - 3$ is equivalent to $(1 - 2) - 3$ and gives -4, not +2. (“Grouping” is sometimes called associativity, although the term is used somewhat differently in programming than it is in mathematics. Not all C operators group from left to right; a few groups from right to left.)

Whenever the default precedence or associativity doesn't give you the grouping you want, you can always use explicit parentheses. For example, if you want to add 1 to 2 and then multiply the result by 3, you could write $(1+2)*3$.

Program 2.1: Program that shows the use of integer arithmetic to convert a given number of days into months and days.

```
/* Program to convert days to months and days */
main()
{
    int months, days;
    printf("Enter days\n");
    scanf("%d",&days);
    months=days/30;
    days=days%30;
    printf("Months=%d Days=%d", months,days);
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. What is the value of the arithmetic expression: $14 \% 3 + 7 \% 2$
(a) 1 (b) 2 (c) 3 (d) 4
2. _____ operator can be only applied to integers.

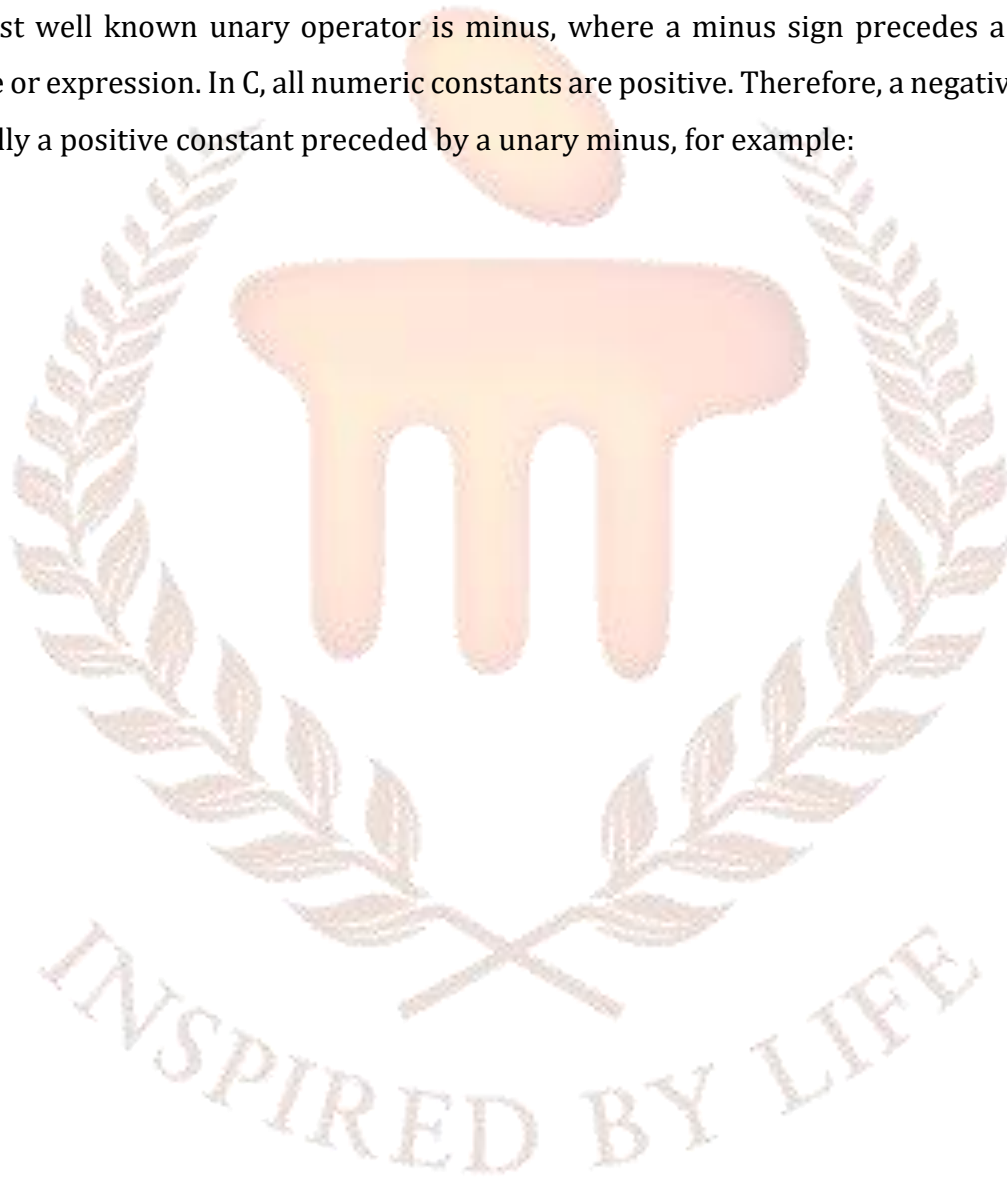
3. UNARY OPERATOR

A unary operator acts upon a single operand to produce a new value.

Unary Minus

The most well known unary operator is minus, where a minus sign precedes a constant, variable or expression. In C, all numeric constants are positive. Therefore, a negative number is actually a positive constant preceded by a unary minus, for example:

-3



4. RELATIONAL AND LOGICAL OPERATORS

An if statement like

```
if(x > max)
```

```
max = x;
```

is perhaps deceptively simple. Conceptually, we say that it checks whether the condition $x > \text{max}$ is "true" or "false". The mechanics underlying C's conception of "true" and "false," however, deserve some explanation. We need to understand how true and false values are represented, and how they are interpreted by statements like if.

As far as C is concerned, a true/false condition can be represented as an integer. (An integer can represent many values; here we care about only two values: "true" and "false." The study of mathematics involving only two values is called Boolean algebra, after George Boole, a mathematician who refined this study.) In C, "false" is represented by a value of 0 (zero), and "true" is represented by any value that is nonzero. Since there are many nonzero values (at least 65,534, for values of type int), when we have to pick a specific value for "true," we'll pick 1.

The *relational operators* such as $<$, $<=$, $>$, and $>=$ are in fact operators, just like $+$, $-$, $*$, and $/$. The relational operators take two values, look at them, and "return" a value of 1 or 0 depending on whether the tested relation was true or false. The complete set of relational operators in C is:

$<$ less than

$<=$ less than or equal

$>$ greater than

$>=$ greater than or equal

$==$ equal

$!=$ not equal

For example, $1 < 2$ is true(1), $3 > 4$ is false(0), $5 == 5$ is true(1), and $6 != 6$ is false(0).

The equality-testing operator is `==`, not a single `=`, which is assignment. If you accidentally write

```
if(a = 0)
```

(and you probably will at some point; everybody makes this mistake), it will not test whether `a` is zero, as you probably intended. Instead, it will *assign* 0 to `a`, and then perform the “true” branch of the if statement if `a` is nonzero. But `a` will have just been assigned the value 0, so the “true” branch will never be taken! (This could drive you crazy while debugging -- you wanted to do something if `a` was 0, and after the test, `a` *is* 0, whether it was supposed to be or not, but the “true” branch is nevertheless not taken.)

The relational operators work with arbitrary numbers and generate true/false values. You can also combine true/false values by using the Boolean operators(also called the logical operators), which take true/false values as operands and compute new true/false values. The three Boolean operators are:

`&&` AND

`||` OR

`!` NOT (takes one operand, “unary”)

The `&&` (“and”) operator takes two true/false values and produces a true

(1) result if both operands are true (that is, if the left-hand side is true and the right-hand side is true). The `||` (“or”) operator takes two true/false values and produces a true (1) result if either operand is true. The `!` (“not”) operator takes a single true/false value and negates it, turning false to true and true to false (0 to 1 and nonzero to 0). The logical operators `&&` and `||` are used when we want to test more than one condition and make decisions.

For example, to test whether the variable `i` lies between 1 and 10, you might use

```
if(1 < i && i < 10)
```

...

Here we're expressing the relation "i is between 1 and 10" as "1 is less than i **and** i is less than 10."

It's important to understand why the more obvious expression

```
if(1 < i < 10)      /* WRONG */
```

would not work. The expression $1 < i < 10$ is parsed by the compiler analogously to $1 + i + 10$. The expression $1 + i + 10$ is parsed as $(1 + i) + 10$ and means "add 1 to i, and then add the result to 10." Similarly, the expression $1 < i < 10$ is parsed as $(1 < i) < 10$ and means "see if 1 is less than i, and then see if the result is less than 10." But in this case, "the result" is 1 or 0, depending on whether i is greater than 1. Since both 0 and 1 are less than 10, the expression $1 < i < 10$ would always be true in C, regardless of the value of i!

Relational and Boolean expressions are usually used in contexts such as an if statement, where something is to be done or not done depending on some condition. In these cases what's actually checked is whether the expression representing the condition has a zero or nonzero value. As long as the expression is a relational or Boolean expression, the interpretation is just what we want. For example, when we wrote

```
if(x > max)
```

the $>$ operator produced a 1 if x was greater than max, and a 0 otherwise. The if statement interprets 0 as false and 1 (or any nonzero value) as true.

But what if the expression is not a relational or Boolean expression? As far as C is concerned, the controlling expression (of conditional statements like if) can in fact be *any expression*: it doesn't have to "look like" a Boolean expression; it doesn't have to contain relational or logical operators. All C looks at (when it's evaluating an if statement, or anywhere else where it needs a true/false value) is whether the expression evaluates to 0 or nonzero. For example, if you have a variable x, and you want to do something if x is nonzero, it's possible to write

```
if(x)
```

statement

and the statement will be executed if x is nonzero (since nonzero means "true").

This possibility (that the controlling expression of an if statement doesn't have to "look like" a Boolean expression) is both useful and potentially confusing. It's useful when you have a variable or a function that is "conceptually Boolean," that is, one that you consider to hold a true or false

(actually nonzero or zero) value. For example, if you have a variable `verbose` which contains a nonzero value when your program should run in verbose mode and zero when it should be quiet, you can write things like

```
if(verbose)
    printf("Starting first pass\n");
```

and this code is both legal and readable, besides which it does what you want. The standard library contains a function `isupper()` which tests whether a character is an upper-case letter, so if `c` is a character, you might write

```
if(isupper(c))
```

...

Both of these examples (`verbose` and `isupper()`) are useful and readable. However, you will eventually come across code like

```
if(n)
    average = sum / n;
```

where `n` is just a number. Here, the programmer wants to compute the average only if `n` is nonzero (otherwise, of course, the code would divide by 0), and the code works, because, in the context of the if statement, the trivial expression `n` is (as always) interpreted as "true" if it is nonzero, and "false" if it is zero.

“Coding shortcuts” like these can seem cryptic, but they're also quite common, so you'll need to be able to recognize them even if you don't choose to write them in your own code. Whenever you see code like

```
if(x)
```

or

```
if(f())
```

where x or f() do not have obvious “Boolean” names, you can read them as “if x is nonzero” or “if f() returns nonzero.”

SELF-ASSESSMENT QUESTIONS - 2

3. The logical operators _____ and _____ are used when we want to test more than one condition and make decisions.
4. State whether the following statement is correct or not. (Correct/Incorrect)

```
if(a<4<c)
```

```
b=c;
```

5. THE CONDITIONAL OPERATOR

The Conditional operator (ternary operator) pair “?:” is available in C to construct conditional expressions of the form

expr1?expr2:expr3

where *expr1*, *expr2* and *expr3* are expressions.

The operator *?:* works as follows: *expr1* is evaluated first. If it is nonzero(true), then the expression *expr2* is evaluated and becomes the value of the expression. If *expr1* is false, *expr3* is evaluated and its value

becomes the value of the expression. For example, consider the following statements:

a=100;

b=200;

c=(a>b)?a:b;

In this example, *c* will be assigned the value of *b*. This can be achieved using the *if..else* statements as follows: *if(a>b)*

c=a;

else

c=b;

6. LIBRARY FUNCTIONS

The C language is accompanied by a number of *library functions* or *built in functions* that carry out various commonly used operations or calculations. There are library functions that carry out standard input/output operations, functions that perform operations on characters, functions that perform operations on strings and functions that carry out various mathematical calculations.

Functionally similar library functions are usually grouped together as object programs in separate library files.

A library function is accessed simply by writing the function name, followed by a list of arguments that represent information being passed to the function. A function that returns a data item can appear anywhere within an expression in place of a constant or an identifier. A function that carries out operations on data items but does not return anything can be accessed simply by writing the function name.

A typical set of library functions will include a large number of functions that are common to most C compilers, such as those shown in table 2.1.

Function	Purpose
abs(i)	Return the absolute value of (i is integer)
ceil(d)	Round up to the next integer value(the smallest integer that is greater than or equal to d)
cos(d)	Return the cosine of d
exp(d)	Raise e to the power d(e=Naperian constant)
fabs(d)	Return the absolute value of d(d is double)
floor(d)	Round down to the next integer value(the largest integer that does not exceed d)

getchar()	Enter a character from the standard input device
log(d)	Return the natural logarithm of d
pow(d1,d2)	Return d1 raised to the power d2
putchar(c)	Send a character to the standard output device
rand()	Return a random positive integer
sin(d)	Return sine of d
sqrt(d)	Return the square root of d
tan(d)	Return the tangent of d
toascii(c)	Convert value of argument to ASCII
tolower(c)	Convert letter to lowercase
toupper(c)	Convert letter to uppercase

Table 2.1

Program 2.2: Program to convert lowercase to uppercase

```
#include <stdio.h> /* Input/Output functions are available in stdio.h */
#include <ctype.h> /* Character functions are available in the file ctype.h
*/ main()
/* read a lowercase character and print its uppercase equivalent
*/ {
    int lower, upper;
    lower=getchar();
    upper=toupper(lower);
    putchar(upper);
}
```

Program 2.3: Program to illustrate the use of library functions

```
#include <stdio.h>
#include <ctype.h>
#include <math.h> /* Mathematical functions are available in math. h*/
main()
{
    int i=-10, e=2, d=10;
    float rad=1.57;
    double d1=2.0, d2=3.0;
    printf("%d\n", abs(i));
    printf("%f\n", sin(rad));

    printf("%f\n", cos(rad));
    printf("%f\n", exp(e));
    printf("%d\n", log(d));
    printf("%f\n", pow(d1,d2));
}
```

Execute the above program and observe the result

SELF-ASSESSMENT QUESTIONS - 3

5. _____ are built-in functions that carry out various commonly used operations or calculations
6. Fill in the blanks as indicated by alphabets. The value of (a) floor(5.8), (b) floor(-5.8), (c) ceil(5.8) and (d) ceil(-5.8) are ____ (a) ____, (b) ____, (c) ____ and (d) ____ respectively.



7. BITWISE OPERATORS

The *bitwise* operators `&`, `|`, `^`, and `~` operate on integers thought of as binary numbers or strings of bits. The `&` operator is bitwise AND, the `|` operator is bitwise OR, the `^` operator is bitwise exclusive-OR (XOR), and the `~` operator is a bitwise negation or complement. (`&`, `|`, and `^` are “binary” in that they take two operands; `~` is unary.) These operators let you work with the individual bits of a variable; one common use is to treat an integer as a set of single-bit *flags*. You might define the 3rd bit as the “verbose” flag bit by defining

```
#define VERBOSE 4
```

Then you can “turn the verbose bit on” in an integer variable `flags` by executing

```
flags = flags | VERBOSE;
```

and turn it off with

```
flags = flags & ~VERBOSE;
```

and test whether it's set with

```
if(flags & VERBOSE)
```

The left-shift and right-shift operators `<<` and `>>` let you shift an integer left or right by some number of bit positions; for example, `value << 2` shifts `value` left by two bits.

The **comma operator** can be used to link the related expressions together. The expressions are executed one after the other. The most common use for comma operators is when you want multiple variables controlling a for loop, for example:

```
for(i = 0, j = 10; i < j; i++, j--)
```

SELF-ASSESSMENT QUESTIONS - 4

7. _____ let you work with the individual bits of a variable; one common use is to treat an integer as a set of single-bit flags.
8. Fill in the blanks as indicated by alphabets. If flag1=5, flag2=8, then
(a) flag1&flag2, (b) flag1|flag2, (c) ~flag1 and (d) flag1^flag2 is computed to be
____ (a) ____, ____ (b) ____, ____ (c) ____ and ____ (d) ____ respectively.



8. INCREMENT AND DECREMENT OPERATORS

When we want to add or subtract constant 1 to a variable, C provides a set of shortcuts: the *autoincrement* and *autodecrement* operators. In their simplest forms, they look like this:

`++i` add 1 to i

`--j` subtract 1 from j

These correspond to the forms $i = i + 1$ and $j = j - 1$. They are also equivalent to the short hand forms `i+=1` and `j-=1`. C has a set of 'shorthand' assignment operators of the form:

`v op=exp;`

where v is a variable, exp is an expression and op is a C binary arithmetic operator.

The assignment statement

`v op=exp;`

is equivalent to

`v= v op(exp);`

Example:

`x+=y+1;`

This is same as the statement

`x=x+(y+1);`

The `++` and `--` operators apply to one operand (they're unary operators). The expression `++i` adds 1 to i, and stores the incremented result back in i. This means that these operators don't just compute new values; they also modify the value of some variable. (They share this property -- modifying some variable -- with the assignment operators; we can say that these operators all have *side effects*. That is, they have some effect, on the side, other than just computing a new value.)

The incremented (or decremented) result is also made available to the rest of the expression, so an expression like

```
k = 2 * ++i
```

means “add one to i, store the result back in i, multiply it by 2, and store *that* result in k.” (This is a pretty meaningless expression; our actual uses of ++ later will make more sense.)

Both the ++ and -- operators have an unusual property: they can be used in two ways, depending on whether they are written to the left or the right of the variable they're operating on. In either case, they increment or decrement the variable they're operating on; the difference concerns whether it's the old or the new value that's “returned” to the surrounding expression. The *prefix* form ++i increments i and returns the incremented value. The *postfix* form i++ increments i, but returns the *prior*, non-incremented value. Rewriting our previous example slightly, the expression

```
k = 2 * i++
```

means “take i's old value and multiply it by 2, increment i, store the result of the multiplication in k.”

The distinction between the prefix and postfix forms of ++ and -- will probably seem strained at first, but it will make more sense once we begin using these operators in more realistic situations. For example,

```
a[i] = c;
```

```
i = i + 1;
```

using the ++ operator, we could simply write this as

```
a[i++] = c;
```

We wanted to increment i *after* deciding which element of the array to store into, so the postfix form i++ is appropriate.

Notice that it only makes sense to apply the ++ and -- operators to variables (or to other "containers," such as a[i]). It would be meaningless to say something like

```
1++
```

or

```
(2+3)++
```

The ++ operator doesn't just mean "add one"; it means "add one to a variable" or "*make a variable's* value one more than it was before." But (1+2) is not a variable, it's an expression; so there's no place for ++ to store the incremented result.

Another unfortunate example is

```
i = i++;
```

which some confused programmers sometimes write, presumably because they want to be extra sure that i is incremented by 1. But i++ all by itself is sufficient to increment i by 1; the extra (explicit) assignment to i is unnecessary and in fact counterproductive, meaningless, and incorrect. If you want to increment i (that is, add one to it, and store the result back in i), either use

```
i = i + 1;
```

or

```
i += 1;
```

or

```
++i;
```

or

```
i++;
```

Did it matter whether we used ++i or i++ in this last example? Remember, the difference between the two forms is what value (either the old or the new) is passed on to the

surrounding expression. If there is no surrounding expression, if the ++i or i++ appears all by itself, to increment i and do nothing else, you can use either form; it makes no difference. (Two ways that an expression can appear "all by itself," with "no surrounding expression," are when it is an expression statement terminated by a semicolon, as above, or when it is one of the controlling expressions of a for loop.) For example, both the loops

```
for(i = 0; i < 10; ++i)
```

```
    printf("%d\n", i);
```

and

```
for(i = 0; i < 10; i++)
```

```
    printf("%d\n", i);
```

will behave exactly the same way and produce exactly the same results. (In real code, postfix increment is probably more common, though prefix definitely has its uses, too.)

SELF-ASSESSMENT QUESTIONS - 5

9. Increment and Decrement operators are binary operators. (True/False)
10. The prefix form to increment i is _____ whereas the postfix form to increments i is _____.

9. THE SIZE OF OPERATOR

The sizeof is a compile time operator and, when used with an operand, it returns the number of bytes the operand occupies. The operand may be a variable, a constant or a data type qualifier.

Examples:

```
m=sizeof(sum);
```

```
n=sizeof(long int);
```

```
k=sizeof(235L);
```

The size of operator is normally used to determine the lengths of arrays and structures when their sizes are not known to the programmer. It is also used to allocate memory space dynamically to variables during execution of a program.

Program 2.4: Program to illustrate the use of size of operator

```
#include<stdio.h>
main()
{
    int i=10;
    printf("integer: %d\n", sizeof(i);
}
```

The above program might generate the following output:

```
integer: 2
```

Thus we see that this version of C allocates 2 bytes to each integer quantity.

Program 2.5: Program to illustrate arithmetic operators

```
#include<stdio.h>
main()
{
    int a, b, c, d;
    a=10;
    b=15;
    c=++a-b;
    printf("a=%d b=%d c=%d\n", a, b, c);
    d=b++ +a;
    printf("a=%d b=%d d=%d\n", a, b, d);
    printf("a/b=%d\n", a/b);
    printf("a%%b=%d\n", a%b);
    printf("a*=b=%d\n", a*=b);
    printf("%d\n", (c>d)?1:0);
    printf("%d\n", (c<d)?1:0);
}
```

Execute the above program and observe the result.

10. PRECEDENCE OF OPERATORS

The precedence of C operators dictates the order of evaluation within an expression. The precedence of the operators introduced here is summarised in the Table 2.2. The highest precedence operators are given first.

Operators	Associativity
()->.	left to right
!~+---&*	right to left
* / %	left to right
+ -	left to right
<<=>>=	left to right
== !=	left to right
&	left to right
	left to right
&&	left to right
	right to left
=*/=%+=-=-	right to left

Table 2.2

Where the same operator appears twice (for example *) the first one is the unary version.

Program 2.6: A program to illustrate evaluation of expressions

```
#include<stdio.h>
main()
/* Evaluation of expressions */
{
    float a, b, c, x, y, z;
    a=20;
    b=2;
    c=-23;
    x = a + b / ( 3 + c * 4 - 1);
    y = a - b / (3 + c) * ( 4 - 1);
    z= a - ( b / ( 3 + c ) * 2 ) - 1;
    printf( "x=%f\n", x);
    printf("y=%f\n", y);
    printf("z=%f\n", z);
}
```

Execute the above program and observe the result.

Program 2.7: Program to convert seconds to minutes and seconds #include <stdio.h>

```
#define SEC_PER_MIN 60    // seconds in a minute
int main(void)
{
    int sec, min, left;
    printf("Convert seconds to minutes and seconds!\n");
    printf("Enter the number of seconds you wish to convert.\n");
    scanf("%d", &sec);    *number of seconds is read in
    min = sec / SEC_PER_MIN; *truncated number of minutes
    left = sec % SEC_PER_MIN; *number of seconds left over
```

```
printf("%d seconds is %d minutes, %d seconds.\n", sec, min,  
left);  
return 0;  
}
```

11. SUMMARY

C supports a rich set of operators. An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations. Operators are used in programs to manipulate data and variables. A binary operator acts on two operands. A unary operator acts upon a single operand to produce a new value. Multiplication, division, and modulus all have higher precedence than addition and subtraction. Relational and Boolean expressions are usually used in contexts such as an if statement, where something is to be done or not done depending on some condition. The C language is accompanied by a number of library functions or built-in functions that carry out various commonly used operations or calculations. The **size of** operator is normally used to determine the lengths of arrays and structures when their sizes are not known to the programmer. It is also used to allocate memory space dynamically to variables during execution of a program. Associativity is the order in which consecutive operations within the same precedence group are carried out.

12. TERMINAL QUESTIONS

1. If $i=10$ and $j=12$, what are the values of c and d after executing the following program segment:
 $i++; c=j++ + i; d=++i + j++;$
2. Suppose that x, y and z are integer variables which have been assigned the values 2, 3 and 4, respectively. Evaluate the following expression and determine the value of x .
 $x*=-2 * (y + z) / 3$
3. Suppose that i is an integer variable whose value is 7 and c is a character variable that represents the character 'w', evaluate the following logical expression:
 $(i \geq 6) \&\& (c == 'w')$
4. Suppose that i is an integer variable whose value is 7 and f is a floating -point variable whose value is 8.5. Evaluate the following expression: $(i + f) \% 4$
5. What is meant by associativity?

13. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. 3
2. % (modulus)
3. &&, ||
4. Incorrect
5. Library functions
6. (a) 5, (b) -6, (c) 6, (d) -5
7. Bitwise operator
8. (a) 0, (b) 13, (c) 10, (d) 13
9. False
10. ++i, i++

14. ANSWERS TO TERMINAL QUESTIONS

1. c=23 and d=25
2. -8
3. true
4. Given expression is invalid because a floating point variable can not be used in a modulus operation.
5. Associativity is the order in which consecutive operations within the same precedence group are carried out.

15. EXERCISES

1. Suppose $a=3$, $b=5$, $c=8$ and $d=4$, give the output of the following:
a) $x=a*b-c/d+4$ b) $z=a-b/d*c+10$
2. Suppose $i=5$, $j=8$, $k=10$, then, what is the output of the following:
a) $x=a++-j$ b) $y=k++*j$ —
3. What is the precedence of operators? How expressions are evaluated using the precedences?
4. Suppose $a=7$, $b=11$, find the output of the following:
a) $x=(a>b)?b:a$ b) $x=(a<b)?a:b$
5. Explain the use of bitwise operators with suitable examples.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 5

Flow of Control Part 1

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3 - 4
	1.1 Objectives	-	-	
2	The goto statement	-	1	5 - 7
3	The if statement	-	2	8 - 14
	3.1 The if-else statement	-	-	
	3.2 Nesting of if statements	-	-	
4	The conditional expression	-	-	15
5	The switch statement	-	3	16 - 19
6	Summary	-	-	20
7	Terminal Questions	-	-	20
8	Answers to Self Assessment Questions	-	-	21
9	Answers to Terminal Questions	-	-	21 - 22
10	Exercises	-	-	22

1. INTRODUCTION

In the previous unit, you studied about the data types that are supported in C and the types of Input/output operators available in C. This unit will enable you to use the various types of control statements for making a decision in C. Statements are the “steps” of a program. Most statements compute and assign values or call functions, but we will eventually meet several other kinds of statements as well. By default, statements are executed in sequence, one after another. We can, however, modify that sequence by using *control flow constructs* such that a statement or group of statements is executed only if some condition is true or false. This involves a kind of decision making to see whether a particular condition has occurred or not and then direct the computer to execute certain statements accordingly.

C language possesses such decision making capabilities and supports the following statements known as the control or *decision making statements*.

- **if** statement
- **switch** statement
- **goto** statement
- conditional operator statement

You will also study about the loops in this unit. Loops generally consist of two parts: one or more control expressions which control the execution of the loop, and the body, which is the statement or set of statements which is executed over and over.

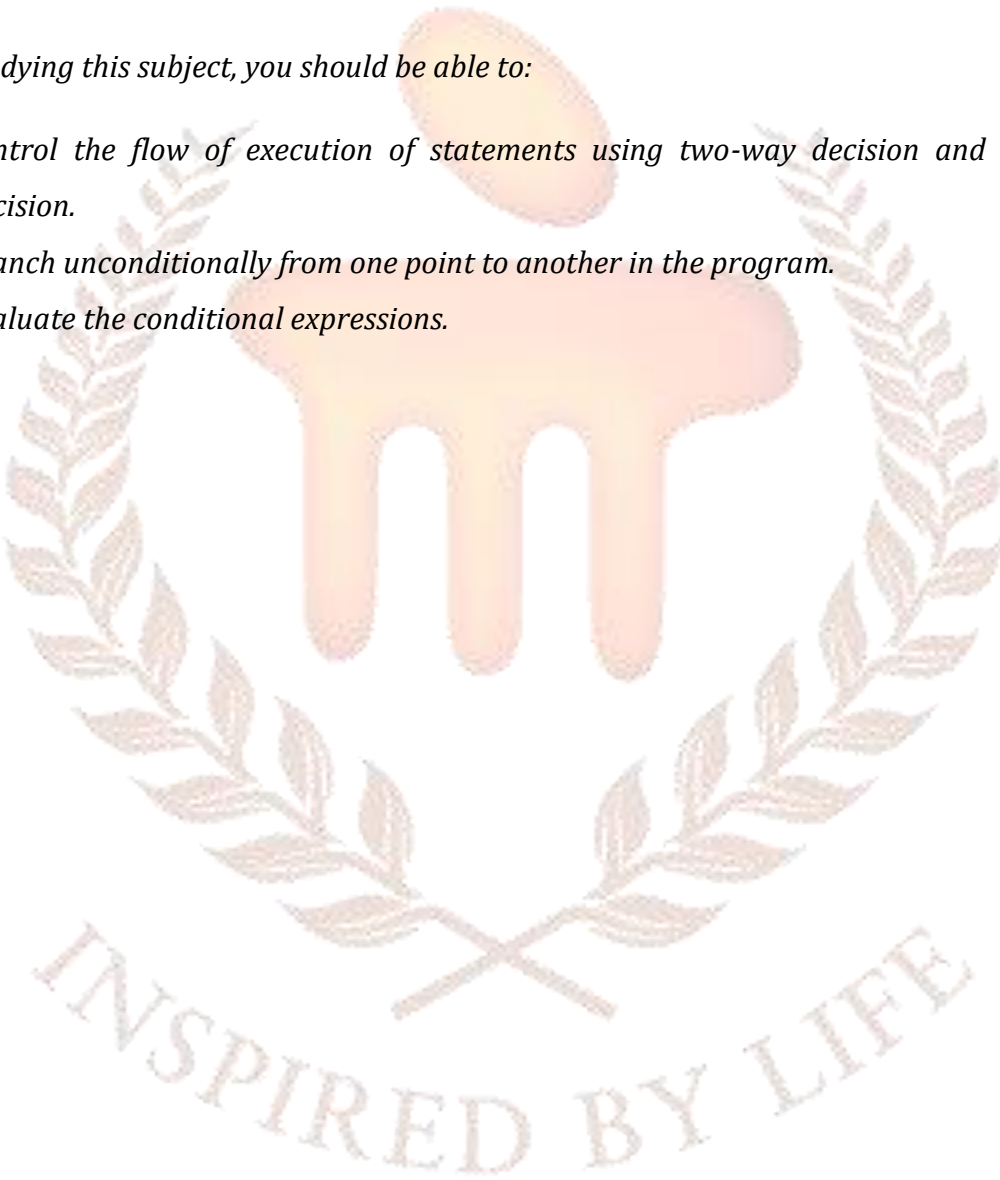
As far as C is concerned, a true/false condition can be represented as an integer. (An integer can represent many values; here we care about only two values: “true” and “false.” The study of mathematics involving only two values is called Boolean algebra, after George Boole, a mathematician who refined this study.) In C, “false” is represented by a value of 0 (zero), and “true” is represented by any value that is nonzero. Since there are many nonzero values (at least 65,534, for values of type int), when we have to pick a specific value for “true,” we’ll pick 1.

Do...while loop is used in a situation where we need to execute the body of the loop before the test is performed. The for loop is used to execute the body of the loop for a specified number of times. The break statement is used to exit any loop.

1.1. Objectives:

After studying this subject, you should be able to:

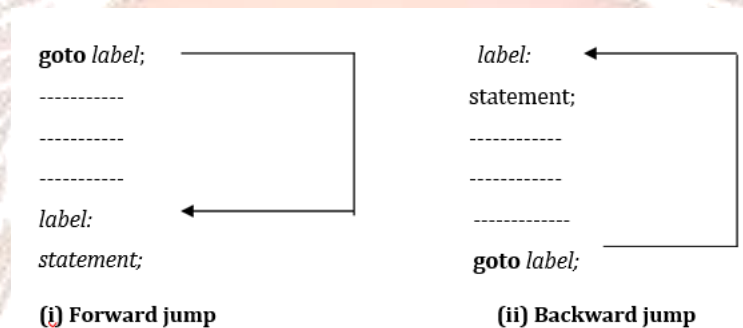
- ❖ *control the flow of execution of statements using two-way decision and multipath decision.*
- ❖ *branch unconditionally from one point to another in the program.*
- ❖ *evaluate the conditional expressions.*



2. THE GOTO STATEMENT

C supports the **goto** statement to branch unconditionally from one point to another in the program. Although it may not be essential to use the **goto** statement in a highly structured language like C, there may be occasions when the use of **goto** might be desirable.

The **goto** requires a *label* in order to identify the place where the branch is to be made. A *label* is any valid variable name, and must be followed by a colon. The *label* is placed immediately before the statement where the control is to be transferred. The general forms of **goto** and *label* statements are shown below:



The *label* can be anywhere in the program either before the **goto** or after the **goto label;** statement.

During execution of the program when a statement like

goto first;

is met, the flow of control will jump to the statement immediately following the label **first**. This happens unconditionally.

Note that a **goto** breaks the normal sequential execution of the program. If the *label* is before the statement `goto label;` a loop will be formed and some statements will be executed repeatedly. Such a jump is known as a *backward jump*. On the other hand, if the *label* is placed after the `goto label;` some statements will be skipped and the jump is known as the *forward jump*.

A **goto** is often used at the end of a program to direct the control to go to the input statement, to read further data. Consider the following example:

Program 5.1: Program showing unconditional branching

```
main()
{
    double a, b;
    read:
    printf("enter the value of a\n");
    scanf("%f", &a);
    if (a<0) goto read;
    b=sqrt(a);
    printf("%f %f \n",a, b);
    goto read;
}
```

This program is written to evaluate the square root of a series of numbers read from the terminal. The program uses two **goto** statements, one at the end, after printing the results to transfer the control back to the input statements and the other to skip any further computation when the number is negative.

Due to the unconditional **goto** statement at the end, the control is always transferred back to the input statement. In fact, this program puts the computer in a permanent loop known as an *infinite* loop.

SELF-ASSESSMENT QUESTIONS - 1

1. The goto requires a _____ in order to identify the place where the branch is to be made.
2. goto is an unconditional branching statement. (True/False)



3. THE IF STATEMENT

The simplest way to modify the control flow of a program is with an if statement, which in its simplest form looks like this:

```
if(x > max)

    max = x;
```

Even if you didn't know any C, it would probably be pretty obvious that what happens here is that if x is greater than max, x gets assigned to max. (We'd use code like this to keep track of the maximum value of x we'd seen--for each new x, we'd compare it to the old maximum value max, and if the new value was greater, we'd update max.)

More generally, we can say that the syntax of an if statement is:

```
if( expression )

    statement
```

where expression is any expression and statement is any statement.

What if you have a series of statements, all of which should be executed together or not at all depending on whether some condition is true? The answer is that you enclose them in braces:

```
if( expression )

{

    statement 1;

    statement 2;

    statement n;

}
```

As a general rule, anywhere the syntax of C calls for a statement, you may write a series of statements enclosed by braces. (You do not need to, and should not, put a semicolon after the closing brace, because the series of statements enclosed by braces is not itself a simple expression statement.)

Program 5.2: Program to calculate the absolute value of an integer

```
# include < stdio.h > void main ( )
{
    int number;
    printf ("Type a number:");
    scanf ("%d", & number);
    if (number < 0) /* check whether the number is a negative number */ number
        = - number; /* If it is negative then convert it into positive. */
    printf ("The absolute value is %d \n", number);
}
```

3.1. The if-else statement

An **if** statement may also optionally contain a second statement, the "**else** clause," which is to be executed if the condition is not met. Here is an example:

```
if(n > 0)
    average = sum / n;
else {
    printf("can't compute average\n");
    average = 0;
}
```

The first statement or block of statements is executed if the condition is true, and the second statement or block of statements (following the keyword **else**) is executed if the condition is not true. In this example, we can compute a meaningful average only if *n* is greater than 0; otherwise, we print a message saying that we cannot compute the average. The general syntax of an **if** statement is therefore

if(expression)

statement(s)

else

statement(s)

(if there are more than one statements, they should be enclosed within braces).

Program 5.3: To find whether a number is negative or positive

```
#include <stdio.h>
void main ( )
{
    int num;
    printf ("Enter the number");
    scanf ("%d", &num);
    if (num < 0)
        printf ("The number is negative")
    else
        printf ("The number is positive");
}
```

3.2. Nesting of *if* statements

It's also possible to nest one **if** statement inside another. (For that matter, it's in general possible to nest any kind of statement or control flow construct within another.) For example, here is a little piece of code which decides roughly which quadrant of the compass you're walking into, based on an x value which is positive if you're walking east, and a y value which is positive if you're walking north:

```
if(x > 0)
{
    if(y > 0)
        printf("Northeast.\n");
    else    printf("Southeast.\n");
}

else {
    if(y > 0)
        printf("Northwest.\n");
    else    printf("Southwest.\n");
}
```

When you have one **if** statement (or loop) nested inside another, it's a very good idea to use explicit braces {}, as shown, to make it clear (both to you and to the compiler) how they're nested and which **else** goes with which **if**. It's also a good idea to indent the various levels, also as shown, to make the code more readable to humans. Why do both? You use indentation to make the code visually more readable to yourself and other humans, but the compiler doesn't pay attention to the indentation (since all whitespace is essentially equivalent and is essentially ignored). Therefore, you also have to make sure that the punctuation is right.

Here is an example of another common arrangement of **if** and **else**. Suppose we have a variable `grade` containing a student's numeric **grade**, and we want to print out the corresponding letter grade. Here is the code that would do the job:

```
if(grade >= 90)
    printf("A");
else if(grade >= 80)
    printf("B");
else if(grade >= 70)
    printf("C");
else if(grade >= 60)
    printf("D");
else    printf("F");
```

What happens here is that exactly one of the five **printf** calls is executed, depending on which of the conditions is true. Each condition is tested in turn, and if one is true, the corresponding statement is executed, and the rest are skipped. If none of the conditions is true, we fall through to the last one, printing "F".

In the cascaded **if/else/if/else/...** chain, each **else** clause is another **if** statement. This may be more obvious at first **if** we reformat the example, including every set of braces and indenting each if statement relative to the previous one:

```
if(grade >= 90)
{
    printf("A");
}
Else {
    if(grade >= 80)
    {
        printf("B");
    }
    else {
        if(grade >= 70)
        {
            printf("C");
        }
    }
}
```

```
        }  
    else {  
        if(grade >= 60)  
        {  
            printf("D");  
        }  
    else {  
        printf("F");  
    }  
    }  
}  
}
```

By examining the code this way, it should be obvious that exactly one of the printf calls is executed, and that whenever one of the conditions is found true, the remaining conditions do not need to be checked and none of the later statements within the chain will be executed. But once you've convinced yourself of this and learned to recognize the idiom, it's generally preferable to arrange the statements as in the first example, without trying to indent each successive if statement one tabstop further out.

Program 5.4: Program to print the largest of three numbers

```
#include<stdio.h>  
main()  
{  
    int a,b,c,big;  
    printf ("Enter three numbers");  
    scanf ("%d %d %d", &a, &b, &c);  
    if (a>b) // check whether a is greater than b if true  
        then if(a>c) // check whether a is greater than c  
            big = a ; // assign a to big  
    else big = c ; // assign c to big
```

```
else if (b>c) // if the condition (a>b) fails check whether b is greater than
    c big = b ; // assign b to big
else big = c ; // assign C to big
printf ("Largest of %d,%d&%d = %d", a,b,c,big);
}
```

SELF-ASSESSMENT QUESTIONS - 2

3. The series of statements enclosed by braces after the expression in simple *if statement* is itself a simple expression statement. **(True/False)**
4. In the cascaded if/else/if/else/... chain, each else clause is another_____ statement.

4. THE CONDITIONAL EXPRESSION

The conditional operator (`?:`) takes three operands. It tests the result of the first operand and then evaluates one of the other two operands based on the result of the first. Consider the following example:

E1?E2:E3

If expression *E1* is nonzero (true), then *E2* is evaluated, and that is the value of the conditional expression. If *E1* is 0 (false), *E3* is evaluated, and that is the value of the conditional expression. Conditional expressions associate from right to left. In the following example, the conditional operator is used to get the minimum of *x* and *y*:

```
a = (x < y) ? x : y;    /* a = min(x, y) */
```

There is a sequence point after the first expression (*E1*). The following example's result is predictable, and is not subject to unplanned side effects:

```
i++ > j ? y[i] : x[i];
```

The conditional operator does not produce a lvalue. Therefore, a statement such as

a ? x : y = 10 is not valid.

5. THE SWITCH STATEMENT

The **switch case** statements are a substitute for long **if** statements that compare a variable to several "integral" values ("integral" values are simply values that can be expressed as an integer, such as the value of a **char**). The basic format for using **switch case** is outlined below. The value of the variable given into **switch** is compared to the value following each of the cases, and when one value matches the value of the variable, the computer continues executing the program from that point. `switch ( <variable> ) {`

`case this-value:`

`Code to execute if <variable> == this-value`

`break;`

`case that-value:`

`Code to execute if <variable> == that-value`

`break;`

`...`

`default:`

`Code to execute if <variable> does not equal the value following any of the cases`

`break;`

`}`

The condition of a **switch** statement is a value. The **case** says that if it has the value of whatever is after that **case** then do whatever follows the colon. The **break** is used to break out of the case statements. **break** is a keyword that **breaks** out of the code block, usually surrounded by braces, which it is in. In this case, **break** prevents the program from falling through and executing the code in all the other **case** statements. An important thing to note about the **switch** statement is that the **case** values may only be constant integral expressions. It isn't legal to use **case** like this: `int a = 10;`

`int b = 10;`

`int c = 20;`

`switch ( a ) {`

`case b:`

`/* Code */`

```
break;
case c:
/* Code */
break;
default:
/* Code */
break;
}
```

The **default** case is optional, but it is wise to include it as it handles any unexpected cases. It can be useful to put some kind of output to alert you to the code entering the **default** case if you don't expect it to. Switch statements serve as a simple way to write long if statements when the requirements are met. Often it can be used to process input from a user.

Example: Below is a sample program, in which not all of the proper functions are actually declared, but which shows how one would use switch in a program.

```
#include <stdio.h>
void playgame();
void loadgame();
void playmultiplayer();
int main()
{
    int input;
    printf( "1. Play game\n" );
    printf( "2. Load game\n" );
    printf( "3. Play multiplayer\n" );
    printf( "4. Exit\n" );
    printf( "Selection: " );
    scanf( "%d", &input );
```

```
switch ( input ) {  
    case 1: /* Note the colon, not a semicolon */  
        playgame();  
        break;  
    case 2:  
        loadgame();  
        break;  
    case 3:  
        playmultiplayer();  
        break;  
    case 4:  
        printf( "Thanks for playing!\n" );  
        break;  
    default:  
        printf( "Bad input, quitting!\n" );  
        break;  
}  
    getchar();  
}
```

This program will compile, but cannot be run until the undefined functions are given bodies, but it serves as a model (albeit simple) for processing input. **If** you do not understand this then try mentally putting in if statements for the case statements. **default** simply skips out of the **switch case** construction and allows the program to terminate naturally. If you do not like that, then you can make a loop around the whole thing to have it wait for valid input. You could easily make a few small functions if you wish to test the code.

SELF-ASSESSMENT QUESTIONS - 3

5. The conditional operator does not produce a lvalue. (True/False)
6. The condition of a switch statement is a ____.
7. Switch statement is an unconditional branching statement. (True/False)



6. SUMMARY

In C by default, statements are executed in sequence, one after another. We can, however, modify that sequence by using **control flow constructs**. C language possesses decision making capabilities and supports the following statements known as the control or *decision making* statements: **goto**, **if**, **switch**. The **goto** statement is used to branch unconditionally from one point to another in the program. The simplest way to modify the control flow of a program is with an **if** statement. **switch** statements serve as a simple way to write long **if** statements when the requirements are met.

7. TERMINAL QUESTIONS

1. Consider the following program segment:

```
x=1;
y=1;
if (n>0)
    x=x+1;
    y=y-1;
    printf("%d %d",x,y);
```

What will be the values of x and y if n assumes a value of (a) 1 and (b) 0.

2. Rewrite the following without using compound relation: if (grade<=59 && grade>=50)
second = second +1;
3. Write a program to check whether an input number is odd or even.

8. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. label
2. true
3. false
4. if
5. true
6. value
7. false

9. ANSWERS TO TERMINAL QUESTIONS

1. (a) $x=2, y=0$

(b) $x=1, y=0$

2. `if (grade<=59)`

`if (grade>=50)`

`second = second+1;`

3. `void main()`

`{`

`int no;`

`printf("enter a number\n"); scanf("%d",&no);`

`if (no%2==0)`

`printf("even number\n");`

`else printf("odd number\n");`

`}`

4. 0 5 10 15

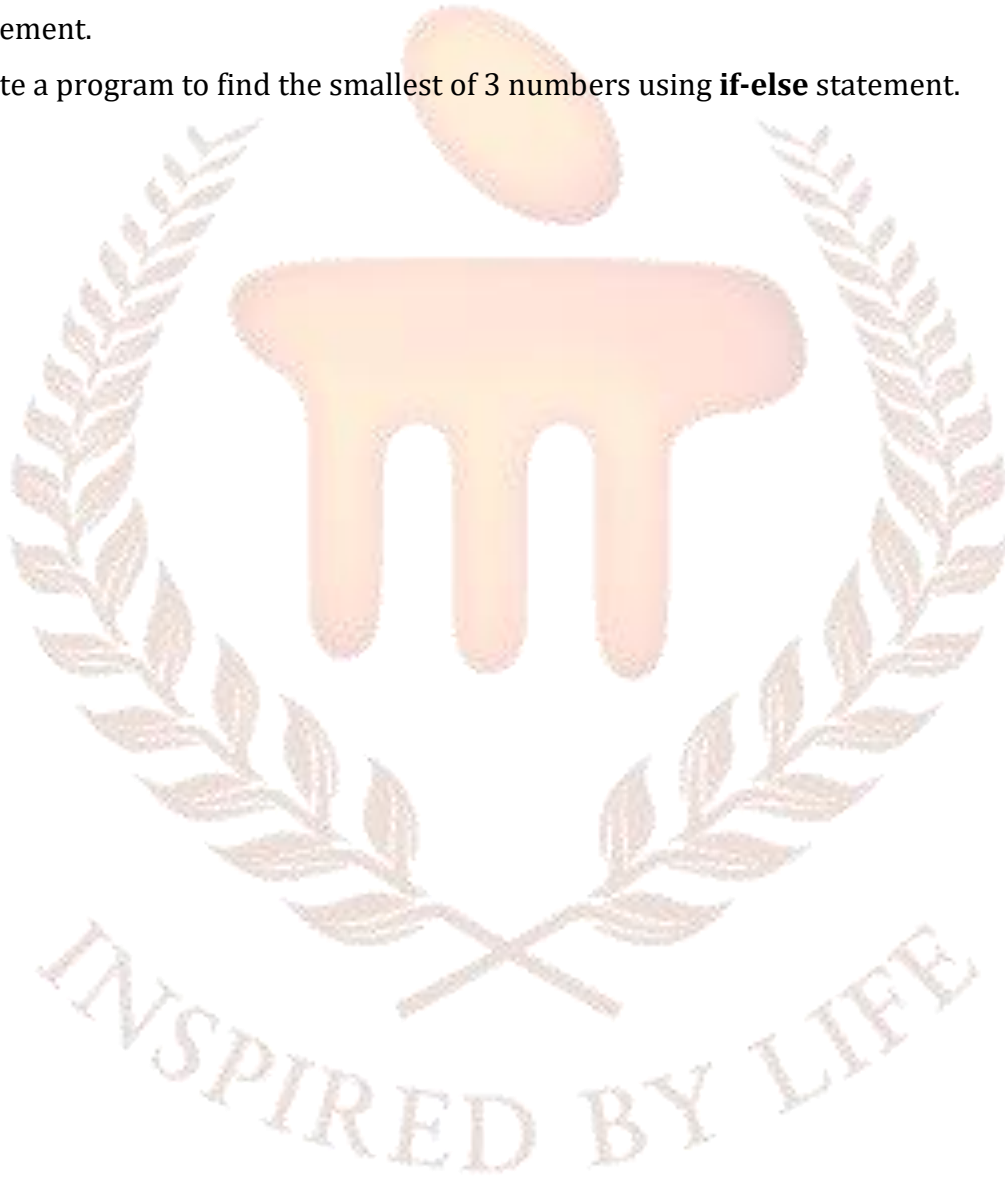
$x = 30$

5. 1 2 3 4

$x = 4$

10. EXERCISES

1. Explain different types of **if** statements with examples.
2. Explain the syntax of **switch** statement with an example.
3. Write a program to check whether a given number is odd or even using **switch** statement.
4. Write a program to find the smallest of 3 numbers using **if-else** statement.





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 6

Flow of Control Part 2

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	The While Loop	-	1	4-7
3	The Do...While Loop	-	2	7-9
4	THE For LOOP	-	-	10-15
	4.1 The Nesting Of For Loops	-	3	
5	The Break Statement And Continue Statement	-	-	15-17
6	Summary	-	-	18
7	Terminal Questions	-	-	19-20
8	Answers To Self Assessment Questions	-	-	20
9	Answers To Terminal Questions	-	-	20
10	Exercises	-	-	20-21

1. INTRODUCTION

In the prior unit, you explored decision control statements in C. This unit focuses on utilizing loops in C, pivotal for automating repetitive tasks. C offers three main loop types: 'for', 'while', and 'do-while', each with distinct applications. The 'for' loop is ideal for predetermined iterations, 'while' for indefinite conditions, and 'do-while' for executing at least once. Through this unit, you'll master loop syntax, understand optimal use cases, and explore advanced concepts like nested loops. Do...while loop is used in a situation where we need to execute the body of the loop before the test is performed. The for loop is used to execute the body of the loop for a specified number of times. The break statement is used to exit any loop. In C programming, the continue statement is used within loops to skip the remaining code within the current iteration of the loop and proceed directly to the next iteration.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Understand the basic concepts of loops in C programming.*
- ❖ *Learn to implement 'for', 'while', and 'do-while' loops in C.*
- ❖ *Explore common use cases and best practices for loop usage.*

2. THE WHILE LOOP

Loops generally consist of two parts: one or more control expressions which control the execution of the loop, and the body, which is the statement or set of statements which is executed over and over.

The most basic loop in C is the while loop. A while loop has one control expression, and executes as long as that expression is true. Here before executing the body of the loop, the condition is tested. Therefore it is called an entry-controlled loop. The following example repeatedly doubles the number 2 (2, 4, 8, 16, ...) and prints the resulting numbers as long as they are less than 1000:

```
int x = 2;
while(x < 1000)
{
    printf("%d\n", x);
    x = x * 2;
}
```

(Once again, we've used braces {} to enclose the group of statements which are to be executed together as the body of the loop.)

The general syntax of a **while** loop is

```
while( expression )
```

```
statement(s)
```

A **while** loop starts out like an if statement: if the condition expressed by the expression is true, the statement is executed. However, after executing the statement, the condition is tested again, and if it's still true, the statement is executed again. (Presumably, the condition depends on some value which is changed in the body of the loop.) As long as the condition

remains true, the body of the loop is executed over and over again. (If the condition is false right at the start, the body of the loop is not executed at all.)

As another example, if you wanted to print a number of blank lines, with the variable `n` holding the number of blank lines to be printed, you might use code like this:

```
while(n > 0)
{
    printf("\n");
    n = n - 1;
}
```

After the loop finishes (when control “falls out” of it, due to the condition being false), `n` will have the value 0.

You use a while loop when you have a statement or group of statements which may have to be executed a number of times to complete their task. The controlling expression represents the condition “the loop is not done” or “there's more work to do.” As long as the expression is true, the body of the loop is executed; presumably, it makes at least some progress at its task. When the expression becomes false, the task is done, and the rest of the program (beyond the loop) can proceed. When we think about a loop in this way, we can see an additional important property: if the expression evaluates to “false” before the very first trip through the loop, we make zero trips through the loop. In other words, if the task is already done (if there's no work to do) the body of the loop is not executed at all. (It's always a good idea to think about the “boundary conditions” in a piece of code, and to make sure that the code will work correctly when there is no work to do, or when there is a trivial task to do, such as sorting an array of one number. Experience has shown that bugs at boundary conditions are quite common.)

Program 1: Program to find largest of n numbers

```
main()
{
    int num, large, n, i;
    clrscr();
    printf("enter number of numbers \n");
    scanf("%d",&n);
    large=0;
    i=0;
    while(i<n)
    {
        printf("\n enter number ");
        scanf("%d", &num);
        if(large<num)
            large=num;
        i++;
    }

    printf("\n large = %d", large);
}
```

Program 2: Program to evaluate sine series $\sin(x)=x-x^3/3!+x^5/5!-x^7/7!+-----$ depending on accuracy

```
#include<stdio.h>
#include <math.h> void main()
{
    int n, i=1,count;
    float acc, x, term, sum;
```

```
printf("enter the angle\n");
scanf("%d", &x); x=x*3.1416/180.0;
printf("\nenter the accuracy");
scanf("%f", &acc);
sum=x;
term=x;
while ((fabs(term))>acc)
{
    term=-term*x*x/((2*i)*(2*i+1)); sum+=term;
    i++;
}
printf("\nsum of sine series is %f", sum);
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. A _____ loop starts out like an if statement .
2. while is an entry-controlled loop. (True/False)

3. THE DO...WHILE LOOP

The do...while loop is used in a situation where we need to execute the body of the loop before the test is performed. Therefore, the body of the loop may not be executed at all if the condition is not satisfied at the very first attempt. Where as while loop makes a test of condition before the body of the loop is executed.

For above reasons while loop is called an entry-controlled loop and do..while loop is called an exit-controlled loop.

do while loop takes the following form:

```
do
{
Body of the loop
}
while ( expression);
```

On reaching the do statement , the program proceeds to evaluate the body of the loop first. At the end of the loop, the conditional expression in the while statement is evaluated. If the expression is true, the program continues to evaluate the body of the loop once again. This process continues as long as the expression is true. When the expression becomes false, the loop will be terminated and the control goes to the statement that appears immediately after the while statement.

On using the do loop, the body of the loop is always executed at least once irrespective of the expression.

Program 6.3: A program to print the multiplication table from 1 x 1 to 10 x 10 as shown below using do-while loop.

1	2	3	4		10
2	4	6	8		20
3	6	9	12		30
4					40
.					
.					
.					
10					100

Program to print multiplication table main()

```
{
    int
    rowmax=10,colmax=10,row,col,x;
    printf(" Multiplication table\n");
    printf(" ..... \n");
    row=1;
do
{
    col=1
    ; do
    {
        x=row*col;
        printf("%4d",
        x); col=col+1;
    }
    while
    (col<=colmax);
    printf("\n");
    row=row+1;
}
while(row<=rowmax);
Printf(" ..... \
n");
}
```

SELF-ASSESSMENT QUESTIONS - 2

3. On using the _____, the body of the loop is always executed at least once irrespective of the expression.
4. do...while is an entry-controlled loop. (True/False)

4. THE for LOOP

The for loop is used to repeat the execution of set of statements for a fixed number of times. The for loop is also an entry-controlled loop.

Generally, the syntax of a for loop is

```
for(expr1; expr2; expr3) statement(s)
```

(Here we see that the for loop has three control expressions. As always, the statement can be a brace-enclosed block.)

Many loops are set up to cause some variable to step through a range of values, or, more generally, to set up an initial condition and then modify some value to perform each succeeding loop as long as some condition is true. The three expressions in a for loop encapsulate these conditions: expr1 sets up the initial condition, expr 2 tests whether another trip through the loop should be taken, and expr3 increments or updates things after each trip through the loop and prior to the next one. Consider the following :

```
for (i = 0; i < 10; i = i + 1)
    printf("i is %d\n", i);
```

In the above example, we had $i = 0$ as expr1, $i < 10$ as expr2, $i = i + 1$ as expr3, and the call to printf as statement, the body of the loop. So the loop began by setting i to 0, proceeded as long as i was less than 10, printed out i 's value during each trip through the loop, and added 1 to i between each trip through the loop.

When the compiler sees a for loop, first, expr1 is evaluated. Then, expr2 is evaluated, and if it is true, the body of the loop (statement) is executed. Then, expr3 is evaluated to go to the next step, and expr2 is evaluated again, to see if there is a next step. During the execution of a for loop, the sequence is:

```
expr1
expr2
statement
expr3
expr2
statement
expr3
...
expr2
statement
expr3
expr2
```

The first thing executed is `expr1`. `expr3` is evaluated after every trip through the loop. The last thing executed is always `expr2`, because when `expr2` evaluates false, the loop exits.

All three expressions of a for loop are optional. If you leave out `expr1`, there simply is no initialization step, and the variable(s) used with the loop had better have been initialized already. If you leave out `expr2`, there is no test, and the default for the for loop is that another trip through the loop should be taken (such that unless you break out of it some other way, the loop runs forever). If you leave out `expr3`, there is no increment step.

The semicolons separate the three controlling expressions of a for loop. (These semicolons, by the way, have nothing to do with statement terminators.) If you leave out one or more of the expressions, the semicolons remain. Therefore, one way of writing a deliberately infinite loop in C is

```
for(;;)
```

```
...
```

It's also worth noting that a for loop can be used in more general ways than the simple, iterative examples we've seen so far. The "control variable" of a for loop does not have to be an integer, and it does not have to be incremented by an additive increment. It could be "incremented" by a multiplicative factor (1, 2, 4, 8, ...) if that was what you needed, or it could be a floating-point variable, or it could be another type of variable which we haven't met yet which would step, not over numeric values, but over the elements of an array or other data structure. Strictly speaking, a for loop doesn't have to have a "control variable" at all; the three expressions can be anything, although the loop will make the most sense if they are related and together form the expected initialize, test, increment sequence.

The powers-of-two example using for is:

```
int x;  
for(x = 2; x < 1000; x = x * 2)  
printf("%d\n", x);
```

There is no earth-shaking or fundamental difference between the while and for loops. In fact, given the general for loop

```
for(expr1; expr2; expr3)  
    statement
```

you could usually rewrite it as a while loop, moving the initialize and increment expressions to statements before and within the loop:

```
expr1;  
while(expr2)  
    {
```

```
        statement
        expr3;
    }
```

Similarly, given the general while loop

```
while(expr)
```

```
    statement
```

you could rewrite it as a for loop:

```
for(; expr; )
```

```
    statement
```

Another contrast between the for and while loops is that although the test expression (expr2) is optional in a for loop, it is required in a while loop. If you leave out the controlling expression of a while loop, the compiler will complain about a syntax error. (To write a deliberately infinite while loop, you have to supply an expression which is always nonzero. The most obvious one would simply be while(1) .)

If it's possible to rewrite a **for** loop as a **while** loop and vice versa, why do they both exist? Which one should you choose? In general, when you choose a for loop, its three expressions should all manipulate the same variable or data structure, using the initialize, test, increment pattern. If they don't manipulate the same variable or don't follow that pattern, wedging them into a **for** loop buys nothing and a **while** loop would probably be clearer. (The reason that one loop or the other can be clearer is simply that, when you see a **for** loop, you expect to see an idiomatic initialize/ test/increment of a single variable, and if the **for** loop you're looking at doesn't end up matching that pattern, you've been momentarily misled.)

Program 3: A Program to find the factorial of a number

```
void main()
{
    int M,N;
    long int F=1;
```

```
clrscr();
printf("enter the number\n");
scanf("%d",&N);
if(N<=0)
F=1;
else
{
    for(M=1;M<=N;M++)
        F*=M;
}
printf("the factorial of the number is %ld",F);
getch();
}
```

SELF-ASSESSMENT QUESTIONS - 3

5. for loop is an exit-controlled loop. (True/False)
6. The "control variable" of a for loop does not have to be an integer. (True/False)

4.1. The Nesting of for loops

Nesting of for loops, that is, one for statement within another for statement, is allowed in C. For example, two loops can be nested as follows:

```
.....
.....
for(i=1;i<10;i++)
{
    .....
    .....
}
```

```
for(j=1;j<5;j++)  
{  
.....  
.....  
}  
.....  
.....  
}  
.....  
.....
```

5. THE BREAK STATEMENT AND CONTINUE STATEMENT

The purpose of break statement is to break out of a loop (while, do while, or for loop) or a switch statement. When a break statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop. When the loops are nested, the break would only exit from the loop containing it. That is, the break would exit only a single loop.

Syntax : break;

Program 4: Program to illustrate the use of break statement.

```
void main ( )  
{  
int x;  
for (x=1; x<=10; x++)  
{  
    if (x==5)
```

```
        break; /*break loop only if x==5 */
    printf("%d", x);
}
printf ("\nBroke out of loop");
printf( "at x =%d");
}
```

The above program displays the numbers from 1 to 4 and prints the message "Broke out of loop" when 5 is encountered.

The continue statement

The **continue** statement is used to continue the next iteration of the loop by skipping a part of the body of the loop (for, **do/while** or **while** loops). The continue statement does not apply to a **switch**, like a **break** statement.

Unlike the **break** which causes the loop to be terminated, the continue, causes the loop to be continued with the next iteration after skipping any statements in between.

Syntax: continue;

Program 5: Program to illustrate the use of continue statement.

```
void main ( ) {
    int x;
    for (x=1; x<=10; x++)
    { if (x==5)
        continue; /* skip remaining code in loop
        only if x == 5 */
        printf ("%d\n", x);
    }
    printf("\nUsed continue to skip");
}
```

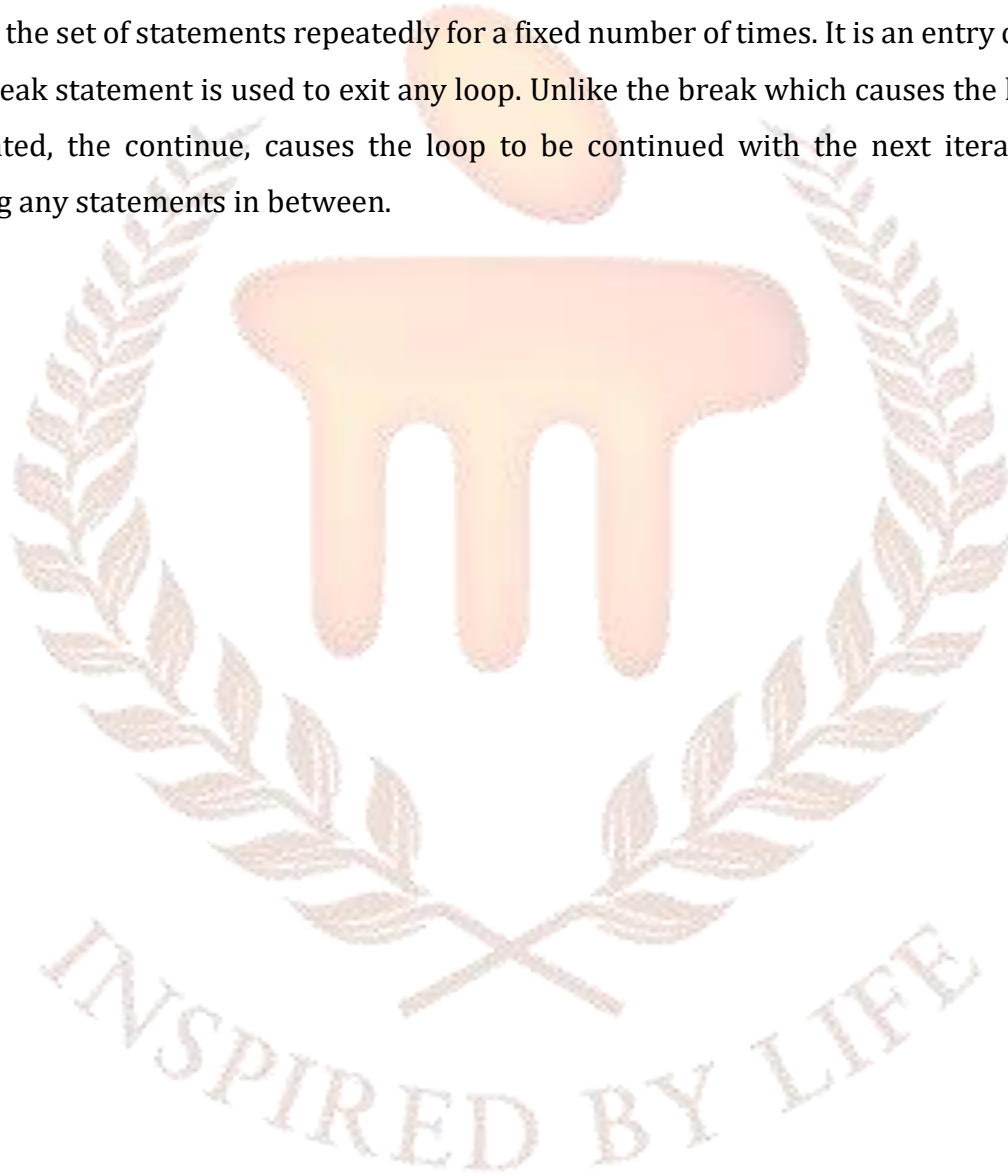
The above program displays the numbers from 1 to 10, except the number 5.

Program 6: Program to sum integers entered interactively

```
#include <stdio.h>
int main(void)
{
    long num;
    long sum = 0L;      /* initialize sum to zero */
    int status;
    printf("Please enter an integer to be summed. ");
    printf("Enter q to quit.\n"); status = scanf("%ld", &num);
    while (status == 1)
    {
        sum = sum + num;
        printf("Please enter next integer to be summed. ");
        printf("Enter q to quit.\n");
        status = scanf("%ld", &num);
    }
    printf("Those integers sum to %ld.\n", sum);
    return 0;
}
```

6. SUMMARY

The most basic loop in C is the while loop. A while loop has one control expression, and executes as long as that expression is true. do..while loop is used in a situation where we need to execute the body of the loop before the test is performed. The for loop is used to execute the set of statements repeatedly for a fixed number of times. It is an entry controlled loop. break statement is used to exit any loop. Unlike the break which causes the loop to be terminated, the continue, causes the loop to be continued with the next iteration after skipping any statements in between.



7. TERMINAL QUESTIONS

1. Write the **output** that will be generated by the following C program:

```
void main()
{
    int i=0, x=0;
    while (i<20)
    {
        if (i%5 == 0)
        {
            x+=i;
            printf("%d\t", i);
        }
        i++;
    }
    printf("\nx=%d", x);
}
```

2. Write the **output** that will be generated by the following C program:

```
void main()
{
    int i=0, x=0;
    do
    {
        if (i%5 == 0)
        {
```

```
        x++;  
        printf("%d\t", x);  
    }  
    ++i;  
} while (i<20);  
printf("\nx=%d", x);  
}
```

8. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. while
2. true
3. do...while
4. false
5. false
6. true

9. ANSWERS TO TERMINAL QUESTIONS

1. 0 5 10 15

x = 30

2. 1 2 3 4

x = 4

10. EXERCISES

1. Write a program to compute the sum of digits of a given number using while loop.
2. Write a program that will read a positive integer and determine and print its binary equivalent using do...while loop.
3. The numbers in the sequence

1 1 2 3 5 8 13.....

are called Fibonacci numbers. Write a program using **do...while** loop to calculate and print the first n Fibonacci numbers.





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 7

Arrays Part 1

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	One Dimensional Arrays	-	-	4-9
	2.1 Passing Arrays To Functions	-	1	
3	Summary	-	-	10
4	Terminal Questions	-	-	11
5	Answers To Self Assessment Questions	-	-	11
6	Answers To Terminal Questions	-	-	11
7	Exercises	-	-	12

1. INTRODUCTION

In the previous unit, you studied about the various types of storage classes that are used in C. You studied how those storage classes are used in different situations in C. In this unit, you will study about the arrays and strings. You will learn how arrays and strings are formed and manipulated.

Many applications require processing of multiple data items that have common characteristics. In such situations it is always convenient to place the data items into an array, where they will share the same name. An array is a collection of similar type of elements. All elements in the array are referred with the array name. Since arrays hold a group of data, it is very easy to perform looping and arithmetic operations on group of data. This chapter covers the processing of both one-dimensional and two-dimensional arrays.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Declare, initialize and process one-dimensional*
- ❖ *Array Initialization with Declaration*
- ❖ *Array Initialization Using Loops*

2. ONE DIMENSIONAL ARRAYS

So far, we've been declaring simple variables: the declaration

```
int i;
```

declares a single variable, named `i`, of type **int**. It is also possible to declare an array of several elements. The declaration

```
int a[10];
```

declares an array, named `a`, consisting of ten elements, each of type `int`. Simply speaking, an array is a variable that can hold more than one value. You specify which of the several values you're referring to at any given time by using a numeric subscript. (Arrays in programming are similar to vectors or matrices in mathematics.) We can represent the array `a` above with a picture like this:

`a`:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

In C, arrays are zero-based: the ten elements of a 10-element array are numbered from 0 to 9. The subscript which specifies a single element of an array is simply an integer expression in square brackets. The first element of the array is `a[0]`, the second element is `a[1]`, etc. You can use these "array subscript expressions" anywhere you can use the name of a simple variable, for example:

```
a[0] = 10;
```

```
a[1] = 20;
```

```
a[2] = a[0] + a[1];
```

Notice that the subscripted array references (i.e. expressions such as `a[0]` and `a[1]`) can appear on either side of the assignment operator.

The subscript does not have to be a constant like 0 or 1; it can be any integral expression. For example, it's common to loop over all elements of an array:

```
int i;  
for(i = 0; i < 10; i = i + 1)  
    a[i] = 0;
```

This loop sets all ten elements of the array a to 0

Arrays are a real convenience for many problems, but there is not a lot that C will do with them for you automatically. In particular, you can neither set all elements of an array at once nor assign one array to another; both of the assignments

```
a = 0;                                /* WRONG */  
and  
int b[10];  
b = a;                                /* WRONG */  
are illegal.
```

To set all of the elements of an array to some value, you must do so one by one, as in the loop example above. To copy the contents of one array to another, you must again do so one by one:

```
int b[10];  
for(i = 0; i < 10; i = i + 1)  
    b[i] = a[i];
```

Remember that for an array declared

```
int a[10];
```

there is no element a[10]; the topmost element is a[9]. This is one reason that zero-based loops are also common in C. Note that the for loop

```
for(i = 0; i < 10; i = i + 1)
```

.....

does just what you want in this case: it starts at 0, the number 10 suggests (correctly) that it goes through 10 iterations, but the less-than comparison means that the last trip through the loop has i set to 9. (The comparison `i <= 9` would also work, but it would be less clear and therefore poorer style.)

In the little examples so far, we've always looped over all 10 elements of the sample array `a`. It's common, however, to use an array that's bigger than necessarily needed, and to use a second variable to keep track of how many elements of the array are currently in use. For example, we might have an integer variable

```
int na;          /* number of elements of a[] in use */
```

Then, when we wanted to do something with `a` (such as print it out), the loop would run from 0 to `na`, not 10 (or whatever `a`'s size was):

```
for(i = 0; i < na; i = i + 1)
    printf("%d\n", a[i]);
```

Naturally, we would have to ensure that `na`'s value was always less than or equal to the number of elements actually declared in `a`.

Arrays are not limited to type `int`; you can have arrays of `char` or `double` or any other type.

Here is a slightly larger example of the use of arrays. Suppose we want to investigate the behavior of rolling a pair of dice. The total roll can be anywhere from 2 to 12, and we want to count how often each roll comes up. We will use an array to keep track of the counts: `a[2]` will count how many times we've rolled 2, etc.

We'll simulate the roll of a die by calling C's random number generation function, `rand()`. Each time you call `rand()`, it returns a different, pseudo-random integer. The values that `rand()` returns typically span a large range, so we'll use C's modulus (or "remainder") operator `%` to produce random numbers in the range we want. The expression `rand() % 6` produces random numbers in the range 0 to 5, and `rand() % 6 + 1` produces random numbers in the range 1 to 6.

Program 2.1: Program to simulate the roll of a die

```
#include <stdio.h>
#include <stdlib.h>
main()
{
    int i;
    int d1, d2;
    int a[13]; /* uses [2..12] */

    for(i = 2; i <= 12; i = i + 1)
        a[i] = 0;
    for(i = 0; i < 100; i = i + 1)
    {
        d1 = rand() % 6 + 1;
        d2 = rand() % 6 + 1;
        a[d1 + d2] = a[d1 + d2] + 1;
    }
    for(i = 2; i <= 12; i = i + 1)
        printf("%d: %d\n", i, a[i]);
    return 0;
}
```

We include the header `<stdlib.h>` because it contains the necessary declarations for the `rand()` function. We declare the array of size 13 so that its highest element will be `a[12]`. (We're wasting `a[0]` and `a[1]`; this is no great loss.) The variables `d1` and `d2` contain the rolls of the two individual dice; we add them together to decide which cell of the array to increment, in the line

```
a[d1 + d2] = a[d1 + d2] + 1;
```

After 100 rolls, we print the array out. Typically, we'll see mostly 7's, and relatively few 2's and 12's.

2.1. Passing Arrays to Functions

An array name can be used as an argument to a function, thus permitting the entire array to be passed to the function. To pass an array to a function, the array name must appear by itself, without brackets or subscripts, as an actual argument within the function call. The corresponding formal argument is written in the same manner, though it must be declared as an array within the formal argument declarations. When declaring a one-dimensional array as a formal argument, the array name is written with a pair of empty square brackets. The size of the array is not specified within the formal argument declaration.

Program 2.2: The following program illustrates the passing of an array from the main to a function. This program is used to find the average of n floating point numbers.

```
#include<stdio.h>
main()
{
    int n, i;
    float avg;
    float list[100];

    float average(int, float[]); /* function prototype */
    printf("How many numbers:");
    scanf("%d",&n);
    printf(" Enter the numbers:");
    for(i=1;i<=n;i++)
        scanf("%f", &list[i]);
    avg=average(n, list); /* Here list and n are actual arguments */
    printf("Average=%f\n", avg);
```

```
}  
float average(int a, float x[ ])  
{  
    float avg;  
    float sum=0;  
    int i;  
    for(i=0;i<a;i++)  
        sum=sum+x[i]; /* find sum of all the numbers */  
    avg=sum/a; /* find average */  
    return avg;  
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. In C, an array subscript starts from _____
2. An array name is a pointer. (True/False)
3. Will there be a compilation error for the following program segment?(Yes/No)

```
int a[5] = {1, 2, 3, 4, 5};
```

```
int b[5] = {5, 4, 3, 2, 1};
```

```
int c[5][5];
```

```
...
```

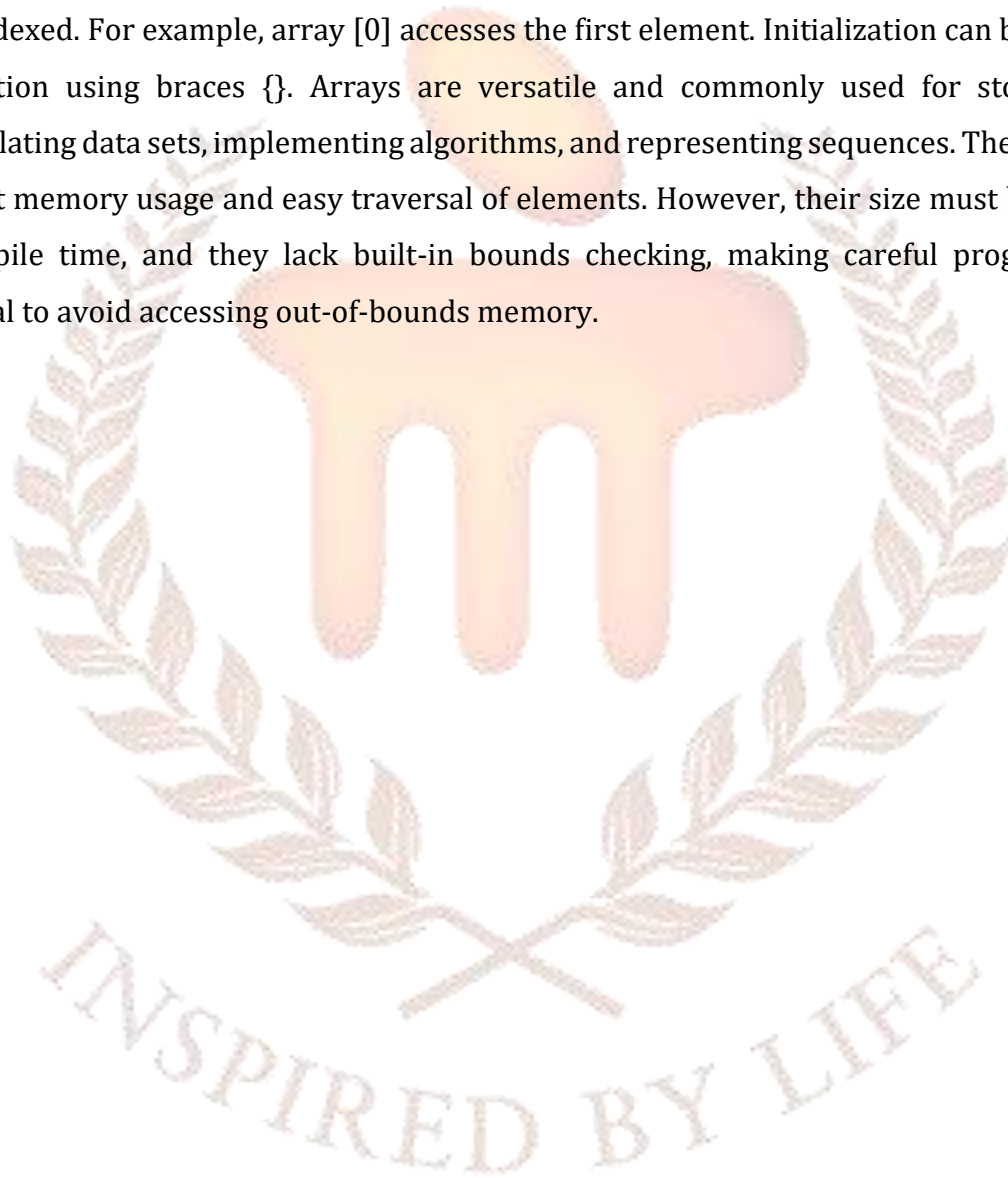
```
c=a+b;
```

```
...
```

4. What is the correct way to declare a 1D array in C?
 - (a) int array[10];
 - (b) int array[] = {1, 2, 3, 4, 5};
 - (c) int array() = {1, 2, 3, 4, 5};
 - (d) int array{} = {1, 2, 3, 4, 5};

3. SUMMARY

An array is a variable that can hold more than one value. In C, arrays are zero-based. An array name can be used as an argument to a function, thus permitting the entire array to be passed to the function. The C language allows arrays of any dimension to be defined. Arrays in C are zero-indexed. For example, array [0] accesses the first element. Initialization can be done at declaration using braces {}. Arrays are versatile and commonly used for storing and manipulating data sets, implementing algorithms, and representing sequences. They provide efficient memory usage and easy traversal of elements. However, their size must be known at compile time, and they lack built-in bounds checking, making careful programming essential to avoid accessing out-of-bounds memory.



4. TERMINAL QUESTIONS

1. Write a program for 10 times summation of square of a number.
2. Write a Program that uses loops for array processing.

5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. 0
2. True
3. Yes
4. (a) `int array[10];`

6. ANSWERS TO TERMINAL QUESTIONS

1.

```
#include<stdio.h>

main()
{
    int i=0, sum=0, x;
    printf('Enter a number:');
    scanf("%d", &x);
    while(i<10)
    {
        sum+=x*x;
        i++;
    }
    printf("Sum=%d", sum);
}
```
2.

```
// loops for array processing #include <stdio.h>

#define SIZE 10
#define PAR 72
int main(void)
{
```

```
int index, score[SIZE]; int sum = 0;
float average;
printf("Enter %d golf scores:\n", SIZE);
for (index = 0; index < SIZE; index++)
{
    scanf("%d", &score[index]); /*read in the ten scores
    printf("The scores read in are as follows:\n");
    for (index = 0; index < SIZE; index++)
        printf("%5d", score[index]); /*verify input
    printf("\n");
}
for (index = 0; index < SIZE; index++)
{
    sum += score[index]; /*add them up
    average = (float) sum / SIZE; /* time-honored method
    printf("Sum of scores = %d, average = %.2f\n", sum, average);
    printf("That's a handicap of %.0f.\n", average - PAR);
    return 0;
}
}
```

7. EXERCISES

1. Write a program to count the number of vowels and consonants in a given string.
2. Write a c program Maximum Element in Array.
3. Write a c program to reverse array elements.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 8

Array Part 2

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Sorting	-	-	4-8
	2.1 Bubble Sort	-	-	
	2.2 Selection Sort	-	-	
	2.3 Insertion Sort	-	1	
3	Multidimensional Arrays	-	2	9-13
4	Summary	-	-	14
5	Terminal Questions	-	-	15
6	Answers To Self Assessment Questions	-	-	15
7	Answers To Terminal Questions	-	-	15-17

1. INTRODUCTION

Sorting is a fundamental operation in computer science used to arrange elements of a list or array in a specific order. Sorting in C is vital for problem-solving, performance optimization, understanding algorithms, laying foundations for advanced topics, real-world applications, and educational purposes, enhancing problem-solving skills, and optimizing performance. In this Unit, we will explore the basics of sorting algorithms like Bubble Sort, Selection Sort, and Insertion Sort, as well as delve into the concept of multi-dimensional arrays, including their declaration, initialization, and memory layout.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Understand fundamental sorting algorithms like Bubble sort, Selection Sort, and Insertion sort*
- ❖ *Comprehend the concept of multi-dimensional arrays and their applications in representing complex data structures.*
- ❖ *Declare and initialize multi-dimensional arrays in C to store and manipulate data efficiently.*

2. SORTING

Sorting in C involves arranging elements of an array or list in a specific order, typically ascending or descending. Common sorting algorithms like Bubble Sort, Selection Sort, and Insertion Sort are implemented to achieve this. Sorting in C enhances problem-solving skills, improves algorithmic understanding, and optimizes performance in various applications. Sorting algorithms can be implemented in C using functions or procedures. These implementations typically involve loops, conditional statements, and array manipulation techniques. It's important to consider factors such as efficiency, stability, and space complexity when choosing a sorting algorithm for a specific task.

Applications: Sorting algorithms find applications in various domains, including:

- Database systems for sorting and indexing records.
- Operating systems for process scheduling and resource allocation.
- Web applications for sorting and displaying data in user interfaces.
- Scientific computing for analyzing and visualizing datasets.

Typical sorting methods are Bubble Sort, Selection Sort, and Insertion Sort.

2.1. Bubble Sort

Bubble Sort is a comparison-based sorting algorithm that iterates through an array multiple times. During each iteration, it compares adjacent elements in the array and swaps them if they are in the wrong order. This process is repeated until the entire array is sorted. The algorithm gets its name from the way smaller elements "bubble up" to their correct positions at the beginning of the array, while larger elements "sink down" to the end. At the end of each iteration, the largest unsorted element is guaranteed to be in its correct position, which reduces the size of the unsorted portion of the array by one element. Here's a step-by-step explanation of the Bubble Sort algorithm:

- Start from the first element (index 0) of the array.
- Compare each element with its adjacent element.
- If the current element is greater than the next element, swap them.

- Continue this process until the end of the array is reached.
- After the first iteration, the largest element will be at the end of the array.
- Repeat steps 1-5 for the remaining elements of the array, excluding the last sorted element.
- Continue these iterations until no more swaps are needed, indicating that the array is sorted.

Implementation:

```
void bubbleSort(int arr[], int n) {  
    int i, j;  
    for (i = 0; i < n-1; i++) {  
        for (j = 0; j < n-i-1; j++) {  
            if (arr[j] > arr[j+1]) {  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
            }  
        }  
    }  
}
```

2.2. Selection Sort

Selection Sort is a comparison-based sorting algorithm that divides the input array into two parts: a sorted subarray and an unsorted subarray. It repeatedly selects the smallest (or largest) element from the unsorted subarray and swaps it with the first element of the unsorted subarray. This process continues until the entire array is sorted.

Here's a simplified explanation of the Selection Sort algorithm in C:

Iterate through the array: Start from the first element and iterate through the array.

Find the minimum element: In each iteration, find the smallest element in the unsorted part of the array.

Swap with the first unsorted element: Swap the minimum element with the first unsorted element.

Move the boundary: Move the boundary of the sorted subarray one element ahead.

Repeat: Repeat steps 2-4 until the entire array is sorted.

Implementation:

```
void selectionSort(int arr[], int n) {  
    int i, j, min_idx;  
    for (i = 0; i < n-1; i++) {  
        min_idx = i;  
        for (j = i+1; j < n; j++) {  
            if (arr[j] < arr[min_idx]) {  
                min_idx = j;  
            }  
        }  
        int temp = arr[min_idx];  
        arr[min_idx] = arr[i];  
        arr[i] = temp;  
    }  
}
```

2.3. Insertion Sort

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time. It iterates through the array, removing one element from the input data, and inserting it into the correct position in the sorted part of the array. This process repeats until the entire array is sorted.

Here's a simplified explanation of the Insertion Sort algorithm in C:

- Iterate through the array: Start from the second element (index 1) and iterate through the array.
- Compare and insert: For each element, compare it with the elements before it in the sorted subarray.
- If the current element is smaller (or larger) than the elements before it, shift those elements to the right (or left) to make space for the current element.
- Insert the current element into the correct position in the sorted subarray.
- Repeat: Repeat steps 2 for each element in the array until the entire array is sorted.

Insertion Sort is efficient for small datasets or nearly sorted arrays but becomes less efficient as the dataset size increases. Despite its inefficiency for large datasets, Insertion Sort is often used in practice for its simplicity and stability.

Implementation:

```
void insertionSort(int arr[], int n) {  
    int i, key, j;  
    for (i = 1; i < n; i++) {  
        key = arr[i];  
        j = i - 1;  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j = j - 1;  
        }  
        arr[j + 1] = key;  
    }  
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. Which sorting algorithm repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order?
 - (a) Bubble Sort
 - (b) Selection Sort
 - (c) Insertion Sort
2. Which sorting algorithm divides the input list into two parts: a sorted part at the beginning and an unsorted part at the end?
 - (a) Bubble Sort
 - (b) Selection Sort
 - (c) Insertion Sort
3. Which sorting algorithm builds the final sorted array one element at a time by repeatedly picking the next element and inserting it into the correct position in the sorted part?
 - (a) Bubble Sort
 - (b) Selection Sort
 - (c) Insertion Sort

3. MULTIDIMENSIONAL ARRAYS

In C, multi-dimensional arrays are used to store data in more than one dimension. They are commonly used to represent matrices, tables, grids, and other structured data. The C language allows arrays of any dimension to be defined.

Declaring and Initializing Multi-dimensional Arrays:

Declaration:

// 2D Array Declaration and Initialization

```
int arr_2d[3][3] = {{0, 1, 2}, {3, 4, 5}, {6, 7, 8}};
```

// 3D Array Declaration and Initialization

```
int arr_3d[2][2][2] = {{{0, 1}, {2, 3}}, {{4, 5}, {6, 7}}};
```

Accessing Elements:

// Accessing elements in a 2D array

```
printf("%d", arr_2d[1][2]); // Outputs: 5
```

// Accessing elements in a 3D array

```
printf("%d", arr_3d[0][1][0]); // Outputs: 2
```

In this section, we will take a look at two-dimensional arrays. One of the most natural applications for a two-dimensional array arises in the case of a matrix. In C, the two-dimensional matrix can be declared as follows:

```
int array[3][6];
```

Following is the way of declaring as well as initializing two-dimensional arrays.

```
int array[3][6] = {
```

```
{4,5,6,7,8,9},
```

```
{1,5,6,8,2,4},
```

```
{0,4,4,3,1,1}
```

```
};
```

Such arrays are accessed like so:

```
array[1][4]= -2;
```

```
if (array[2][1] > 0) {
```

```
printf ("Element [2][1] is %d", array[2][1]);
```

```
}
```

Remember that, like ordinary arrays, two-dimensional arrays are numbered from 0. Therefore, the array above has elements from array[0][0] to array[2][5].

Program: Program to add two matrices

```
#include <stdio.h>
```

```
int main()
```

```
{  
int a[5][5], b[5][5], c[5][5];  
  
int i, j, m, n;  
  
printf("Enter the order of the matrices:");  
  
scanf("%d%d", &m, &n);  
  
printf(" Enter the elements of A matrix:\n");  
for(i=0;i<m;i++)  
for(j=0;j<n;j++)  
scanf("%d", &a[i][j]);  
printf("Enter the elements of B matrix:\n");  
for(i=0;i<m;i++)  
for(j=0;j<n;j++)  
scanf("%d", &b[i][j]);  
/* Add the matrices */  
for(i=0;i<m;i++)  
for(j=0;j<n;j++)  
c[i][j] = a[i][j]+b[i][j];  
/* Print the sum */  
printf("The sum of matrices:\n");  
for(i=0;i<m;i++)  
{  
for(j=0;j<n;j++)  
printf("%d\t", c[i][j]);  
printf("\n");  
}  
}
```

Multidimensional arrays are processed in the same manner as one-dimensional arrays, on an element-by-element basis. However, some care is required when passing multidimensional arrays to a function. In particular, the formal argument declarations within the function definition must include explicit size specifications in all of the subscript

positions except the first. These size specifications must be consistent with the corresponding size specifications in the calling program. The first subscript position may be written as an empty pair of square brackets, as with a one-dimensional array. The corresponding function prototypes must be written in the same manner. But while calling the function the array name may be passed as the actual argument as in the case of one-dimensional arrays. E.g:

```
void process_array (int[][6]); /* function prototype */ void process_array (int  
array[][6])/*function definition */ {  
  
...  
}
```

Memory Layout of Multi-dimensional Arrays:

Description: In memory, multi-dimensional arrays are stored linearly. For example, in a 2D array, the elements of each row are stored in contiguous memory locations.

Example:

[0, 1, 2]

[3, 4, 5]

[6, 7, 8]

In memory, this 2D array might be represented as [0, 1, 2, 3, 4, 5, 6, 7, 8], where each row is stored continuously.

This means that the elements of the first row (0, 1, 2) are stored first, followed by the elements of the second row (3, 4, 5), and then the elements of the third row (6, 7, 8).

The linear storage of multi-dimensional arrays allows for efficient access to elements using pointer arithmetic and indexing. It also enables cache-friendly memory access patterns, which can improve performance in algorithms that traverse multi-dimensional arrays.

SELF-ASSESSMENT QUESTIONS - 2

4. In a two-dimensional matrix, the first subscript in the declaration specifies number of _____.
5. A two-dimensional array is considered as an array of one-dimensional arrays. (True/False)
6. How are multi-dimensional arrays stored in memory?
 - (a) Randomly
 - (b) Linearly
 - (c) Dispersed
7. In a 2D array, where are the elements of each row stored in memory?
 - (a) Non-contiguously
 - (b) Dispersed
 - (c) Contiguously
8. If a 2D array has dimensions 3x3, how many contiguous memory locations are required to store it?
 - (a) 3
 - (b) 6
 - (c) 9

4. SUMMARY

Sorting algorithms like Bubble Sort, Selection Sort, and Insertion Sort are essential techniques for arranging array elements. Each algorithm offers insights into algorithmic principles, varying in implementation complexity and efficiency.

Multi-dimensional arrays, including 2D and 3D arrays, enable the storage of data in structured formats, such as matrices or grids. Understanding their declaration, initialization, and memory layout is crucial. Multi-dimensional arrays store elements linearly in contiguous memory, enhancing access efficiency. Mastery of these concepts is fundamental for effective data organization, algorithmic problem-solving, and memory management in C programming. Proficiency in sorting algorithms and multi-dimensional arrays provides a solid foundation for handling diverse programming tasks and optimizing code performance.

5. TERMINAL QUESTIONS

1. Write a C program that sorts a 1D array of integers using the Insertion Sort algorithm.
2. Discuss the memory layout of multi-dimensional arrays in C.
3. How does Bubble Sort differ from Selection Sort and Insertion Sort

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) Bubble sort
2. (b) Selection sort
3. (c) Insertion sort
4. Rows
5. True
6. (b) Linearly
7. (c) Contiguously
8. (c) 9

7. ANSWERS TO TERMINAL QUESTIONS

1.

```
#include <stdio.h>

// Function to perform Insertion Sort
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        // Move elements of arr[0..i-1], that are greater than key, to one position
        // ahead of their current position
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
```

```
        j = j - 1;
    }
    arr[j + 1] = key;
}
}
```

```
int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Call the insertionSort function to sort the array
    insertionSort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
    return 0;  
}
```

2. (Refer Section 3 for more details)
3. (Refer Section 2 for more details)





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 9

Strings Part 1

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Strings	-	-	4
3	Built-In String Handling Functions	-	1	4-14
4	Summary	-	-	15
5	Terminal Questions	-	-	16
6	Answers To Self Assessment Questions	-	-	16
7	Answers To Terminal Questions	-	-	16
8	Exercises	-	-	16

1. INTRODUCTION

In the preceding unit of your coursework, you delved into the fundamental concepts surrounding arrays within the C programming language. This exploration encompassed an in-depth examination of both one-dimensional and two-dimensional arrays, elucidating their mechanisms and functionalities. As you transition into the current unit, your focus will now shift towards the comprehensive study of strings within the realm of C programming. Strings in C are defined as sequences of characters terminated by a null character ('\0'). Through this module, you will embark on an exploration of the intricacies inherent in manipulating strings, delving into their structure, operations, and practical applications within programming contexts.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Discuss String Initialization Methods*
- ❖ *String Input and Output*
- ❖ *Null Termination*
- ❖ *Built-in string handling functions*

2. STRINGS

Strings in C are represented by arrays of characters. The end of the string is marked with a special character, the null character, which is simply the character with the value 0. (The null character has no relation except in name to the null pointer. In the ASCII character set, the null character is named NULL.) The null or string-terminating character is represented by another character escape sequence, `\0`.

Because C has no built-in facilities for manipulating entire arrays (copying them, comparing them, etc.), it also has very few built-in facilities for manipulating strings.

In fact, C's only truly built-in string-handling is that it allows us to use string constants (also called string literals) in our code. Whenever we write a string, enclosed in double quotes, C automatically creates an array of characters for us, containing that string, terminated by the `\0` character. For example, we can declare and define an array of characters, and initialize it with a string constant:

```
char string[ ] = "Hello, world!";
```

In this case, we can leave out the dimension of the array, since the compiler can compute it for us based on the size of the initializer (14, including the terminating `\0`). This is the only case where the compiler sizes a string array for us, however; in other cases, it will be necessary that we decide how big the arrays and other data structures we use to hold strings are.

3. BUILT-IN STRING HANDLING FUNCTIONS

To do anything else with strings, we must typically call functions. The C library contains a few basic string manipulation functions, and to learn more about strings, we'll be looking at how these functions might be implemented.

Since C never lets us assign entire arrays, we use the `strcpy` function to copy one string to another:

```
#include <string.h>
char string1[ ] = "Hello, world!";
```

```
char string2[20];  
strcpy(string2, string1);
```

The destination string is `strcpy`'s first argument, so that a call to `strcpy` mimics an assignment expression (with the destination on the left-hand side). Notice that we had to allocate `string2` big enough to hold the string

that would be copied to it. Also, at the top of any source file where we're using the standard library's string-handling functions (such as `strcpy`) we must include the line

```
#include <string.h>
```

which contains external declarations for these functions.

Since C won't let us compare entire arrays, either, we must call a function to do that, too. The standard library's `strcmp` function compares two strings, and returns 0 if they are identical, or a negative number if the first string is alphabetically "less than" the second string, or a positive number if the first string is "greater." (Roughly speaking, what it means for one string to be "less than" another is that it would come first in a dictionary or telephone book, although there are a few anomalies.) Here is an example:

```
char string3[] = "this is";  
char string4[] = "a test";  
if(strcmp(string3, string4) == 0)  
    printf("strings are equal\n");  
else  
    printf("strings are different\n");
```

This code fragment will print "strings are different". Notice that `strcmp` does not return a Boolean, true/false, zero/nonzero answer, so it's not a good idea to write something like

```
if(strcmp(string3, string4))
```

...

because it will behave backwards from what you might reasonably expect. (Nevertheless, if you start reading other people's code, you're likely to come across conditionals like `if(strcmp(a, b))` or even `if(!strcmp(a, b))`. The first does something if the strings are unequal; the second does something if they're equal. You can read these more easily if you pretend for a moment that `strcmp`'s name were `strdiff`, instead.)

Another standard library function is `strcat`, which concatenates strings. It does not concatenate two strings together and give you a third, new string; what it really does is append one string onto the end of another. (If it gave you a new string, it would have to allocate memory for it somewhere, and the standard library string functions generally never do that for you automatically.) Here's an example:

```
char string5[20] = "Hello, ";  
char string6[] = "world!";  
printf("%s\n", string5);  
strcat(string5, string6);  
printf("%s\n", string5);
```

The first call to `printf` prints "Hello, ", and the second one prints "Hello, world!", indicating that the contents of `string6` have been tacked on to the end of `string5`. Notice that we declared `string5` with extra space, to make room for the appended characters.

If you have a string and you want to know its length (perhaps so that you can check whether it will fit in some other array you've allocated for it), you can call `strlen`, which returns the length of the string (i.e. the number of characters in it), not including the `\0`:

```
char string7[ ] = "abc";  
int len = strlen(string7);  
printf("%d\n", len);
```

Finally, you can print strings out with `printf` using the `%s` format specifier, as we've been doing in these examples already (e.g. `printf("%s\n", string5);`).

Since a string is just an array of characters, all of the string-handling functions we've just seen can be written quite simply, using no techniques more complicated than the ones we already know. In fact, it's quite instructive to look at how these functions might be implemented. Here is a version of **`strcpy`**:

```
mystrcpy(char dest[ ], char src[ ])
{
    int i = 0;
    while(src[i] != '\0')
    {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
}
```

We've called it `mystrcpy` instead of `strcpy` so that it won't clash with the version that's already in the standard library. Its operation is simple: it looks at characters in the `src` string one at a time, and as long as they're not `\0`, assigns them, one by one, to the corresponding positions in the `dest` string. When it's done, it terminates the `dest` string by appending a `\0`. (After exiting the `while` loop, `i` is guaranteed to have a value one greater than the subscript of the last character in `src`.) For comparison, here's a way of writing the same code, using a `for` loop:

```
for(i = 0; src[i] != '\0'; i++)
    dest[i] = src[i];
dest[i] = '\0';
```

Yet a third possibility is to move the test for the terminating `\0` character out of the for loop header and into the body of the loop, using an explicit if and break statement, so that we can perform the test after the assignment and therefore use the assignment inside the loop to copy the `\0` to dest, too:

```
for(i = 0; ; i++)
{
    dest[i] = src[i];
    if(src[i] == '\0')
        break;
}
```

(There are in fact many, many ways to write `strcpy`. Many programmers like to combine the assignment and test, using an expression like `(dest[i] = src[i]) != '\0'`)

Here is a version of `strcmp`:

```
mystrcmp(char str1[ ], char str2[ ])
{
    int i = 0;
    while(1)
    {
        if(str1[i] != str2[i])
            return str1[i] - str2[i];
        if(str1[i] == '\0' || str2[i] == '\0')
            return 0;
        i++;
    }
}
```

Characters are compared one at a time. If two characters in one position differ, the strings are different, and we are supposed to return a value less than zero if the first string (str1) is alphabetically less than the second string. Since characters in C are represented by their numeric character set values, and since most reasonable character sets assign values to characters in alphabetical order, we can simply subtract the two differing characters from each other: the expression `str1[i] - str2[i]` will yield a negative result if the *i*'th character of str1 is less than the corresponding character in str2. (As it turns out, this will behave a bit strangely when comparing upper and lower-case letters, but it's the traditional approach, which the standard versions of `strcmp` tend to use.) If the characters are the same, we continue around the loop, unless the characters we just compared were (both) `\0`, in which case we've reached the end of both strings, and they were both equal. Notice that we used what may at first appear to be an infinite loop--the controlling expression is the constant 1, which is always true. What actually happens is that the loop runs until one of the two return statements breaks out of it (and the entire function). Note also that when one string is longer than the other, the first test will notice this (because one string will contain a real character at the `[i]` location, while the other will contain `\0`, and these are not equal) and the return value will be computed by subtracting the real character's value from 0, or vice versa. (Thus the shorter string will be treated as "less than" the longer.)

Finally, here is a version of `strlen`:

```
int mystrlen(char str[ ])
{
    int i;
    for(i = 0; str[i] != '\0'; i++)
        {}
    return i;
}
```

In this case, all we have to do is find the `\0` that terminates the string, and it turns out that the three control expressions of the for loop do all the work; there's nothing left to do in the

body. Therefore, we use an empty pair of braces { } as the loop body. Equivalently, we could use a null statement, which is simply a semicolon:

```
for(i = 0; str[i] != '\0'; i++);
```

Everything we've looked at so far has come out of C's standard libraries. As one last example, let's write a substr function, for extracting a substring out of a larger string. We might call it like this:

```
char string8[ ] = "this is a test";  
char string9[10];  
substr(string9, string8, 5, 4);  
printf("%s\n", string9);
```

The idea is that we'll extract a substring of length 4, starting at character 5 (0-based) of string8, and copy the substring to string9. Just as with strcpy, it's our responsibility to declare the destination string (string9) big enough. Here is an implementation of substr. Not surprisingly, it's quite similar to strcpy:

```
substr(char dest[ ], char src[ ], int offset, int len)  
{  
    int i;  
    for(i = 0; i < len && src[offset + i] != '\0'; i++)  
        dest[i] = src[i + offset];  
    dest[i] = '\0';  
}
```

If you compare this code to the code for mystrcpy, you'll see that the only differences are that characters are fetched from src[offset + i] instead of src[i], and that the loop stops when len characters have been copied (or when the src string runs out of characters, whichever comes first).

When working with strings, it's important to keep firmly in mind the differences between characters and strings. We must also occasionally remember the way characters are represented, and about the relation between character values and integers.

As we have had several occasions to mention, a character is represented internally as a small integer, with a value depending on the character set in use. For example, we might find that 'A' had the value 65, that 'a' had the value 97 and that '+' had the value 43. (These are, in fact, the values in the ASCII character set, which most computers use. However, you don't need to learn these values, because the vast majority of the time, you use character constants to refer to characters, and the compiler worries about the values for you. Using character constants in preference to raw numeric values also makes your programs more portable.)

As we may also have mentioned, there is a big difference between a character and a string, even a string which contains only one character (other than the `\0`). For example, 'A' is not the same as "A". To drive home this point, let's illustrate it with a few examples.

If you have a string:

```
char string[ ] = "hello, world!";
```

you can modify its first character by saying

```
string[0] = 'H';
```

(Of course, there's nothing magic about the first character; you can modify any character in the string in this way. Be aware, though, that it is not always safe to modify strings in-place like this) Since you're replacing a character, you want a character constant, 'H'. It would not be right to write

```
string[0] = "H";    /* WRONG */
```

because "H" is a string (an array of characters), not a single character. (The destination of the assignment, `string[0]`, is a `char`, but the right-hand side is a string; these types don't match.)

On the other hand, when you need a string, you must use a string. To print a single newline, you could call

```
printf("\n");
```

It would not be correct to call

```
printf('\n');    /* WRONG */
```

printf always wants a string as its first argument. (As one final example, putchar wants a single character, so putchar('\n') would be correct, and putchar("\n") would be incorrect.)

We must also remember the difference between strings and integers. If we treat the character '1' as an integer, perhaps by saying

```
int i = '1';
```

we will probably not get the value 1 in i; we'll get the value of the character '1' in the machine's character set. (In ASCII, it's 49.) When we do need to find the numeric value of a digit character (or to go the other way, to get the digit character with a particular value) we can make use of the fact that, in any character set used by C, the values for the digit characters, whatever they are, are contiguous. In other words, no matter what values '0' and '1' have, '1' - '0' will be 1 (and, obviously, '0' - '0' will be 0). So, for a variable c holding some digit character, the expression

```
c - '0'
```

gives us its value. (Similarly, for an integer value i, i + '0' gives us the corresponding digit character, as long as $0 \leq i \leq 9$.)

Just as the character '1' is not the integer 1, the string "123" is not the integer 123. When we have a string of digits, we can convert it to the corresponding integer by calling the standard function atoi:

```
char string[] = "123";  
  
int i = atoi(string);  
  
int j = atoi("456");
```

The `atof()` function in C stands for "ASCII to float." It serves to convert a string that represents a floating-point number into its equivalent floating-point value. By accepting a string as input, it facilitates the process of converting textual representations of numbers into their numerical equivalents.

```
char str[] = "3.14";  
  
float num = atof(str);  
  
printf("The floating-point number is: %f\n", num);
```

This will output: The floating-point number is: 3.14.

`itoa()` stands for "integer to ASCII". This function converts an integer to a string representation. It takes three arguments: the integer to be converted, a character array to store the result, and the base of the numerical system to be used (e.g., base 10 for decimal).

```
int num = 123;  
  
char str[10];  
  
itoa(num, str, 10);  
  
printf("The string representation is: %s\n", str);
```

This will output: The string representation is: 123

SELF-ASSESSMENT QUESTIONS - 1

1. Will there be a compilation error for the following program? (Yes/No).

```
char str1[10];  
    str1="Hello, world"; printf("%s", str1);
```
2. The library function used to copy one string to another is _____.
3. The library function atoi can be used for any string. (True/False)
4. What function is used to find the length of a string in C?
 - (a) strlen()
 - (b) strlenlength()
 - (c) length()
 - (d) strcount()
5. What does the strcmp() function return if the two strings are equal?
 - (a) 0
 - (b) 1
 - (c) -1
 - (d) True
6. What function compares two strings in C?
 - (a) strcmpcompare()
 - (b) strcmp()
 - (c) compare()
 - (d) strcoll()

4. SUMMARY

C string handling in C programming revolves around character arrays terminated by a null character ('\0'). Essential string handling functions are provided in the <string.h> header. These include strlen() for determining string length, strcpy() for copying strings, strcat() for concatenation, strcmp() for comparison, strchr() for finding characters, and strtok() for tokenization. Custom implementations may be required for tasks like string reversal (strrev()), case conversion (strlwr(),strupr()), as standard libraries lack these functions. Understanding and utilizing these functions proficiently enables efficient manipulation and processing of strings, fundamental for various applications in C programming.

5. TERMINAL QUESTIONS

1. Define a null-terminated string in C programming and explain its significance.
2. What is the purpose of the null character ('\0') in a null-terminated string?
3. Explain the difference between a null-terminated string and a character array.
4. How is the length of a null-terminated string determined in C?
5. Discuss the importance of null-terminated strings in functions like `strlen()`, `strcpy()`, and `strcmp()`.

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. Compilation error
2. `strcpy`
3. false
4. (a) `strlen()`
5. (a) 0
6. (b) `strcmpare()`

7. ANSWERS TO TERMINAL QUESTIONS

1. Refer section 2
2. Refer section 2
3. Refer section 3
4. Refer section 3
5. Refer section 3

8. EXERCISES

1. Write a program to count the number of vowels and consonants in a given string.
2. Write a program to reverse a string.
3. Write a program check a string is palindrome or not.
4. Write a program to find the length of the given string.



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 10

String Part 2

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
1.1	Objectives	-	-	
2	Operations On Strings	-	-	4-13
2.1	String Searching And Extraction	-	-	
2.2	String Case Conversion	-	-	
2.3	String Modification	-	-	
2.4	String Formatting: Sprintf() And Snprintf(), Scanf() And Sscanf()	-	1	
3	String Manipulation Programs		2	13-18
4	Summary	-	-	19
5	Terminal Questions	-	-	20
6	Answers To Self Assessment Questions	-	-	20
7	Answers To Terminal Questions	-	-	20

1. INTRODUCTION

Strings serve as fundamental components in programming, offering versatile means for handling textual data. In C programming, understanding various string manipulation techniques is pivotal for effective text processing and data analysis. String searching involves locating specific patterns or substrings within a larger string, essential for tasks like data parsing and pattern matching. Case conversion functions like `toupper()` and `tolower()` facilitate altering character cases, ensuring consistency in text processing and simplifying comparison tasks. String modification operations encompass character replacement, whitespace removal, and content insertion, enabling dynamic manipulation of string data. String formatting functions like `sprintf()` and `snprintf()` aid in constructing formatted strings from other data types, while `scanf()` and `sscanf()` facilitate parsing formatted input strings to extract specific data values. These operations are vital for generating output messages, formatting data for display, and parsing input data effectively. Mastering string manipulation techniques in C programming empowers developers to handle textual data efficiently, enhancing the versatility and robustness of their programs. By understanding string searching, case conversion, modification, and formatting, programmers can implement a wide range of functionalities, from basic text processing tasks to complex data parsing and analysis operations.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Understand fundamental string manipulation techniques in C programming for effective text processing and analysis.*
- ❖ *learn case conversion functions (`toupper()` and `tolower()`) to standardize input data and simplify comparison tasks.*
- ❖ *Enhance skills in string modification operations*
- ❖ *Utilize string formatting functions (`sprintf()`, `snprintf()`, `scanf()`, `sscanf()`) for constructing formatted strings and parsing input data accurately.*

2. OPERATIONS ON STRINGS

Operations on strings in C encompass a variety of functionalities vital for manipulating textual data efficiently. These include initialization of strings using character arrays or string literals, input/output operations for reading from and writing to streams, concatenation to merge multiple strings, and length calculation with functions like `strlen()`. String copying is achieved using `strcpy()` or `strncpy()`, while comparison utilizes `strcmp()`. String searching, extraction, and tokenization involve functions such as `strstr()`, `strchr()`, and `strtok()`. Modification operations include character replacement, whitespace removal, and content insertion. Case conversion alters character cases with functions like `toupper()` and `tolower()`. Finally, string formatting constructs formatted strings from other data types using `sprintf()` and `snprintf()`, and parses input strings using `scanf()` and `sscanf()`. These operations equip programmers with essential tools to effectively handle and process textual data in C programming, facilitating tasks ranging from simple text manipulation to complex data parsing and analysis.

2.1. String Searching and Extraction

String searching and extraction in C refers to the process of locating specific patterns or substrings within a given string and extracting relevant information from it. This involves various operations such as searching for substrings, finding individual characters, tokenizing a string into smaller parts, and extracting substrings based on position and length. These operations are essential for tasks like data parsing, pattern matching, and text analysis in C programming. By utilizing functions and techniques provided by the `<string.h>` library, programmers can efficiently search for desired patterns, characters, or substrings within text data and extract relevant information for further processing.

Finding a Substring:

To find a substring within a string, we can use functions like `strstr()` from the standard library. This function searches for the occurrence of a substring within a given string and returns a pointer to the first occurrence if found, or `NULL` if not found.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char str[] = "hello world";  
    char *substr = strstr(str, "world");  
  
    if (substr != NULL) {  
        printf("Substring found at index: %ld\n", substr - str);  
    } else {  
        printf("Substring not found\n");  
    }  
    return 0;  
}
```

This program demonstrates the usage of `strstr()` function to find a substring ("world") within the string "hello world".

Finding Characters in a String:

We can iterate through each character of a string using a loop and check for specific characters using conditional statements like `if`. Alternatively, functions like `strchr()` and `strrchr()` can be used to find the first or last occurrence of a character in a string, respectively.

```
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char str[] = "hello world";  
    char *ch = strchr(str, 'o');  
  
    if (ch != NULL) {
```

```
        printf("Character found at index: %ld\n", ch - str);  
    } else {  
        printf("Character not found\n");  
    }  
  
    return 0;  
}
```

Here, `strchr()` function is used to find the first occurrence of character 'o' in the string "hello world".

Tokenizing a String:

Tokenizing involves breaking a string into smaller parts, or tokens, based on delimiters such as spaces or punctuation marks. The `strtok()` function from the standard library can be used to tokenize a string. It returns a pointer to the next token found in the string and modifies the original string to mark the end of the token.

```
#include <stdio.h>  
  
#include <string.h>  
  
int main() {  
    char str[] = "apple,banana,orange";  
    char *token = strtok(str, ",");  
  
    while (token != NULL) {  
        printf("%s\n", token);  
        token = strtok(NULL, ",");  
    }  
}
```

```
    return 0;
}
```

This program tokenizes the string "apple,banana,orange" using strtok() function and prints each token (separated by comma).

Extracting Substrings Based on Position and Length:

We can extract substrings from a string using array indexing or functions like memcpy() or strncpy(). These functions allow you to copy a specified number of characters from one location in memory to another, effectively extracting the desired substring.

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "hello world";
    char substr[6];

    strncpy(substr, str + 6, 5);
    substr[5] = '\0';

    printf("Substring: %s\n", substr);

    return 0;
}
```

In this example, strncpy() function is used to extract a substring of length 5 starting from index 6 of the string "hello world".

2.2. String Case Conversion

String case conversion in C refers to the process of changing the case of characters within a string. This typically involves converting all characters to uppercase or lowercase. In C programming, you can achieve string case conversion using functions like `toupper()` and `tolower()` provided by the `<ctype.h>` header file.

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

int main() {

    char str[] = "Hello World";

    // Convert all characters to uppercase
    for (int i = 0; i < strlen(str); i++) {

        str[i] = toupper(str[i]);

    }

    printf("Uppercase string: %s\n", str);

    // Convert all characters to lowercase
    for (int i = 0; i < strlen(str); i++) {

        str[i] = tolower(str[i]);

    }

    printf("Lowercase string: %s\n", str);

    return 0;

}
```

This program demonstrates how to convert a string to uppercase and then to lowercase using `toupper()` and `tolower()` functions. These functions take a character as input and return the corresponding uppercase or lowercase equivalent. By iterating through each character of the string, you can apply these functions to perform case conversion.

2.3. String Modification

String modification in C involves altering the content of a string. This can include tasks such as replacing characters, removing whitespace, inserting additional content, or making other changes as required by the program.

```
#include <stdio.h>

#include <string.h>

int main() {
    char str[] = "Hello, World!";

    // Replace a character
    str[7] = 'P'; // Replace 'W' with 'P'

    // Remove whitespace
    int src = 0, dst = 0;
    while (str[src]) {
        if (str[src] != ' ') {
            str[dst++] = str[src];
        }
        src++;
    }
```

```
str[dst] = '\0'; // Null-terminate the modified string

// Insert additional content
strcat(str, " Welcome");

printf("Modified string: %s\n", str);

return 0;
}
```

In this C Program:

We replace the character at index 7 with 'P'.

We remove all whitespace characters from the string.

We insert the string " Welcome" at the end using the strcat() function.

This demonstrates how to perform various string modification operations in C to achieve the desired changes in the string. Depending on the specific requirements, you may need to use different techniques and functions for string modification.

2.4. String Formatting: sprintf() and snprintf(), scanf() and sscanf()

String formatting in C involves constructing formatted strings from other data types and parsing formatted input strings to extract specific data values. This is commonly done using functions like sprintf(), snprintf(), scanf(), and sscanf().

sprintf(): This function formats and stores a series of characters and values in a string. It is similar to printf(), but instead of printing to the standard output, it writes to a string.

snprintf(): Similar to sprintf(), snprintf() formats and stores characters and values in a string, but with additional functionality to specify the maximum number of characters to be written.

scanf(): This function reads formatted input from the standard input stream (usually the keyboard) and assigns the values to variables based on the specified format string.

sscanf(): Similar to scanf(), sscanf() reads formatted input from a string instead of the standard input stream. It parses the input string according to the specified format and assigns the extracted values to variables.

void insertionSort(int arr[], int n)

```
#include <stdio.h>
```

```
int main() {
```

```
    char str[100];
```

```
    int num1 = 10, num2 = 20;
```

```
    // Using sprintf() to format a string
```

```
    sprintf(str, "Sum of %d and %d is %d", num1, num2, num1 + num2);
```

```
    printf("Formatted string: %s\n", str);
```

```
    // Using snprintf() to avoid buffer overflow
```

```
    snprintf(str, sizeof(str), "Sum of %d and %d is %d", num1, num2, num1 + num2);
```

```
    printf("Formatted string: %s\n", str);
```

```
    // Using scanf() to read formatted input
```

```
    int input;
```

```
    printf("Enter a number: ");
```

```
    scanf("%d", &input);
```

```
    printf("You entered: %d\n", input);
```

```
    // Using sscanf() to parse formatted input from a string
```

```
char data[] = "Name: John Age: 25";  
  
char name[20];  
  
int age;  
  
sscanf(data, "Name: %s Age: %d", name, &age);  
  
printf("Name: %s, Age: %d\n", name, age);  
  
return 0;  
}
```

This c program showcases the basic usage of sprintf(), snprintf(), scanf(), and sscanf() for string formatting and parsing in C programming.

SELF-ASSESSMENT QUESTIONS - 1

1. What function is used to read formatted input from a string in C?
 - (a) sscanf()
 - (b) scanf()
 - (c) fscanf()
 - (d) sprint()
2. Which function is used to tokenize a string into smaller strings based on delimiters in C?
 - (a) strsep()
 - (b) strtok()
 - (c) strspn()
 - (d) strcspn()

3. Which function is used to find the first occurrence of a substring within a string in C?
- (e) strstr()
 - (f) strchr()
 - (g) strtok()
 - (h) strcspn()

3. STRING MANIPULATION PROGRAMS

To facilitate string manipulation, C provides a range of functions in the standard library "string.h". These functions offer efficient solutions to common tasks, reducing the complexity and size of programs.

Here are a few commonly used string handling functions:

Operation	Function
String Length Calculation	strlen()
String Concatenation	strcat()
String Copy	strcpy()
String Comparison	strcmp()
Finding Substring	strstr()
Case Conversion	toupper() and tolower() for individual characters, custom implementation for whole string

These string handling functions are declared in the "string.h" header file.

Here are the string manipulation operations implemented in C:

String Length Calculation: To calculate the length of a string in C, you can use the `strlen()` function from the `<string.h>` header.

```
#include <stdio.h>

#include <string.h>

int main() {
    char str[] = "Hello, World!";
    int length = strlen(str);
    printf("Length of the string: %d\n", length);
    return 0;
}
```

String Concatenation: To concatenate two strings in C, you can use the `strcat()` function from the `<string.h>` header. Here is an example:

```
#include <stdio.h>

#include <string.h>

int main() {
    char str1[20] = "Hello";
    char str2[] = ", World!";
    strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
    return 0;
}
```

String Copy: To copy one string to another in C, you can use the `strcpy()` function from the `<string.h>` header.

```
#include <stdio.h>

#include <string.h>

int main() {

    char src[] = "Hello, World!";

    char dest[20];

    strcpy(dest, src);

    printf("Copied string: %s\n", dest);

    return 0;

}
```

String Comparison: To compare two strings in C, you can use the `strcmp()` function from the `<string.h>` header.

```
#include <stdio.h>

#include <string.h>

int main() {

    char str1[] = "hello";

    char str2[] = "world";

    int result = strcmp(str1, str2);

    if (result == 0) {

        printf("Strings are equal\n");

    } else {

        printf("Strings are not equal\n");

    }

    return 0;

}
```

```
}
```

String Reversal:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char str[] = "Hello, World!";  
    int length = strlen(str);  
    for (int i = length - 1; i >= 0; i--) {  
        printf("%c", str[i]);  
    }  
    printf("\n");  
    return 0;  
}
```

The code reverses the string "Hello, World!" by iterating backward through its characters and printing them, resulting in the reversed string "dlroW ,olleH".

Substring Extraction:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {  
    char str[] = "Hello, World!";  
    char sub[20];  
    strncpy(sub, str + 7, 5);  
}
```

```
sub[5] = '\0';  
printf("Substring: %s\n", sub);  
return 0;  
}
```

Finding Substring: To find a substring within a string in C, you can use the `strstr()` function from the `<string.h>` header.

```
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char str[] = "Hello, World!";  
    char *substr = strstr(str, "World");  
    if (substr != NULL) {  
        printf("Substring found at index: %ld\n", substr - str);  
    } else {  
        printf("Substring not found\n");  
    }  
    return 0;  
}
```

Case Conversion: To convert the case of characters in a string in C, we can use the standard library functions `toupper()` and `tolower()`.

```
#include <stdio.h>  
#include <ctype.h>  
  
int main() {
```

```
char str[] = "Hello, World!";  
for (int i = 0; str[i] != '\0'; i++) {  
    str[i] = toupper(str[i]);  
}  
printf("Uppercase string: %s\n", str);  
return 0;  
}
```

These programs demonstrate various string manipulation operations in C.

SELF-ASSESSMENT QUESTIONS – 2

4. Which function is used for calculating the length of a string in C?
 - (a) strlen()
 - (b) lengthstr()
 - (c) strlen()
 - (d) strcount()
5. Which function is used for extracting a substring from a string in C?
 - (a) subextract()
 - (b) strsubstr()
 - (c) substr()
 - (d) strstr()
6. Which functions are used for case conversion of characters in C?
 - (a) toupper() and tolower()
 - (b) convertcase() and casechange()
 - (c) changecase() and casemanipulate()
 - (d) switchcase() and transformcase()

4. SUMMARY

String searching and extraction tasks involve finding substrings within strings, locating specific characters, tokenizing strings based on delimiters, and extracting substrings based on specified positions and lengths. String case conversion functions, including `toupper()` and `tolower()`, enable the transformation of characters to uppercase and lowercase, respectively, facilitating versatile text processing. String modification operations encompass various tasks such as appending, replacing, or removing characters, allowing for flexible string manipulation. String formatting functions like `sprintf()` and `snprintf()` facilitate precise string representation, while `scanf()` and `sscanf()` enable reading formatted input from strings, enhancing input processing capabilities. Additional string operations include determining string lengths without relying on library functions, concatenating strings, copying string contents, comparing strings, reversing string characters, extracting substrings based on specified indices, and finding substrings within strings. Mastering these operations equips C programmers with essential skills for developing efficient and reliable software solutions across diverse domains, enhancing code readability, efficiency, and functionality. Understanding and effectively utilizing these string operations are crucial for developing robust software solutions that meet specific project requirements.

5. TERMINAL QUESTIONS

1. Explain the difference between `strchr()` and `strstr()` functions in C with respect to string searching.
2. Explain the purpose of the `toupper()` and `tolower()` functions in C. Provide an example.
3. Describe the process of modifying a string in C. Provide an example demonstrating a modification operation.

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) `sscanf()`
2. (b) `strtok()`
3. (a) `strstr()`
4. (c) `strlen()`
5. (c) `substr()`
6. (a) `toupper()` and `tolower()`

7. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 2 for more details)
2. (Refer Section 3 for more details)
3. (Refer Section 2 for more details)



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 11

Functions

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Function Basics	-	1	4-12
3	Function Prototypes	-	2	13-16
4	Summary	-	-	17
5	Terminal Questions	-	-	18
6	Answers To Self Assessment Questions	-	-	18
7	Answers To Terminal Questions	-	-	18-19
8	Exercises	-	-	19

1. INTRODUCTION

In the preceding module, we delved into the realm of C arrays. Now, as we transition into this new unit, our focus shifts towards understanding the concept of a function. A function can be likened to a "black box," encapsulating a specific portion of our program. It serves as a compartmentalized entity, embodying certain key properties:

Firstly, a function is designed to execute a well-defined task, serving a specific purpose within the program's logic. This task is intended to be reusable, not only within the current program but potentially across various other programs as well, thus promoting code reusability and modularity. In essence, functions provide a structured approach to organizing and modularizing code, enhancing readability, maintainability, and the overall efficiency of software development.

Functions are probably the most important weapon in our battle against software complexity. You'll want to learn when it's appropriate to break processing out into functions (and also when it's not), and how to set up function interfaces to best achieve the qualities mentioned above: reusability, information hiding, clarity, and maintainability.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *explain the importance of functions*
- ❖ *implement the concepts of formal arguments and actual arguments*
- ❖ *explain function declaration(function prototypes) and function definition*

2. FUNCTION BASICS

A function is a self-contained program segment that carries out some specific, well-defined task. Every C program contains one or more functions. One of these functions must be called `main`. Program execution will always begin by carrying out the instructions in `main`. Additional functions will be subordinate to `main`, and perhaps to one another.

So what defines a function? It has a name that you call it by, and a list of zero or more arguments or parameters. Parameters (also called formal parameters) or arguments are the special identifiers through which information can be passed to the function. A function has a body containing the actual instructions (statements) for carrying out the task the function is supposed to perform; and it may give you back a return value, of a particular type.

In general terms, the first line can be written as

data-type name(data-type parameter 1, data-type parameter 2, ..., data-type parameter n)

Example 11.1: Here is a very simple function, which accepts one argument, multiplies it by 4, and hands that value back.

```
int multbyfour(int x)
{
    int retval;
    retval = x * 4;
    return retval;
}
```

On the first line we see the return type of the function (`int`), the name of the function (`multbyfour`), and a list of the function's arguments, enclosed in parentheses. Each argument has both a name and a type; **`multbyfour`** accepts one argument, of type **`int`**, named `x`. The name `x` is arbitrary, and is used only within the definition of **`multbyfour`**. The caller of this function only needs to know that a single argument of type **`int`** is expected; the caller does

not need to know what name the function will use internally to refer to that argument. (In particular, the caller does not have to pass the value of a variable named *x*.)

Next we see, surrounded by the familiar braces, the body of the function itself. This function consists of one declaration (of a local variable *retval*) and two statements. The first statement is a conventional expression statement, which computes and assigns a value to *retval*, and the second statement is a *return* statement, which causes the function to return to its caller, and also specifies the value which the function returns to its caller.

In general term, a *return* statement is written as

return expression

The *return* statement can return the value of any expression, so we don't really need the local *retval* variable; this function can also be written as

```
int multbyfour(int x)
{
    return x * 4;
}
```

How do we call a function? We've been doing so informally since day one, but now we have a chance to call one that we've written, in full detail. The arguments in the function call are referred to as *actual arguments* or *actual parameters*, in contrast to the formal arguments that appear in the first line of the function definition.

Here is a tiny skeletal program to call *multbyfour*:

```
#include <stdio.h>

extern int multbyfour(int);

int main()
{
    int i, j;

    i = 5;

    j = multbyfour(i);

    printf("%d\n", j);

    return 0;
}
```

This looks much like our other test programs, with the exception of the new line

```
extern int multbyfour(int);
```

This is an *external function prototype declaration*. It is an external declaration, in that it declares something which is defined somewhere else. (We've already seen the defining instance of the function **multbyfour**, but may be the compiler hasn't seen it yet.) The function prototype declaration contains the three pieces of information about the function that a caller needs to know: the function's name, return type, and argument type(s). Since we don't care what name the **multbyfour** function will use to refer to its first argument, we don't need to mention it. (On the other hand, if a function takes several arguments, giving them names in the prototype may make it easier to remember which is which, so names may optionally be used in function prototype declarations.) Finally, to remind us that this is an external declaration and not a defining instance, the prototype is preceded by the keyword **extern**.

The presence of the function prototype declaration lets the compiler know that we intend to call this function, **multbyfour**. The information in the prototype lets the compiler generate the correct code for calling the function, and also enables the compiler to check up on our code (by making sure, for example, that we pass the correct number of arguments to each function we call).

Down in the body of main, the action of the function call should be obvious:

the line

```
j = multbyfour(i);
```

calls **B**, passing it the value of *i* as its argument. When **multbyfour** returns, the return value is assigned to the variable *j*. (Notice that the value of main's local variable *i* will become the value of **multbyfour**'s parameter *x*; this is absolutely not a problem, and is a normal sort of affair.)

This example is written out in "longhand," to make each step equivalent. The variable *i* isn't really needed, since we could just as well call

```
j = multbyfour(5);
```

And the variable *j* isn't really needed, either, since we could just as well call

```
printf("%d\n", multbyfour(5));
```

Here, the call to **multbyfour** is a subexpression which serves as the second argument to **printf**. The value returned by **multbyfour** is passed immediately to **printf**. (Here, as in general, we see the flexibility and generality of expressions in C. An argument passed to a function may be an arbitrarily complex subexpression, and a function call is itself an expression which may be embedded as a subexpression within arbitrarily complicated surrounding expressions.)

We should say a little more about the mechanism by which an argument is passed down from a caller into a function. Formally, C is *call by value*, which means that a function receives *copies* of the values of its arguments. We can illustrate this with an example. Suppose, in our

implementation of **multbyfour**, we had gotten rid of the unnecessary **retval** variable like this:

```
int multbyfour(int x)
{
    x = x * 4;
    return x;
}
```

We might wonder, if we wrote it this way, what would happen to the value of the variable *i* when we called

```
j = multbyfour(i);
```

When our implementation of **multbyfour** changes the value of *x*, does that change the value of *i* up in the caller? The answer is no. *x* receives a copy of *i*'s value, so when we change *x* we don't change *i*.

However, there is an exception to this rule. When the argument you pass to a function is not a single variable, but is rather an array, the function does *not* receive a copy of the array, and it therefore *can* modify the array in the caller. The reason is that it might be too expensive to copy the entire array, and furthermore, it can be useful for the function to write into the caller's array, as a way of handing back more data than would fit in the function's single return value. We will discuss more about passing arrays as arguments to a function later.

There may be several different calls to the same function from various places within a program. The actual arguments may differ from one function call to another. Within each function call, however, the actual arguments must correspond to the formal arguments in the function definition; i.e the actual arguments must match in number and type with the corresponding formal arguments.

Program 11.1: A program to find the largest of three integers

```
#include<stdio.h>

main()
{
    int x, y, z, w;

    /* read the integers */

    int max(int, int);

    printf("\nx= ");
    scanf("%d", &x);
    printf("\ny= ");
    scanf("%d", &y);
    printf("\nz= ");
    scanf("%d", &z);

    /* Calculate and display the maximum value */

    w= max(x,y);
    printf("\n\nmaximum=%d\n", max(z,w));
}

int max(int a, int b)
{
    int c;

    c=(a>=b)?a:b;

    return c;
}
```

Please execute this program and observe the result.

Function calls can span several levels within a program; function A can call function B, which can call function C and so on.

Program 11.2: Program to check whether a given integer is a perfect square or not.

```
#include<stdio.h>

main()
{
    int psquare(int);
    int num;
    printf(" Enter the number:");
    scanf("%d", &num);
    if(psquare(num))    /* main() calls the function psquare() */
    {
        printf("%d is a perfect square\n");
    }
    else
        printf("%d is not a perfect square\n");
}

int psquare(int x)
{
    int positive(int);
    float sqr;
    if(positive(x)) /* psquare() in turn calls the function
    positive() */ {
```

```
        sqr=sqrt(x));  
        if(sqr-int(sqr))==0)  
            return 1;  
        else  
            return 0;  
    }  
    int positive(int m)  
    {  
        if(m>0)  
            return 1;  
        else return 0;  
    }
```

Execute the above program and observe the result.

In the above program the **main** function calls the function `psquare()` and it in turn calls the function `positive()` to check whether the number to be checked for perfect square is a positive or not before checking.

The **return** statement can be absent altogether from a function definition, though this is generally regarded as a poor programming practice. If a function reaches end of the block without encountering a return statement, control simply reverts back to the calling portion of the program without returning any information. Using an empty return statement (without the accompanying expressions) is recommended.

Example 11.2: The following function accepts two integers and determines the larger one, which is then written out. The function doesn't return any information to the calling program.

```
void max(int x, int y)
{
    int m;
    m=(x>=y)?x:y;
    printf(" The larger integer is=%d\n", m);
    return;
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. The function main() is optional in a C program. (True/False)
2. If the function is defined elsewhere (not in the same program where it is called), the function prototype must be preceded by the keyword _____.
3. The arguments that appear in function definition are called _____ arguments whereas the arguments that appear when the function is called are the _____ arguments.
4. The return data type, function name and the list of formal parameters enclosed in brackets are separated by _____.

3. FUNCTION PROTOTYPES

In modern C programming, it is considered good practice to use prototype declarations for all functions that you call. As we mentioned, these prototypes help to ensure that the compiler can generate correct code for calling the functions, as well as allowing the compiler to catch certain mistakes you might make.

In general terms, a function prototype can be written

as *data-type name(type1, type2, ..., type n)* Examples:

`int sample(int, int) or int sample(int a, int b);`

`float fun(int, float) or float fun( int a, float b);`

`void demo(void);` Here **void** indicates function neither return any value to the caller nor it has any arguments.

If you write the function definition after the definition of its caller function, then the prototype is required in the caller, but the prototype is optional if you write the definition of the function before the definition of the caller function. But it is good programming practice to include the function prototype wherever it is defined.

If prototypes are a good idea, and if we're going to get in the habit of writing function prototype declarations for functions we call that we've written (such as `multbyfour`), what happens for library functions such as **printf**? Where are their prototypes? The answer is in that boilerplate line

```
#include <stdio.h>
```

we've been including at the top of all of our programs. `stdio.h` is conceptually a file full of external declarations and other information pertaining to the "Standard I/O" library functions, including `printf`. The `#include` directive arranges all of the declarations within `stdio.h` that are considered by the compiler, rather as if we'd typed them all in ourselves. Somewhere within these declarations is an external function prototype declaration for `printf`, which satisfies the rule that there should be a prototype for each function we call. (For

other standard library functions we call, there will be other “header files” to include.) Finally, one more thing about external function prototype declarations: we’ve said that the distinction between external declarations and defining instances of normal variables hinges on the presence or absence of the keyword `extern`. The situation is a little bit different for functions. The “defining instance” of a function is the function, including its body (that is, the brace-enclosed list of declarations and statements implementing the function). An external declaration of a function, even without the keyword `extern`, looks nothing like a function declaration. Therefore, the keyword `extern` is optional in function prototype declarations.

If you wish, you can write

```
int multbyfour(int);
```

and this is just like an external function prototype declaration as

```
extern int multbyfour(int);
```

(In the first form, without the `extern`, as soon as the compiler sees the semicolon, it knows it’s not going to see a function body, so the declaration can’t be a definition.) You may want to stay in the habit of using `extern` in all external declarations, including function declarations, since “`extern` = external declaration” is an easier rule to remember.

Program 11.3: Program to illustrate that the function prototype is optional in the caller function. The program is to convert a character from lowercase to uppercase.

```
#include<stdio.h>

char lower_to_upper(char ch) /* Function definition precedes main() */
{
    char c;

    c=(ch>='a' && ch<='z') ? ('A'+ch-'a'):ch;

    return c;
}
```

```
main()
{
    char lower, upper;

    /* char lower_to_upper(char lower); */ /* Function prototype is optional
    here*/

    printf("Please enter a lowercase character:");

    scanf("%c", &lower);

    upper=lower_to_upper(lower);

    printf("\nThe uppercase equivalent of %c is %c\n", lower, upper);

}
```

SELF-ASSESSMENT QUESTIONS - 2

5. Function prototype is also called _____.
6. The function prototypes are optional. (True/False)
7. The function prototypes of the library functions is in the corresponding _____ files.
8. The function prototype for the function fun() called in main() below is _____.

```
main()
{
    double x, y, z;
    ...
    z=fun(x, y);
    ...
}
```



4. SUMMARY

A function in programming is akin to an autonomous segment of code dedicated to executing a particular, clearly defined task. When grappling with the complexity of a program, it often stems from a lack of organization, where the code hasn't been appropriately divided into distinct functions. Essentially, a function acts as a self-contained "black box," housing a portion of our program logic. The fundamental concept behind functions lies in their ability to compartmentalize program components, thereby enhancing modularity and maintainability.

The `main()` function holds utmost significance in every C program, serving as the entry point for program execution. Additionally, a function prototype serves as an early declaration of the function, outlining its name, parameters, and return type, facilitating modular design and enabling the compiler to recognize function calls before their actual definitions.

5. TERMINAL QUESTIONS

1. What is the purpose of a function prototype?
2. How do you define a function in C?
3. What is the syntax for calling a function in C?
4. Can a function return a value? If so, how?
5. How are parameters passed to a function in C?
6. What is the difference between formal parameters and actual parameters?

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. False
2. extern
3. formal, actual
4. comma
5. Function declaration
6. False
7. header
8. double fun(double, double);

7. ANSWERS TO TERMINAL QUESTIONS

1. 'void' is the keyword used to specify that the function doesn't return any value. It can also be used to specify the absence of arguments.
2. Function declaration is a direction to the compiler that what type of data is returned by the function, the function name and about the arguments where as the function definition is actually writing the body of the function along with the function header.
3. #include<stdio.h>

```
main()
```

```
{
```

```
int n=10;
```

```
int fun(int n); printf("%d", fun(n));  
  
}
```

```
int fun(int n)  
{  
if(n>0) return (n+fun(n-2));  
}
```

4.

```
gcd(m,n)= {      m or n      if m=n  
              GCD(m, m-n) if m>n  
              GCD(n,m)   if m<n }
```

8. EXERCISES

1. Suppose function F1 calls function F2 within a C program. Does the order of function definitions make any difference? Explain.
2. When a program containing recursive function calls is executed, how are the local variables within the recursive function interpreted?
3. Express the following algebraic formula in a recursive form: $Y = (x_1 + x_2 + \dots + x_n)$
4. Write a function that will allow a floating point number to be raised to an integer power.
5. Write a function to swap two numbers using pass by value technique. What is the drawback of the function?



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 12

Functions Part 2

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3-4
	1.1 Objectives	-	-	
2	Types Of Functions	-	1	5-7
3	Function Returning More Values	-	-	7-10
4	Recursion	-	2	11-16
3	Summary	-	-	17
4	Terminal Questions	-	-	18
5	Answers To Self Assessment Questions	-	-	18
6	Answers To Terminal Questions	-	-	18

1. INTRODUCTION

In C programming, functions serve as fundamental building blocks, offering modularity, reusability, and improved code organization. Among the key concepts related to functions are Recursive Functions, Function Pointers, and Inline Functions.

Recursive Functions are those that call themselves, directly or indirectly. They are pivotal in solving problems that can be broken down into smaller, similar subproblems. Recursive functions follow a repetitive approach, where the function calls itself with modified parameters until reaching a base condition, thereby halting the recursion. This concept finds applications in algorithms like factorial calculation, Fibonacci series generation, and tree traversal.

Function Pointers are pointers that refer to functions rather than data. They provide flexibility by enabling functions to be passed as arguments to other functions or stored in data structures like arrays or linked lists. Function pointers are invaluable in scenarios where the behavior of a function must be determined dynamically at runtime, such as callback functions in event-driven programming or customizable sorting algorithms.

Inline Functions are those that are expanded directly at the call site rather than being invoked as separate entities. This eliminates the overhead associated with function call and return, making inline functions ideal for frequently invoked, small-scale operations. They are commonly employed in performance-critical sections or for small utility functions where the cost of function call overhead is deemed unwarranted.

Additionally, understanding how to handle arrays in function contexts is vital. This includes techniques like Passing and Returning Arrays, which involves passing arrays to functions as arguments, returning arrays from functions, and understanding memory management and the lifetime of returned arrays. Mastery of these techniques enhances a programmer's ability to manipulate array data effectively within C programs. Finally, the concept of Recursion entails functions calling themselves, directly or indirectly. We explore the definition of recursion, its various types, including nested recursion, and the comparison between recursion and iteration. Proficiency in recursion is crucial for efficiently solving complex problems in C programming, as it offers elegant solutions and facilitates code clarity.

By grasping these concepts, programmers can develop modular, efficient, and flexible C code, empowering them to address diverse programming challenges with confidence and proficiency.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Explain various types of functions*
- ❖ *Explain function pointers and explain their purpose in C programming.*
- ❖ *Explain the concept of inline functions and their benefits in C programming.*
- ❖ *Understand techniques for passing arrays to functions as arguments and returning arrays from functions.*
- ❖ *Define recursion and explain its significance in problem-solving.*

2. TYPES OF FUNCTIONS

Functions in C are blocks of code that perform a specific task and can be reused throughout a program. They offer modularity, readability, and ease of maintenance by breaking down a program into smaller, manageable parts. The types of functions in C include:

1. Recursive Functions:

Recursive functions are functions that call themselves either directly or indirectly. They are often used to solve problems that can be broken down into smaller, similar subproblems. In a recursive function, the function calls itself with a modified version of the input until a base case is reached, at which point the recursion stops. Examples of problems that can be solved using recursion include factorial calculation, Fibonacci series generation, and tree traversal algorithms such as depth-first search or binary tree traversal.

2. Function Pointers:

Function pointers are pointers that point to functions instead of data. They allow functions to be treated as first-class citizens in C, meaning they can be passed as arguments to other functions or stored in data structures like arrays or linked lists. Function pointers are commonly used in scenarios where the behavior of a function needs to be determined at runtime. For example, callback functions in event-driven programming or sorting algorithms that accept custom comparison functions use function pointers to achieve flexibility and modularity.

3. Inline Functions:

Inline functions are functions that are expanded in place at the point of call, rather than being called as separate functions. This eliminates the overhead of function call and return, making inline functions efficient for small, frequently called functions. In C, the `inline` keyword is used to declare functions as inline, but it is ultimately up to the compiler to decide whether to honor the request for inlining. Inline functions are commonly used for performance-critical code sections or for small utility functions where the overhead of a function call is not justified.

```
#include <stdio.h>

// Inline function definition

inline int add(int a, int b) {

    return a + b;

}

int main() {

    int num1 = 5, num2 = 10;

    int result;

    // Call to the inline function

    result = add(num1, num2);

    printf("Result: %d\n", result);

    return 0;

}
```

In this program the add function is defined as an inline function using the inline keyword.

When the function is called in the main function, the compiler replaces the function call with the function body at the point of call. Inline functions are typically used for small, frequently called functions where the overhead of a function call is not justified. However, the decision of whether a function is actually expanded inline is ultimately up to the compiler, as it may choose to ignore the inline suggestion in certain cases. It's important to note that excessive use of inline functions can increase code size and may not necessarily improve performance. Inline functions should be used judiciously and selectively for performance-critical code sections.

These types of functions offer different advantages and are used in various programming scenarios to achieve specific goals efficiently. Understanding how and when to use each type of function is essential for writing effective and maintainable C code.

SELF-ASSESSMENT QUESTIONS - 1

1. What does the inline keyword indicate in C?
 - (a) Specifies that the function must be called explicitly
 - (b) Suggests the compiler to expand the function at the point of call
 - (c) Requires the function to be defined within the main function
 - (d) Indicates that the function must be called using function pointers
2. What happens if the function definition is too complex for the compiler to inline?
 - (a) The compiler raises a warning.
 - (b) The function will be inlined regardless.
 - (c) The compiler may ignore the inline suggestion.
 - (d) The compiler raises an error.

3. FUNCTION RETURNING MORE VALUES

In C programming, functions typically return a single value using the return statement. However, there are several techniques for simulating multiple return values from a function. One common approach is to use pointers or arrays to pass output parameters to the function, allowing it to modify the values stored at those memory locations.

Passing and returning arrays: Passing arrays to functions as arguments

Passing arrays to functions as arguments in C involves passing a pointer to the first element of the array along with the size of the array. When you pass an array to a function in C, you are essentially passing the memory address of the first element of the array. The function

can then access and manipulate the elements of the array using pointer arithmetic. Remember that the size of the array is also usually passed as a separate argument to ensure the function knows the length of the array.

```
#include <stdio.h>

// Function prototype to calculate the sum of an array
int calculateSum(int arr[], int size);

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    // Calling the function and passing the array
    int sum = calculateSum(numbers, size);
    printf("Sum of array elements: %d\n", sum);
    return 0;
}

// Function definition to calculate the sum of an array
int calculateSum(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}
```

The program calculates the sum of elements in an array by passing the array and its size to a function. Inside the function, it iterates through the array, accumulating the sum, and returns the result.

Returning arrays from functions

In C, you cannot directly return arrays from functions. However, you can return a pointer to the first element of the array.

```
#include <stdio.h>

// Function to create and return an array
int* createArray(int size) {
    static int arr[5] = {1, 2, 3, 4, 5};
    return arr;
}

int main() {
    int* arr = createArray(5);

    // Accessing elements of the returned array
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

The createArray() function returns a pointer to a statically defined array of integers. In the main function, the returned array is accessed and its elements are printed using a loop.

Memory management and a lifetime of returned arrays:

In C programming, managing memory is a critical aspect to ensure efficient resource utilization and prevent memory leaks, which can degrade program performance and stability. When returning arrays from functions, it's essential to understand the lifetime of dynamically allocated memory. Upon returning an array from a function, whether statically or dynamically allocated, the memory allocated for the array persists until it is explicitly deallocated. For dynamically allocated arrays, this means that the memory remains allocated until it is released using the `free()` function. Failure to deallocate dynamically allocated memory leads to memory leaks, where memory that is no longer needed remains allocated, ultimately consuming system resources.

Proper memory management involves tracking dynamically allocated memory and ensuring that it is released when no longer needed. When returning dynamically allocated arrays from functions, it is the responsibility of the caller to free the memory after using the returned array. This ensures that the memory is returned to the system for reuse, promoting efficient memory utilization.

Additionally, understanding the lifetime of returned arrays helps prevent undefined behavior, which occurs when attempting to access memory that has been freed. Accessing freed memory can lead to unpredictable program behavior, including crashes or incorrect results, making it essential to deallocate memory appropriately.

By adhering to best practices in memory management, such as deallocating dynamically allocated memory after use and understanding the lifetime of returned arrays, programmers can write more robust and reliable C programs. Proper memory management not only improves program stability but also helps optimize resource usage, leading to better overall performance.

4. RECURSION

Recursion is a process by which a function calls itself repeatedly, until some specified condition has been met. The process is used for repetitive computations in which each action is stated in terms of a previous result. Many repetitive problems can be written in this form.

In order to solve a problem recursively, two conditions must be satisfied. First, the problem must be written in a recursive form, and the second, the problem statement must include a stopping condition.

Example Factorial of a number. Suppose we wish to calculate the factorial of a positive integer, n . We would normally express this problem as $n! = 1 \times 2 \times 3 \times \dots \times n$.

This can also be written as $n! = n \times (n-1)!$. This is the recursive statement of the problem in which the desired action (the calculation of $n!$) is expressed in terms of a previous result (the value of $(n-1)!$ which is assumed to be known). Also, we know that $0! = 1$ by definition. This expression provides stopping condition for the recursion.

Thus the recursive definition for finding factorial of positive integer n can be written as:

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n=0 \\ n \times \text{fact}(n-1) & \text{otherwise} \end{cases}$$

Program : Program to find factorial of a given positive integer

```
#include<stdio.h>

main()
{
    int n;

    long int fact(int);

    /* Read in the integer quantity*/
    scanf("%d", &n);
```

```
/*calaculate and display the factorial*/  
printf("n!=%ld\n", fact(n));  
}  
  
long int fact(int n)  
{  
if(n==0)  
return(1);  
else  
return (n*fact(n-1));  
}
```

Please execute this program and observe the result.

Types of Recursion

In C programming, recursion can take different forms, each with its own characteristics and applications. Here are the types of recursion and a comparison with iteration:

Direct Recursion: In direct recursion, a function directly calls itself.

Example:

```
void directRecursion(int n) {  
    if (n > 0) {  
        printf("%d ", n);  
        directRecursion(n - 1); // Recursive call  
    }  
}
```

Indirect Recursion: In indirect recursion, two or more functions call each other in a cycle.

Example:

```
void indirectRecursionA(int n);  
void indirectRecursionB(int n);  
  
void indirectRecursionA(int n) {  
    if (n > 0) {  
        printf("%d ", n);  
        indirectRecursionB(n - 1); // Recursive call to another function  
    }  
}  
  
void indirectRecursionB(int n) {  
    if (n > 1) {  
        printf("%d ", n);  
        indirectRecursionA(n / 2); // Recursive call to another function  
    }  
}
```

Nested Recursion: In nested recursion, a function calls itself with the result of another function call as an argument.

Example:

```
int nestedRecursion(int n) {  
    if (n > 100) {  
        return n - 10;  
    } else {  
        return nestedRecursion(nestedRecursion(n + 11)); // Recursive call with result  
        of another call  
    }  
}
```

Tail Recursion:

Tail recursion occurs when a recursive call is the last operation performed in a function.

Example:

```
void tailRecursion(int n) {  
    if (n > 0) {  
        printf("%d ", n);  
        tailRecursion(n - 1); // Recursive call is the last operation  
    }  
}
```

Recursion vs. Iteration:

Recursion and iteration are both techniques used to achieve repetition in programming.

Recursion is often preferred for solving problems that can be naturally expressed in terms of smaller instances of the same problem. Iteration, using loops like for, while, or do-while, is typically used when the number of repetitions is known or easily determinable. Recursion may result in more concise and elegant code for certain problems, but it may also incur overhead due to function calls and stack usage.

Iteration is generally more efficient in terms of memory usage and performance for simple repetitive tasks.

SELF-ASSESSMENT QUESTIONS - 2

3. When passing an array to a function in C, which of the following methods is used?
 - (a) Pass the entire array as a single argument.
 - (b) Pass the address of the first element of the array.
 - (c) Pass each element of the array separately.
 - (d) Pass the size of the array along with each element.
4. How are arrays typically returned from functions in C?
 - (a) As a single value representing the entire array.
 - (b) As individual elements of the array.
 - (c) As a pointer to the first element of the array.
 - (d) As a string containing the array elements.
5. What is the lifetime of an array returned from a function in C?
 - (a) Until the end of the program execution.
 - (b) Until the function is called again.
 - (c) Until the array is explicitly deallocated using `free()`.
 - (d) Until the function returns control to the caller.
6. Which of the following is true about a base case in recursion?
 - (a) It is optional and can be omitted.
 - (b) It defines the maximum depth of recursion.
 - (c) It is the condition that terminates the recursive calls.
 - (d) It is a function that calls other recursive functions.

7. What is recursion in C?

- (a) A looping construct used to repeat code blocks.
- (b) A function calling itself directly or indirectly.
- (c) An error occurring during program execution.
- (d) A technique for handling runtime errors.



4. SUMMARY

In C programming, recursive functions call themselves, function pointers store addresses of functions, and inline functions suggest expanding code at the call site to reduce overhead. Arrays can be passed to and returned from functions, requiring memory management to avoid leaks. Recursion comes in various types like direct, indirect, nested, and tail recursion, each with unique characteristics. Recursion and iteration offer solutions for repetitive tasks, with recursion calling functions and iteration using loops. Understanding these concepts is crucial for efficient and scalable C programming. In addition, understanding memory management is essential when dealing with returned arrays, as it's the caller's responsibility to deallocate dynamically allocated memory. This ensures efficient resource utilization and prevents memory leaks. Furthermore, grasping the differences between recursion and iteration helps developers choose the most appropriate technique for solving specific problems, optimizing code efficiency and maintainability. Overall, proficiency in these concepts empowers C programmers to write robust, scalable, and efficient software solutions.

5. TERMINAL QUESTIONS

1. What is the primary characteristic of recursive functions?.
2. When is it appropriate to use inline functions?.
3. Can you name two types of recursion commonly used in programming?

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (b) Suggests the compiler to expand the function at the point of call
2. (c) The compiler may ignore the inline suggestion.
3. (b) Pass the address of the first element of the array.
4. (c) As a pointer to the first element of the array. (c)
5. (c) Until the array is explicitly deallocated using free().
6. (c) It is the condition that terminates the recursive calls.
7. (b) A function calling itself directly or indirectly.

7. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 3 for more details)
2. (Refer Section 2 for more details)
3. (Refer Section 3 for more details)



BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 13

Functions Part 3

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3-4
	1.1 Objectives	-	-	
2	Function With Operators	-	1	5-8
3	Function And Decision Statements	-	2	8-11
4	Function And Loop Operators	-	3	11-12
3	Summary	-	-	13
4	Terminal Questions	-	-	14
5	Answers To Self Assessment Questions	-	-	14
6	Answers To Terminal Questions	-	-	14-15

1. INTRODUCTION

In C programming, operators, functions, and decision statements are fundamental concepts that enable developers to write efficient and structured code. Operators are symbols that perform specific operations on operands, such as arithmetic, bitwise, logical, and relational operations. Functions, on the other hand, encapsulate blocks of code that perform specific tasks and can be called from other parts of the program. Functions can interact with operators to manipulate data effectively within a program, providing modularity and code reusability.

While C does not support native operator overloading like C++, functions can emulate operator-like behavior for custom data types, providing similar functionality. Unary operators act on a single operand, and functions can be used to implement custom behaviors for unary operations. Bitwise operators manipulate individual bits of integer operands, and functions can encapsulate bitwise operations to perform tasks such as setting, clearing, or toggling specific bits. Logical operators perform logical operations on boolean values, and functions can incorporate logical operators to make decisions and control program flow based on boolean conditions.

The conditional (ternary) operator provides a compact way to express conditional statements within a single line of code. Functions can utilize the conditional operator to implement conditional expressions efficiently. Moreover, decision statements such as if, else if, and else, allow functions to make choices based on certain conditions, enhancing the flexibility and versatility of function behavior. Additionally, loop operators like for, while, and do-while enable functions to execute a block of code repeatedly, providing the ability to iterate over data and perform calculations or tasks multiple times. Understanding these concepts is essential for writing efficient and structured C programs, empowering developers to manipulate data and control program flow effectively.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Learn how to simulate operator overloading in C by defining functions with operator-like behavior for custom data types.*
- ❖ *Gain proficiency in writing functions that incorporate decision statements to perform different actions based on specific conditions.*
- ❖ *Gain proficiency in implementing functions that utilize loop operators to iterate over data, perform calculations, or execute tasks multiple times.*



2. FUNCTION WITH OPERATORS

Operator overloading is not directly supported in standard C language. Unlike languages like C++, C# or Python, where you can define custom behaviors for operators like +, -, *, etc., based on the types of their operands, C does not provide this feature. In C, operators have fixed meanings determined by the language specification, and they cannot be redefined or overloaded. Therefore, you cannot define custom behaviors for operators when working with user-defined types in C.

However, you can simulate operator overloading in C by defining functions that perform the desired operations and then calling those functions explicitly. For example, instead of overloading the + operator for custom types, you can define a function like add() to perform addition operations on objects of those types. In C programming, operator overloading is not directly supported as it is in languages like C++. However, we can still demonstrate unary operators, bitwise operators, logical operators, and the conditional (ternary) operator in a C program. Here's a program showcasing each of these concepts:

```
#include <stdio.h>

// Unary operator example
void unaryOperatorExample() {
    int x = 10;

    printf("Unary Operator Example:\n");
    printf("Original value of x: %d\n", x);

    // Unary operators
    printf("x after unary increment (++x): %d\n", ++x);
    printf("x after unary decrement (--x): %d\n", --x);
}

// Bitwise operator example
void bitwiseOperatorExample() {
```

```
int a = 10, b = 5;

printf("\nBitwise Operator Example:\n");

printf("a: %d, b: %d\n", a, b);

// Bitwise AND
printf("a & b: %d\n", a & b);

// Bitwise OR
printf("a | b: %d\n", a | b);

// Bitwise XOR
printf("a ^ b: %d\n", a ^ b);

// Bitwise NOT
printf("~a: %d\n", ~a);
}

// Logical operator example
void logicalOperatorExample() {
    int x = 10, y = 5;

    printf("\nLogical Operator Example:\n");

    printf("x: %d, y: %d\n", x, y);

    // Logical AND
    printf("x && y: %d\n", x && y);

    // Logical OR
    printf("x || y: %d\n", x || y);

    // Logical NOT
    printf("!x: %d, !y: %d\n", !x, !y);
}
```

```
// Conditional (Ternary) operator example

void conditionalOperatorExample() {

    int a = 10, b = 5;

    int max = (a > b) ? a : b;

    printf("\nConditional (Ternary) Operator Example:\n");

    printf("a: %d, b: %d\n", a, b);

    printf("Maximum value: %d\n", max);

}

int main() {

    unaryOperatorExample();

    bitwiseOperatorExample();

    logicalOperatorExample();

    conditionalOperatorExample();

    return 0;

}
```

This C program showcases different types of operators:

Unary Operator Example:

Demonstrates pre-increment and pre-decrement operations (++x, --x) on the variable 'x'.

Bitwise Operator Example:

Shows bitwise AND (&), OR (|), XOR (^), and NOT (~) operations between two integers 'a' and 'b'.

Logical Operator Example:

Displays logical AND (&&), OR (||), and NOT (!) operations between two integers 'x' and 'y'.

Conditional (Ternary) Operator Example:

Utilizes the ternary operator (?:) to find the maximum value between 'a' and 'b' and assigns it to 'max'. Finally, the main() function calls each of these examples, printing the results to the consol

SELF-ASSESSMENT QUESTIONS - 1

1. In C programming, which operator is used to perform logical AND between two expressions?
 - (a) &&
 - (b) ||
 - (c) &
 - (d) |
2. Which of the following operators is used to perform logical negation in C?
 - (a) +
 - (b) -
 - (c) !
 - (d) ~

3. FUNCTION AND DECISION STATEMENTS

In C programming, decision statements (such as if, else if, and else) can be used within function bodies to make logical decisions based on input parameters or other variables. Functions can perform different actions or return different results depending on the outcome of decision statements. They provide a way to customize function behavior based on specific conditions, enhancing flexibility and versatility in program execution. Here's a simple example illustrating the usage of decision statements within a function:

Example:

```
#include <stdio.h>
```

```
// Function to determine the maximum of two numbers

int max(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}

int main() {
    int num1 = 10, num2 = 20;
    // Call the max function and print the result
    printf("Maximum number: %d\n", max(num1, num2));
    return 0;
}
```

In this example, the max() function takes two integer parameters a and b and uses an if statement to compare them. If a is greater than b, the function returns a, otherwise, it returns b. The main() function calls max() with two numbers and prints the maximum value. This illustrates how decision statements can be effectively used within functions to perform different actions based on specific conditions.

Example:

```
#include <stdio.h>
```

```
// Function to determine the maximum of three numbers
```

```
int maxOfThree(int a, int b, int c) {
    if (a >= b && a >= c) {
        return a;
    }
}
```

```
    } else if (b >= a && b >= c) {  
        return b;  
    } else {  
        return c;  
    }  
}  
  
int main() {  
    int num1 = 10, num2 = 20, num3 = 15;  
    // Call the maxOfThree function and print the result  
    printf("Maximum number: %d\n", maxOfThree(num1, num2, num3));  
    return 0;  
}
```

In this extended example, the `maxOfThree()` function takes three integer parameters `a`, `b`, and `c` and uses decision statements to determine the maximum among them. It compares each number with the others and returns the maximum value. The `main()` function calls `maxOfThree()` with three different numbers (`num1`, `num2`, `num3`) and prints the maximum value. This demonstrates how decision statements can be used within functions to perform more complex tasks based on specific conditions.

SELF-ASSESSMENT QUESTIONS - 2

3. Functions with decision statements allow for:

- (a) Repeating a block of code
- (b) Customizing function behavior based on specific conditions
- (c) Declaring arrays and structures
- (d) Printing output to the console

4. Which keyword is used to introduce a decision statement in C programming?

- (a) decision
- (b) if
- (c) choose
- (d) pick

4. FUNCTION AND LOOP OPERATORS

"Function and Loop Operators" in C programming involves using loop constructs within functions to perform repetitive tasks efficiently. Functions encapsulate blocks of code that perform specific tasks, and loop operators allow executing a block of code repeatedly based on certain conditions. Here's an example demonstrating the usage of loop operators within a function in C:

```
#include <stdio.h>

// Function to calculate the factorial of a number using a loop
int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
}
```

```
        return result;
    }

    int main() {
        int num = 5;

        // Call the factorial function and print the result
        printf("Factorial of %d is: %d\n", num, factorial(num));

        return 0;
    }
```

In this example, the `factorial()` function takes an integer parameter `n` and uses a for loop to calculate the factorial of `n`. The loop iterates from 1 to `n`, multiplying each integer along the way to compute the factorial. The function then returns the final result.

The `main()` function calls `factorial()` with a value of `num` (in this case, 5) and prints the factorial result. This demonstrates how loop operators, in this case, the for loop, can be effectively used within functions to perform repetitive tasks, such as calculating factorials, in C programming.

SELF-ASSESSMENT QUESTIONS - 3

5. What is the purpose of using loop operators within functions in C programming?

- (a) To define variables
- (b) To perform mathematical operations
- (c) To execute a block of code repeatedly based on certain conditions
- (d) To print output to the console

6. Which loop operator allows executing a block of code at least once, regardless of the loop condition?

- (a) for loop
- (b) while loop
- (c) do-while loop
- (d) break statement

4. SUMMARY

In C programming, operators, functions, and decision statements are foundational concepts that enable efficient and structured code development. Operators are symbols performing specific operations on operands, including arithmetic, bitwise, logical, and relational operations. Functions encapsulate code blocks for specific tasks, enhancing modularity and code reusability. While C lacks native operator overloading like C++, functions can mimic operator-like behavior for custom data types.

Unary operators act on a single operand, and functions can implement custom behaviors for unary operations. Bitwise operators manipulate individual bits of integer operands, with functions encapsulating bitwise operations for tasks like setting or clearing specific bits. Logical operators perform boolean operations, aiding decision-making within functions. The conditional operator offers concise conditional expressions.

Decision statements (if, else if, else) allow functions to make choices based on conditions, enhancing flexibility. Loop operators (for, while, do-while) enable repetitive code execution within functions, facilitating data iteration and task repetition. Understanding these concepts is vital for efficient and structured C programming, empowering developers to manipulate data and control program flow effectively.

5. TERMINAL QUESTIONS

1. Discuss how loop operators enable functions to execute a block of code repeatedly and provide examples demonstrating their practical application..
2. Discuss the concept of functions with arrays in C programming. Explain how functions can accept arrays as arguments and return arrays as results.
3. Provide examples to illustrate how decision statements can be used to make choices based on different conditions within functions.

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) &&
2. (c) !
3. (b) Customizing function behavior based on specific conditions
4. (b) if
5. (c) To execute a block of code repeatedly based on certain conditions
6. (c) do-while loop

7. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 4 for more details)
2. (Refer Section 2 for more details)
3. `#include <stdio.h>`

`// Function to check if a number is even or odd`

```
void checkEvenOrOdd(int num) {  
    if (num % 2 == 0) {  
        printf("%d is even.\n", num);  
    } else {  
        printf("%d is odd.\n", num);  
    }  
}
```

```
int main() {  
    checkEvenOrOdd(5);  
    checkEvenOrOdd(10);  
    return 0;  
}
```





BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 14

Function Part 4

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Function with Arrays	-	1	4-10
3	Summary	-	-	11
4	Terminal Questions	-	-	12
5	Answers To Self Assessment Questions	-	-	12
6	Answers To Terminal Questions	-	-	12

1. INTRODUCTION

Functions with arrays in C programming involve passing arrays to functions as arguments, returning arrays from functions, and implementing algorithms that operate on arrays using functions. Passing arrays to functions allows for modular and reusable code by separating the array manipulation logic from the main program. This enables functions to perform specific tasks on arrays without modifying the original array directly. Returning arrays from functions allows functions to compute and generate arrays dynamically based on certain conditions or calculations. This provides flexibility in program design, as functions can generate arrays based on input parameters or other factors.

Additionally, implementing sorting algorithms like bubble sort, selection sort, and insertion sort using functions enhances code organization and readability. Functions encapsulate the sorting logic, making it easier to understand and maintain. By separating the sorting algorithms into functions, they can be reused across different parts of the program or in other programs.

Overall, functions with arrays enhance code modularity, reusability, and readability in C programming, making it easier to work with arrays and implement complex algorithms efficiently. They play a crucial role in developing robust and maintainable software solutions.

1.1. Objectives:

At studying this unit, you should be able to:

- ❖ *Learn how to pass arrays as arguments to functions.*
- ❖ *Implement algorithms like bubble sort, selection sort, Insertion sort, using functions.*

2. FUNCTION WITH ARRAYS

Functions with arrays in C programming involve passing arrays to functions as arguments and returning arrays from functions. This allows for modular and reusable code by separating the array manipulation logic from the main program.

Passing Arrays to Functions:

Arrays can be passed to functions as arguments. The function can then operate on the array elements without modifying the original array directly.

Syntax: void functionName(dataType arrayName[], int arraySize)

Returning Arrays from Functions:

Arrays can also be returned from functions. The function dynamically allocates memory for the array and returns a pointer to the array.

Syntax: dataType* functionName(dataType arrayName[], int arraySize)

Implementing Algorithms with Functions:

Functions with arrays can implement algorithms such as sorting (e.g., bubble sort, selection sort, insertion sort), searching, or other manipulations.

These algorithms operate on array elements and can be encapsulated in separate functions for better code organization.

Syntax: void bubbleSort(int array[], int arraySize)

Overall, functions with arrays enhance code modularity, reusability, and readability in C programming, making it easier to work with arrays and implement complex algorithms efficiently. They play a crucial role in developing robust and maintainable software solutions.

A C program that demonstrates passing an array to a function, modifying the array within the function, and then returning the modified array:

```
#include <stdio.h>

// Function to double each element of the array
void doubleArray(int array[], int arraySize) {
    for (int i = 0; i < arraySize; i++) {
        array[i] *= 2;
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int arrSize = sizeof(arr) / sizeof(arr[0]);

    // Print the original array
    printf("Original array: ");
    for (int i = 0; i < arrSize; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Pass the array to the function to double its elements
    doubleArray(arr, arrSize);

    // Print the modified array
    printf("Modified array: ");
    for (int i = 0; i < arrSize; i++) {
        printf("%d ", arr[i]);
    }
}
```

```
    }  
    printf("\n");  
    return 0;  
}
```

This program defines a function `doubleArray` that takes an array and its size as arguments and doubles each element of the array. In the main function, an array `arr` is declared and initialized. The original array is printed, then passed to the `doubleArray` function to modify its elements. Finally, the modified array is printed to the console.

Implement algorithms like bubble sort, selection sort, Insertion sort, using functions.

Sorting in C refers to arranging elements in a specified order, usually either ascending or descending. There are various sorting algorithms available in C to accomplish this task, each with its own advantages and disadvantages. Here are some common sorting algorithms implemented in C:

Bubble Sort:

Bubble sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

Selection Sort:

Selection sort divides the input list into two parts: a sorted sublist and an unsorted sublist. It repeatedly selects the minimum element from the unsorted part and swaps it with the first element of the unsorted part.

Insertion Sort:

Insertion sort builds the final sorted list one item at a time by taking each element from the unsorted part and inserting it into its correct position in the sorted part.

This C program demonstrates the implementation of three different sorting algorithms: Bubble Sort, Selection Sort, and Insertion Sort.

```
#include <stdio.h>  
  
// Bubble Sort  
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {
```

```
        if (arr[j] > arr[j + 1]) {  
            // Swap arr[j] and arr[j+1]  
            int temp = arr[j];  
            arr[j] = arr[j + 1];  
            arr[j + 1] = temp;  
        }  
    }  
}  
}
```

// Selection Sort

```
void selectionSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIndex]) {  
                minIndex = j;  
            }  
        }  
        // Swap arr[i] and arr[minIndex]  
        int temp = arr[i];  
        arr[i] = arr[minIndex];  
        arr[minIndex] = temp;  
    }  
}
```

// Insertion Sort

```
void insertionSort(int arr[], int n) {  
    int i, key, j;  
    for (i = 1; i < n; i++) {  
        key = arr[i];
```

```
j = i - 1;

// Move elements of arr[0..i-1], that are greater than key, to one position
ahead of their current position
while (j >= 0 && arr[j] > key) {
    arr[j + 1] = arr[j];
    j = j - 1;
}
arr[j + 1] = key;
}
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
    printArray(arr, n);

    // Bubble Sort
    bubbleSort(arr, n);
    printf("Array sorted using Bubble Sort: ");
    printArray(arr, n);
}
```

```
// Selection Sort
int arr2[] = {64, 25, 12, 22, 11};
selectionSort(arr2, n);
printf("Array sorted using Selection Sort: ");
printArray(arr2, n);

// Insertion Sort
int arr3[] = {64, 25, 12, 22, 11};
insertionSort(arr3, n);
printf("Array sorted using Insertion Sort: ");
printArray(arr3, n);
return 0;
}
```

SELF-ASSESSMENT QUESTIONS - 1

1. Which syntax is used to pass an array to a function in C?
 - (a) void functionName(arrayType arrayName[], int arraySize)
 - (b) void functionName(arrayType *arrayName, int arraySize)
 - (c) void functionName(arrayType arrayName[], arraySize)
 - (d) void functionName(arraySize, arrayType arrayName[])
2. When passing an array to a function, which parameter is used to specify the size of the array?
 - (a) arrayType
 - (b) arrayName
 - (c) arraySize
 - (d) functionType

3. How is an array returned from a function in C?

- (a) By using the return statement with the array elements separated by commas
- (b) By returning a pointer to the array
- (c) By declaring the array as a global variable
- (d) By passing the array as a parameter to the function

4. Which sorting algorithm is efficient for smaller data sets or nearly sorted arrays due to its adaptive nature?

- (a) Merge Sort
- (b) Quick Sort
- (c) Insertion Sort
- (d) Selection Sort



3. SUMMARY

Functions with arrays in C programming enable modular and reusable code by allowing arrays to be passed to functions as arguments and returned from functions. This separation of array manipulation logic from the main program enhances code organization and readability. Additionally, implementing sorting algorithms like bubble sort, selection sort, and insertion sort using functions further enhances code modularity and reusability. By encapsulating the sorting logic within functions, it becomes easier to understand and maintain, and the functions can be reused across different parts of the program or in other programs. Overall, functions with arrays enhance code modularity, reusability, and readability in C programming, making it easier to work with arrays and implement complex algorithms efficiently, thereby contributing to the development of robust and maintainable software solutions.

4. TERMINAL QUESTIONS

1. Implement the insertion sort algorithm in C and explain how it works.
2. Explain the process of sorting an array using the bubble sort algorithm.

5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) void functionName(arrayType arrayName[], int arraySize)
2. (c) arraySize
3. (b) By returning a pointer to the array
4. (c) Insertion Sort

6. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 2 for more details)
2. (Refer Section 2 for more details)